Learning to Build Agentic Systems

An instructional compilation guide built from public university courses

J. A. Arroyo

18 courses · 298 lectures · 97 assignments · 15 modules · 3 learning tracks

CMU 11-768 · Stanford CS146S (2025, 2026) · Northeastern Agentic AI · Harvard CS 2680 · Stanford CS 329Z · Michigan EECS 498 · Buffalo CSE 410/510 · UVA CS 6501 · UW–Madison CS 839 · JHU Machine Programming · CU Boulder CSCI 7000 · Mannheim IE 685/686 · UChicago AI Agents for Social Science

plus MCP, Anthropic Engineering, OpenClaw, Berkeley TinyAgent and Colab

Content checked through November 11, 2026.
All materials belong to their respective authors. This guide only links to them.

Use of AI tools. I am the sole author. Claude (Anthropic, through Claude Code) searched for the courses, read their public pages and drafted the summaries and structure under my direction. The descriptions are summaries, each linked to its source; titles are the sources' own. I am responsible for the content.

Contents

I Orientation 6

How to Use This Guide 7
- Conventions . . . 7
- A note on currency . . . 7
- Caveats . . . 7

Tag Legend 9

The Courses at a Glance 10

Course × Module Navigation Matrix 12

Learning Tracks 13
- Track A: Agent Builder (practitioner) . . . 13
- Track B: Coding-Agent Engineer . . . 13
- Track C: Agent Researcher / Trainer . . . 13
- A 16-Week Self-Study Plan . . . 13

II Topic Modules 15

Module 0: Setup: Your Agent Workbench 16
- Learning objectives . . . 16
- Core concepts . . . 16
- Curated learning sequence . . . 16

Module 1: LLM Foundations and Prompting 17
- Learning objectives . . . 17
- Core concepts . . . 17
- Curated learning sequence . . . 17
- Lecture map (34 lectures) . . . 17
- Key readings . . . 18
- Hands-on . . . 18

Module 2: The Agent Loop and Agent Architectures 19
- Learning objectives . . . 19
- Core concepts . . . 19
- Curated learning sequence . . . 19
- Lecture map (67 lectures) . . . 19
- Key readings . . . 21
- Hands-on . . . 21

Module 3: Tool Use and Function Calling 22
- Learning objectives . . . 22
- Core concepts . . . 22
- Curated learning sequence . . . 22
- Lecture map (24 lectures) . . . 22
- Key readings . . . 23
- Hands-on . . . 23

Module 4: MCP and Agent Interoperability **24**
Learning objectives . . . 24
Core concepts . . . 24
Curated learning sequence . . . 24
Lecture map (13 lectures) . . . 24
Key readings . . . 25
Hands-on . . . 25

Module 5: Memory, Retrieval and Context Engineering **26**
Learning objectives . . . 26
Core concepts . . . 26
Curated learning sequence . . . 26
Lecture map (40 lectures) . . . 26
Key readings . . . 27
Hands-on . . . 27

Module 6: Reasoning, Planning, Reflection and Search **29**
Learning objectives . . . 29
Core concepts . . . 29
Curated learning sequence . . . 29
Lecture map (49 lectures) . . . 29
Key readings . . . 30
Hands-on . . . 31

Module 7: Multi-Agent Systems **32**
Learning objectives . . . 32
Core concepts . . . 32
Curated learning sequence . . . 32
Lecture map (33 lectures) . . . 32
Key readings . . . 33
Hands-on . . . 33

Module 8: Coding Agents **34**
Learning objectives . . . 34
Core concepts . . . 34
Curated learning sequence . . . 34
Lecture map (62 lectures) . . . 34
Key readings . . . 36
Hands-on . . . 36

Module 9: Web, GUI, Computer-Use and Multimodal Agents **37**
Learning objectives . . . 37
Core concepts . . . 37
Curated learning sequence . . . 37
Lecture map (13 lectures) . . . 37
Key readings . . . 37
Hands-on . . . 38

Module 10: Evaluating Agents **39**
Learning objectives . . . 39
Core concepts . . . 39
Curated learning sequence . . . 39
Lecture map (44 lectures) . . . 39
Key readings . . . 40
Hands-on . . . 40

Module 11: Training and Optimizing Agents **42**
Learning objectives . . . 42

Core concepts . . . 42
Curated learning sequence . . . 42
Lecture map (33 lectures) . . . 42
Key readings . . . 43
Hands-on . . . 43

Module 12: Safety, Security and Oversight **44**
Learning objectives . . . 44
Core concepts . . . 44
Curated learning sequence . . . 44
Lecture map (27 lectures) . . . 44
Key readings . . . 45
Hands-on . . . 45

Module 13: Deployment, Efficiency and Operations **46**
Learning objectives . . . 46
Core concepts . . . 46
Curated learning sequence . . . 46
Lecture map (33 lectures) . . . 46
Key readings . . . 47
Hands-on . . . 47

Module 14: Applications: Deep Research, Science and Society **49**
Learning objectives . . . 49
Core concepts . . . 49
Curated learning sequence . . . 49
Lecture map (34 lectures) . . . 49
Key readings . . . 50
Hands-on . . . 50

III Course Catalogue 51

CMU: 11-768 AI Agents **52**
Lectures . . . 52
Assignments and projects . . . 53

Stanford '26: CS146S The Modern Software Developer **55**
Lectures . . . 55
Assignments and projects . . . 56

Stanford '25: CS146S The Modern Software Developer (archive) **57**
Lectures . . . 57
Assignments and projects . . . 58

Michigan: EECS 498-016 Applied Agentic Software Engineering **60**
Lectures . . . 60
Assignments and projects . . . 62

Stanford 329Z: CS 329Z Engineering AI Agents **64**
Lectures . . . 64
Assignments and projects . . . 66

Northeastern: Agentic AI **68**
Lectures . . . 68
Assignments and projects . . . 70

Harvard: CS 2680 Modern AI Systems: Agents and System Optimizations **72**
Lectures . . . 72
Assignments and projects . . . 77

Buffalo: CSE 410/510 Introduction to Agentic Engineering **79**
Lectures . . . 79
Assignments and projects . . . 79

UVA: CS 6501 Workshop on Building AI Agents **81**
Lectures . . . 81
Assignments and projects . . . 82

UW-Madison: CS 839 AI Agents **84**
Lectures . . . 84
Assignments and projects . . . 86

JHU: EN.601.727 Machine Programming **87**
Lectures . . . 87
Assignments and projects . . . 89

CU Boulder: CSCI 7000-005 Building AI Agents **90**
Lectures . . . 90
Assignments and projects . . . 91

Mannheim: IE 685/686 Large Language Models and Agents **92**
Lectures . . . 92
Assignments and projects . . . 92

UChicago: AI Agents for Social Science and Society **94**
Lectures . . . 94
Assignments and projects . . . 95

Stanford 329A: CS329A Self-Improving AI Agents **97**
Lectures . . . 97
Assignments and projects . . . 98

Berkeley '25: CS194/294-196 Agentic AI **99**
Lectures . . . 99
Assignments and projects . . . 100

Berkeley Adv.: CS294/194-280 Advanced Large Language Model Agents **101**
Lectures . . . 101
Assignments and projects . . . 102

Stanford 329T: CS329T Trustworthy ML: Building and Evaluating Agentic Systems **103**
Lectures . . . 103
Assignments and projects . . . 104

Supplementary Resources **105**
Model Context Protocol docs (version 2026-07-28) . . . 105
Anthropic Engineering: Code execution with MCP . . . 105
OpenClaw documentation . . . 105
Berkeley BAIR: TinyAgent, Function Calling at the Edge . . . 105
MIT 6.566 guest lecture: AI Agent Security (Anish Athalye, April 2026) . . . 105
Anthropic: Building effective agent automations (Martin and Avilla, October 2026) . . . 106
Notion: How we built a personal data scientist for every Notion employee (David Thomas, October 2026) 106
Jesse Barkley: local agent demos for the CMU executive Agentic AI program (2026) . . . 106

CADSmith: multi-agent CAD generation with programmatic geometric validation (Barkley, Loghmani and Barati Farimani, 2026) . . . 107
Google Colab . . . 107

IV Appendices **108**

Codebase and Tool Index **109**
CMU . . . 109
Stanford '25 . . . 109
Michigan . . . 109
Stanford 329Z . . . 109
Northeastern . . . 110
Harvard . . . 110
Buffalo . . . 111
UVA . . . 111
UW-Madison . . . 111
JHU . . . 111
CU Boulder . . . 112
Mannheim . . . 112
UChicago . . . 112
Stanford 329A . . . 112
Berkeley '25 . . . 112
Berkeley Adv. . . . 112
Stanford 329T . . . 113
Supplements . . . 113

Orientation

How to Use This Guide

This guide compiles 18 public university courses on LLM agents and agentic systems, plus 10 supplementary resources, into one curriculum you can navigate. Every one of the 298 catalogued lectures and 97 assignments is classified with one shared set of topic tags, so the same subject can be followed across courses.

One set of three documents

1. **Learning to Build Agentic Systems** (this guide): the courses, lectures and assignments behind the practices. *Use it to learn a topic.*
2. **Agentic System Design Process** (`design_process/design_process.pdf`): nine phases from framing a problem to running it in production, each citing the courses indexed here. *Use it to design a system.*
3. **Screening Tasks for Agentic Automation** (`design_process/task_screening.pdf`): a one-page worksheet, filled in after Phase 1 of the design process, that feeds Phase 2 and sets up the comparison run in Phase 7. *Use it to decide whether a task is worth it.*

The modules map onto the design phases: ARCH for Phases 2–3, EVAL for Phases 4 and 7, TOOL and MEM for Phase 5, SAFE for Phase 8 and DEPLOY for Phase 9.

Four ways in

- **Follow a track** (Learning tracks): a role-based path through the modules, or the 16-week self-study plan.
- **Study a topic** (Part II, Modules 0–14): each module gives learning objectives, a short conceptual primer, a *curated* sequence of the best lectures across courses, an automatically generated map of *every* relevant lecture, key readings and hands-on assignments.
- **Browse a course** (Part III, Course catalogue): each course card is followed by its full lecture table and assignment table with all public links.
- **Jump around** with the course × module matrix. Every coloured tag box, lecture reference and assignment name is a hyperlink, and the page header always links back to Contents and the Matrix.

Conventions

- Lecture references look like **CMU 2** (course code and session number in that course's own schedule) and link to the row in the catalogue, which has topics, readings and materials.
- Material links are labelled by type: `[slides]`, `[video]`, `[PDF]`, `[PPTX]`, `[GitHub]`, `[arXiv:...]`, `[notebook zip]`, or the site's domain.
- In module lecture maps, ⋆ marks the lectures chosen for the curated sequence.
- Guest talks stay in the catalogue but are left out of the module maps. Exam, presentation and project-work sessions, and course grading and deadlines, are left out of the guide altogether.
- In hands-on lists, "Starter code" means the assignment has a public repository or download, "Spec" links to a written specification with no code, and "Resources" links to benchmarks or data sources the project builds on.

A note on currency

The material spans Spring 2024 (TinyAgent) to courses still running in November 2026. Two things change fast. (1) **MCP:** the 2026-07-28 revision made the protocol stateless (see the MCP supplement), so earlier lectures teach the older handshake. (2) **Model names and SDKs:** courses use whatever was current when they ran. Learn the concepts from the lectures and take API details from current documentation.

Caveats

- Content was checked through November 11, 2026. The in-progress courses (CMU, Stanford '26, Stanford 329Z, Michigan, Northeastern, Harvard, Buffalo, JHU) will post more material after that.
- Tags were assigned from session titles, listed topics and readings. They are a navigation aid, and some sessions could reasonably be tagged differently.
- Some slide decks could not be read (several Mannheim PDFs, a few UVA PPTX files, JHU lectures 7, 8, 10, 13 and 14), so their topics are inferred from the titles.
- Only some repositories state a licence (UW CS 839, TinyAgent and OpenClaw are MIT). Check each licence

before redistributing course material; linking is always safe.

Tag Legend

Every lecture and assignment carries one or more tags. Each tag (except ORG) corresponds to a module in Part II. TOOL lectures that are mainly about MCP or A2A are collected in their own module.

Tag	Meaning	Module	Lectures
FND	Foundations	Module 1: LLM Foundations and Prompting	35
ARCH	Architectures	Module 2: The Agent Loop and Agent Architectures	68
TOOL	Tools	Module 3: Tool Use and Function Calling; Module 4: MCP and Agent Interoperability	37
MEM	Memory/RAG/Context	Module 5: Memory, Retrieval and Context Engineering	41
RSN	Reasoning/Planning	Module 6: Reasoning, Planning, Reflection and Search	51
MULTI	Multi-agent	Module 7: Multi-Agent Systems	34
CODE	Coding agents	Module 8: Coding Agents	72
WEB	Web/GUI/Multimodal	Module 9: Web, GUI, Computer-Use and Multimodal Agents	13
EVAL	Evaluation	Module 10: Evaluating Agents	47
TRAIN	Training	Module 11: Training and Optimizing Agents	34
SAFE	Safety/Security	Module 12: Safety, Security and Oversight	29
DEPLOY	Deployment	Module 13: Deployment, Efficiency and Operations	39
APP	Applications	Module 14: Applications: Deep Research, Science and Society	41
ORG	Logistics	–	26

The Courses at a Glance

The courses fall into a few archetypes, and knowing which kind you are reading tells you how to use it:

- **Systems and training** (CMU): the backbone, from harnesses to RL, with the best assignments.
- **Builder courses** (Northeastern, UVA, Buffalo, Mannheim, Stanford CS 329Z): hands-on and lab- or homework-driven; Stanford CS 329Z builds each component from scratch before using a framework.
- **Coding-agent practice** (Stanford CS146S, both years): how practitioners work with and around coding agents.
- **Coding-agent internals** (Michigan EECS 498): real harnesses taken apart, then extended.
- **Research seminars** (UW–Madison, CU Boulder): curated paper lists, for depth and for the research frontier.
- **Program synthesis** (JHU): the PL view of code generation, from classic search to LLM coding agents.
- **Applied social science** (UChicago): agents as simulated subjects and research instruments.

Course	Archetype / level	Best for
CMU 11-768 AI Agents	Systems + training (build, evaluate, RL-train) *Graduate; assumes you have trained neural LMs*	The backbone course: harness engineering, evals, SFT/RL, safety, search.
Stanford '26 CS146S The Modern Software Developer	Coding-agent practice (practitioner) *Undergrad/grad; CS111/CS161-level programming*	How production coding agents work and how to set up a repo and a team around them.
Stanford '25 CS146S The Modern Software Developer (archive)	Coding-agent practice (practitioner) *Undergrad/grad*	Slide decks on prompting, building a coding agent, MCP servers, AI testing/security, code review, SRE.
Michigan EECS 498-016 Applied Agentic Software Engineering	Coding-agent internals (use, dissect, extend a harness) *Upper-level undergrad; EECS 281 and 201 (no ML background assumed)*	Taking real coding-agent harnesses (Aider, Codex, Claude Code) apart, then extending one.
Stanford 329Z CS 329Z Engineering AI Agents	Engineering discipline (build from scratch, then frameworks; eval-first) *Prereq CS224N/U/V or CS336, or NLP background*	A harness built with no framework, then authoring and validating a benchmark task.
Northeastern Agentic AI	Builder (LangChain/LangGraph stack) *Gentle on-ramp (Python primer provided)*	Best guided, notebook-by-notebook path from an LLM API call to a deployed LangGraph agent.
Harvard CS 2680 Modern AI Systems: Agents and System Optimizations	Agent systems and serving (use, build, optimize, serve) *Graduate systems course; CS61 plus a graduate systems course*	Treating an agent as a system to measure: harness design, evaluation, and cost per solved task, then the serving stack underneath.
Buffalo CSE 410/510 Introduction to Agentic Engineering	Engineering discipline (framework-agnostic) *Upper-undergrad / MS*	Five lab specs and a capstone spec that treat an agent as an engineered, evaluated, bounded system.
UVA CS 6501 Workshop on Building AI Agents	Hands-on lab workshop (small/open models) *Graduate workshop; Python*	The richest set of runnable lab exercises: local LLMs, LangGraph, tools, RAG, VLMs, MCP/A2A, Tinker fine-tuning.
UW-Madison CS 839 AI Agents	Research seminar (paper reading) *Graduate research seminar*	A curated, critical reading list from ReAct to self-improving and evolutionary agents, plus a no-frameworks coding-agent assignment.
JHU EN.601.727 Machine Programming	Program synthesis to coding agents *Graduate (PL + ML)*	Ties classic program synthesis to LLM coding agents, with two MCP lectures and a strong web-app agent assignment.

Course	Archetype / level	Best for
CU Boulder CSCI 7000-005 Building AI Agents	Research seminar (agentic software engineering) *Graduate seminar; competent SE assumed*	Research reading on agentic software engineering: refactoring agents, SWE-agent, skills, memory, MCP design choices.
Mannheim IE 685/686 Large Language Models and Agents	Applied master's course + team project *Master's; Python + basic ML*	Compact slide decks from LLM basics to context engineering, plus nine well-specified team-project briefs.
UChicago AI Agents for Social Science and Society	Applied: agents as research instruments *Grad / advanced undergrad (computational social science)*	Agents as simulated subjects and research assistants; notebooks on multi-agent simulation, LoRA, RL, steering and red-teaming.
Stanford 329A CS329A Self-Improving AI Agents	Research course (self-improvement levers) + homeworks *Graduate; CS224N or CS229S*	The clearest path through test-time compute, verifiers, RL and open-ended self-improvement, with three public homeworks.
Berkeley '25 CS194/294-196 Agentic AI	Guest-lecture series + AgentBeats project (also a MOOC) *Undergraduate and graduate; ML/DL background*	Industry and lab speakers on post-training, agent evaluation and deployment, and an evaluator-agent project (AgentBeats).
Berkeley Adv. CS294/194-280 Advanced Large Language Model Agents	Guest-lecture series + Lean labs (also a MOOC) *Upper-division undergraduate and graduate*	Research talks on reasoning, post-training, coding and web agents and formal math, plus a compiler-verified agent lab.
Stanford 329T CS329T Trustworthy ML: Building and Evaluating Agentic Systems	Project course on agent evaluation *Graduate; CS229 + deep learning*	Evaluation-first agent building: Goal-Plan-Action evaluation, LLM judges, MCP tool descriptions, GEPA and red-teaming.

Course × Module Navigation Matrix

Each cell is the number of lectures from a course in a module. Click a number to open the module, or a course name to open its catalogue entry. Column headers are module numbers: 1 LLM Foundations and Prompting; 2 The Agent Loop and Agent Architectures; 3 Tool Use and Function Calling; 4 MCP and Agent Interoperability; 5 Memory; 6 Reasoning; 7 Multi-Agent Systems; 8 Coding Agents; 9 Web; 10 Evaluating Agents; 11 Training and Optimizing Agents; 12 Safety; 13 Deployment; 14 Applications.

Course	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	All
CMU	0	3	1	0	3	3	2	2	1	3	4	2	3	3	21
Stanford '26	2	3	1	1	2	0	1	5	0	2	0	0	3	0	19
Stanford '25	2	1	1	1	1	1	2	8	0	2	0	1	2	1	20
Michigan	4	8	3	1	5	1	2	7	1	0	0	3	2	0	17
Stanford 329Z	2	5	0	1	4	1	1	1	0	5	3	2	2	1	19
Northeastern	3	9	2	0	8	4	4	0	1	2	1	1	4	0	18
Harvard	2	6	1	2	3	0	2	3	0	1	0	1	10	0	20
Buffalo	1	2	1	0	3	2	1	1	0	1	0	2	2	1	13
UVA	2	4	3	1	2	4	4	1	2	3	2	1	1	3	16
UW-Madison	1	6	2	0	3	5	5	4	1	5	2	3	0	4	22
JHU	2	4	0	2	0	4	0	13	0	0	4	2	0	7	21
CU Boulder	0	6	5	1	1	4	4	10	1	3	1	2	1	0	19
Mannheim	3	2	0	1	2	1	0	0	0	1	1	1	0	0	9
UChicago	3	0	0	1	1	3	2	1	1	2	2	2	0	5	10
Stanford 329A	1	2	1	0	0	5	0	4	1	3	3	0	0	1	17
Berkeley '25	2	1	0	0	0	1	3	1	1	4	4	1	2	3	13
Berkeley Adv.	0	2	2	0	1	9	0	1	2	2	3	2	1	4	12
Stanford 329T	4	3	1	1	1	1	0	0	1	5	3	1	0	1	12
Total	34	67	24	13	40	49	33	62	13	44	33	27	33	34	

A lecture can appear in several modules. Guest talks and introductory sessions count toward *All* but not toward any module.

Learning Tracks

Choose a track by goal. All tracks assume Python fluency. Track C also assumes basic deep learning (PyTorch, transformers). Each module's curated sequence is the default path; the lecture map lists alternatives.

Track A: Agent Builder (practitioner)

For: You want to ship reliable tool-using agents and RAG systems.

Path: M0 → M1 → M2 → M3 → M4 → M5 → M6 → M10 → M12 → M13

Spine courses: Northeastern notebooks for guided builds, UVA labs for experiments, Buffalo lab specs as checkpoints.

Capstone: Buffalo's *useful bounded agent* (**Buffalo: Final Team Project: Useful Bounded Agent**), deployed like Northeastern's **Northeastern: DADA1: The Trip Planner Agent**.

Track B: Coding-Agent Engineer

For: You want to build or deeply customize coding agents and agent-ready workflows.

Path: M0 → M2 → M3 → M4 → M5 → M8 → M10 → M12

Spine courses: Stanford '26 for practice, CMU and Michigan for harness internals, UW and CU Boulder for the research context.

Capstone: **UW-Madison: Build a Coding Agent** without frameworks, then extend it to pass the **JHU: A3: Coding LLM Agents** tasks and evaluate it like **JHU: A2: Evaluating Coding LLMs**. An adapted alternative: **Harvard: A2 Mads-Loop: Build the loop**, a harness scored on SWE-bench Pro tasks plus four tasks you write yourself, run against your own model endpoint because the course API is closed to outsiders.

Track C: Agent Researcher / Trainer

For: You want to train agentic models and do research-grade evaluation.

Path: M1 → M2 → M3 → M5 → M6 → M10 → M11 → M7 → M12 → M14

Spine courses: CMU 11-768 end to end (all three assignments), with UW CS 839 and CU Boulder reading lists; Stanford CS329A for test-time compute and self-improvement, and the two Berkeley series for current research talks.

Capstone: The CMU pipeline: **CMU: A1: Build an Agent Harness** → **CMU: A2: Evaluating a Data-Visualization Agent** → **CMU: A3: Agentic RL Training, "scottyrl"**, then a research question framed as Northeastern requires (mechanism → manipulation → measured behavior).

A 16-Week Self-Study Plan

Roughly 8–10 hours a week: watch or read the listed sessions (slides first, then papers), then build. The plan follows Track A but includes the core of B and C. Adjust it freely.

Wk	Module	Watch / read	Build
1	SETUP, FND	Northeastern 2, Northeastern 3, Stanford '25 2	**UVA: Topic 1 portfolio: Running an LLM**; **Stanford '25: Week 1 — Prompting Techniques**
2	ARCH	CMU 1, Stanford '26 1, UW-Madison 2	Raw ReAct loop from scratch (start **UW-Madison: Build a Coding Agent**)
3	TOOL	CMU 2, UVA 5, UVA 7	**UVA: Topic 3 portfolio: Tool use**; **Buffalo: Lab 1: A Minimal Tool-Using Agent**
4	MCP	MCP docs, Stanford '26 4, JHU 8–JHU 9	**Stanford '26: Week 2 — Build an MCP Server for a Third-Party API**
5	MEM (context)	CMU 3, CMU 4, Northeastern 7	Add compaction and skills to your agent (**Stanford '26: Week 3 — Author a Skill**)

Wk	Module	Watch / read	Build
6	MEM (RAG)	Northeastern 8, Northeastern 9, UVA 9	**UVA: Topic 5: RAG experiments**; **Buffalo: Lab 2: Retrieval and Memory**
7	ARCH (graphs)	Northeastern 6, UVA 3, CMU 15	**Buffalo: Lab 3: Workflow Graphs and State Machines**; **Northeastern: DADA1: The Trip Planner Agent**
8	RSN	CMU 5, Northeastern 10, Stanford 329A 2; before A1: JHU 2–JHU 4	**JHU: A1: Inductive Synthesis**
9	CODE	CMU 6, Stanford '26 2, UW-Madison 5–UW-Madison 6	**CMU: A1: Build an Agent Harness** or **JHU: A3: Coding LLM Agents**
10	EVAL	Northeastern 18, Buffalo 9, Stanford 329T 7, Berkeley '25 5	**CMU: A2: Evaluating a Data-Visualization Agent** or **JHU: A2: Evaluating Coding LLMs** (pick one; each fills the week. Do the other later as a multi-week project)
11	MULTI	Northeastern 12–Northeastern 14, UW-Madison 20	**UVA: Topic 7: MCP + A2A** (A2A)
12	WEB	CMU 7, UVA 10	**UVA: Topic 6: VLM agents**
13	TRAIN (SFT)	CMU 8, UVA 13, TinyAgent	**UVA: Topic 8: Text-to-SQL fine-tuning**
14	TRAIN (RL)	CMU 9, CMU 11, CMU 12, Stanford 329A 6	**CMU: A3: Agentic RL Training, "scottyrl"**
15	SAFE	CMU 13, UW-Madison 11–UW-Madison 12, Buffalo 10, Berkeley '25 13	**Buffalo: Lab 5: Human-in-the-Loop and Safety Boundaries**
16	DEPLOY, APP	Northeastern 19, Stanford '26 15, CMU 10	Capstone (**Buffalo: Final Team Project: Useful Bounded Agent**)

Topic Modules

Module 0: Setup: Your Agent Workbench

Learning objectives

- Have a reproducible Python environment, API access and a sandbox to run course code in.
- Know which courses need which accounts and services.

Core concepts

Every course here assumes a small workbench, and setting it up once saves friction later.

- **Python environment.** CMU and Northeastern use `uv`; Stanford '25 used Poetry/Conda. Keep one virtual environment per course repository.
- **Models.** You need at least one hosted API key (the courses use OpenAI, Anthropic via Bedrock, Gemini and OpenRouter) and, optionally, a local runtime (Ollama is used by Northeastern, UVA and Stanford '25) for cheap, private experiments with small open models.
- **Sandboxes.** Agents execute code. CMU runs tools in Modal sandboxes. Colab is the zero-install choice for notebooks. Never let an agent you are developing run commands directly on the machine that holds your credentials.
- **Tracing.** Log every model call and tool call from day one (plain JSONL is enough). Every evaluation and debugging lecture assumes you have traces.

Install policy reminder. The courses name many packages (LangGraph, FastMCP, Tinker, Modal, ...). Treat each install as a deliberate decision: pin versions and check security advisories before installing.

Curated learning sequence

1. Northeastern 3 *LLM Foundations (II)* — The setup page walks through uv, `.env` keys and a first LLM API call.
2. UVA 1 *Topic 1: Running an LLM* — Run a small open model locally and on Colab, with and without quantization.
3. Supplement: Google Colab — The runtime for every notebook-based course.

Module 1: LLM Foundations and Prompting

Tag: FND

Learning objectives

- Explain next-token prediction, pre-training versus post-training, and why instruction-tuned models follow chat templates.
- Use prompting techniques (few-shot, CoT, self-consistency, structured output) deliberately.
- Run and benchmark a small open model locally.

Core concepts

An agent is only as good as the model in its loop, and most agent bugs trace back to how that model was conditioned.

- **Pre-training** produces a next-token predictor. **Post-training** (instruction tuning, RLHF/RL) turns it into an assistant that follows *chat templates*: role-tagged messages, and for agents, tool-call and tool-result segments. CMU's tool-use lecture shows how those templates encode function calls.
- **Prompting** is programming in natural language. Few-shot examples, chain-of-thought, self-consistency and explicit output schemas are the basic toolkit. Structured output (JSON or typed objects) is what makes a model's response machine-actionable.
- **Cost and latency** scale with tokens, so an agent's prompt is a budget. Context-window limits and KV-cache pricing return in Module 5 (Memory, Retrieval and Context Engineering) and Module 13 (Deployment, Efficiency and Operations).
- **Small versus large models.** UVA's first reading argues that small language models are the future of agentic AI, and the TinyAgent supplement shows a 1.1B model beating a frontier model on a narrow toolset.

Curated learning sequence

1. Northeastern 2 *LLM Foundations (I)* — Transformers, autoregressive generation, base versus instruction-tuned models (with a local model).
2. Northeastern 3 *LLM Foundations (II)* — InstructGPT, CoT, and the LLM API basics notebook.
3. Stanford '25 1 *Introduction and how an LLM is made* and Stanford '25 2 *Power prompting for LLMs* — How an LLM is made, then power prompting (slides).
4. Mannheim 1 *Introduction (to LLMs)*–Mannheim 3 — Compact decks on LLM basics, post-training and prompt engineering.
5. UVA 1 *Topic 1: Running an LLM* — Benchmark Llama-3.2-1B on MMLU across hardware and quantization settings.

Lecture map (34 lectures)

Course	#	Title (links to catalogue row)	Materials	Tags
Stanford '26	2	How SOTA coding agents are designed: deep dive into system prompts	[course site]	CODE FND
Stanford '26	3	Advanced prompting + agentic dev frameworks (RePPIT, spec-driven development)	[course site]	FND MEM
Stanford '25	1	⋆Introduction and how an LLM is made	[slides]	ORG FND
Stanford '25	2	⋆Power prompting for LLMs	[slides]	FND RSN
Michigan	2	How LLMs Code (Under the Hood)	[slides] [podcast]	FND
Michigan	3	The Big Three: Context, Model, Prompt	[slides] [demo] [podcast]	CODE FND
Michigan	7	Inside the Call: Messages, Streams, and Tool Calls	[slides]	TOOL FND ARCH
Michigan	18	Many Models, One Harness	[slides]	FND ARCH
Stanford 329Z	1	Introduction — What Are Agentic Systems?	[slides] [video]	FND ARCH
Stanford 329Z	2	LLMs for Builders	[slides] [video]	FND MEM

Course	#	Title (links to catalogue row)	Materials	Tags
Northeastern	2	⋆LLM Foundations (I)	[agenticllms.github.io] [agenticllms.github.io] [video] [video]	FND DEPLOY
Northeastern	3	⋆LLM Foundations (II)	[agenticllms.github.io] [GitHub]	FND TRAIN
Northeastern	19	Agentic AI Optimization	–	DEPLOY FND
Harvard	12	Efficient LLM serving basics I: intro to LLMs	[PDF] [cs2680.seas.harvard.edu]	FND DEPLOY
Harvard	13	Efficient LLM serving basics II: intro to GPUs	[PDF] [cs2680.seas.harvard.edu]	DEPLOY FND
Buffalo	2	Foundation models as components	–	FND
UVA	1	⋆Topic 1: Running an LLM	[cs.virginia.edu]	FND EVAL DEPLOY
UVA	10	Topic 6: Vision-Language Models	[cs.virginia.edu] [GitHub]	WEB APP FND
UW-Madison	3	Reasoning and acting I	–	RSN FND
JHU	5	Language Modeling for Synthesis	[PDF]	FND CODE
JHU	6	Decoding and Prompting for Synthesis	[PDF]	FND CODE
Mannheim	1	⋆Introduction (to LLMs)	[PDF]	FND ORG
Mannheim	2	⋆Post-training and Efficient Adaptation	[PDF] [notebook zip]	TRAIN FND
Mannheim	3	⋆Prompt Engineering	[PDF] [notebook zip]	FND RSN
UChicago	1	Deep Learning and Social Agents	[GitHub]	FND APP
UChicago	2	Text Learning, Transformers, and Diffusion Models	[GitHub]	FND RSN
UChicago	6	Auto-Encoders, Interpretability, and Steering Agents	[GitHub]	SAFE FND
Stanford 329A	20	Future Research Areas	[video]	FND
Berkeley '25	1	Introduction	[slides]	ORG FND
Berkeley '25	2	LLM Agents Overview	[slides] [video]	FND TRAIN
Stanford 329T	1	Course introduction and overview	[slides]	ORG FND ARCH
Stanford 329T	2	Models, Prompting and RAG	[slides] [HW1]	FND MEM
Stanford 329T	9	Deep dive into LLMs	[slides]	FND TRAIN
Stanford 329T	15	Multi-modal frontier models with agentic capabilities (guest: Deepak Ramachandran, Google DeepMind)	–	FND ARCH

Key readings

- Vaswani et al., *Attention Is All You Need* (2017)
- Ouyang et al., *InstructGPT* (2022)
- Wei et al., *Chain-of-Thought Prompting* (2022)
- Belcak et al., *Small Language Models are the Future of Agentic AI* (2025)
- Karpathy, *Deep Dive into LLMs* (video)

Hands-on

- **Stanford '25: Week 1 — Prompting Techniques** — Fill in prompts for K-shot, chain-of-thought, tool calling, self-consistency, RAG, and reflexion until the test scripts pass, using local Ollama models (mistral-nemo:12b, llama3.1:8b). *Starter code:* [GitHub]
- **UVA: Topic 1 portfolio: Running an LLM** — MMLU timing/accuracy study across models, quantization and hardware; chat agent with bounded context and restartability; optional MT-Bench. *Starter code:* [cs.virginia.edu]
- **Mannheim: Exercise: LLM Introduction** — Intro notebook for working with LLMs. *Starter code:* [notebook zip]

Module 2: The Agent Loop and Agent Architectures

Tag: ARCH

Learning objectives

- Define an agent (model + loop + tools + environment + stopping rule) and distinguish workflows from agents.
- Implement a ReAct-style loop from scratch, without a framework.
- Judge when a graph framework (LangGraph) or an agent SDK earns its complexity.

Core concepts

Anthropic's *Building Effective Agents*, assigned by Northeastern, Buffalo and UW, draws the key distinction. A **workflow** orchestrates LLM calls along predefined code paths (chaining, routing, parallelization, orchestrator–workers, evaluator–optimizer). An **agent** lets the model choose its own next action in a loop until a stopping condition holds.

- **The loop.** Build the prompt (system + history + tool specs), call the model, parse any tool calls, execute them in the environment, append the observations, and repeat until the model answers or a step or token budget runs out. That is a *harness*. CMU A1, JHU A3 and the UW assignment all make you write one.
- **State** is the message history plus anything external (files, a scratchpad, a database). Explicit state machines and graphs (Buffalo Lab 3, LangGraph in Northeastern, UVA and CMU) make control flow inspectable, resumable (checkpointing) and testable.
- **Frameworks versus raw loops.** Write a loop by hand first. Frameworks such as LangGraph, the OpenAI Agents SDK, OpenHands and smolagents then become shortcuts you understand rather than magic. UW bans frameworks for its assignment for exactly this reason.
- **Cognitive architectures** (CoALA) provide a vocabulary for memory, action space and decision procedure that the rest of this guide reuses.

Curated learning sequence

1. CMU 1 *Course Overview: What Is an Agent?* — What an agent is, the agent loop and the course map (slides + video).
2. Stanford '26 1 *Course intro + build Claude Code in 200 lines* — Build a Claude-Code-style agent in about 200 lines, then read the completed code.
3. UW-Madison 2 *Building agents* — Instructor walkthrough with a demo notebook. Then do the UW assignment below.
4. Northeastern 4 *Simple Agent Architecture* and Northeastern 6 *Short-term & Long-term Memory* — From a simple tools-and-memory agent to LangGraph state, nodes, edges and cycles.
5. UVA 3 *Topic 2: Agent Frameworks* — LangGraph lab: routing, parallel models, checkpointing and crash recovery.
6. CMU 14 *Frameworks 1: OpenHands* and CMU 15 *Frameworks 2: Orchestration and Monitoring* — Two production frameworks compared: OpenHands and LangGraph.
7. Northeastern 16 *Deep Agents I: The Agent Harness* — A long-running harness built by hand (a to-do list, files in graph state, a subagent task tool), then the same agent with the `deepagents` library.
8. Supplement: Barkley local agent demos — The same small agent as a hand-written loop and as a LangGraph graph, on a local 4B model.
9. Stanford 329Z 1 *Introduction — What Are Agentic Systems?* and Stanford 329Z 6 *Agent Design Patterns & Scaffolds* — From LLM pipelines to compound systems to agents, then design patterns and scaffolds (ReAct, Reflexion, CodeAct).
10. Stanford 329Z 5 *Frameworks & Orchestration* — Components built from scratch, then the same ones as DSPy and LangGraph abstractions (public notebook).

Lecture map (67 lectures)

Course	#	Title (links to catalogue row)	Materials	Tags
CMU	1	⋆Course Overview: What Is an Agent?	[slides] [video]	ORG ARCH
CMU	14	⋆Frameworks 1: OpenHands	[slides] [video]	ARCH CODE

Course	#	Title (links to catalogue row)	Materials	Tags
CMU	15	⋆Frameworks 2: Orchestration and Monitoring	[slides]	ARCH DEPLOY
Stanford '26	1	⋆Course intro + build Claude Code in 200 lines	[course site]	CODE ARCH
Stanford '26	15	Background agents: launching tasks asynchronously	[course site]	DEPLOY ARCH
Stanford '26	20	The Software Factory: self-running, self-improving software systems	[course site]	ARCH DEPLOY
Stanford '25	3	Building a coding agent from scratch	[slides] [drive.google.com]	ARCH TOOL CODE
Michigan	5	Modern agentic CLIs	[slides] [podcast]	CODE ARCH SAFE
Michigan	6	From pair-coding to agent orchestration	[slides]	CODE TOOL ARCH
Michigan	7	Inside the Call: Messages, Streams, and Tool Calls	[slides]	TOOL FND ARCH
Michigan	9	Stop conditions and the approval layer	[slides]	ARCH SAFE
Michigan	11	Guardrails: Errors, Retries and Approval Gates	[slides]	SAFE ARCH
Michigan	14	Subagents	[slides]	MULTI ARCH MEM
Michigan	15	Compaction and Prompt Caching	[slides]	MEM ARCH
Michigan	18	Many Models, One Harness	[slides]	FND ARCH
Stanford 329Z	1	⋆Introduction — What Are Agentic Systems?	[slides] [video]	FND ARCH
Stanford 329Z	5	⋆Frameworks & Orchestration	[slides] [code]	ARCH DEPLOY
Stanford 329Z	6	⋆Agent Design Patterns & Scaffolds	[slides]	ARCH RSN MEM
Stanford 329Z	18	Coding & Software Agents	–	CODE ARCH
Stanford 329Z	19	Proactive Agents	–	ARCH APP SAFE
Northeastern	1	Introduction to Agentic AI	[agenticllms.github.io] [colab.research.google.com]	ORG ARCH
Northeastern	4	⋆Simple Agent Architecture	[agenticllms.github.io] [GitHub]	ARCH TOOL MEM
Northeastern	6	⋆Short-term & Long-term Memory	[agenticllms.github.io] [agenticllms.github.io] [GitHub]	MEM ARCH DEPLOY
Northeastern	7	Context Management	[agenticllms.github.io] [GitHub]	MEM ARCH
Northeastern	10	Planning & Task Decomposition	[agenticllms.github.io] [GitHub]	RSN ARCH
Northeastern	12	Multi-Agent Orchestration I	[agenticllms.github.io] [GitHub]	MULTI ARCH WEB
Northeastern	14	Multi-Agent Orchestration III	[agenticllms.github.io] [GitHub]	MULTI ARCH
Northeastern	16	⋆Deep Agents I: The Agent Harness	[agenticllms.github.io] [GitHub]	ARCH MULTI MEM
Northeastern	17	Deep Agents II: Backends and Sandboxes	[agenticllms.github.io] [GitHub]	SAFE ARCH MEM
Harvard	1	Course overview and logistics	[PDF]	ORG ARCH
Harvard	4	Agents design I: context management and tool design	[PDF] [cs2680.seas.harvard.edu]	ARCH TOOL MEM
Harvard	6	Agents design II: multi-agent architecture, coordination, and communication	[PDF] [cs2680.seas.harvard.edu]	MULTI ARCH TOOL
Harvard	7	Agents design III: agent memory and reliability	[PDF] [cs2680.seas.harvard.edu]	MEM ARCH
Harvard	10	Advanced agent design I: Cost and performance	[PDF] [PDF] [cs2680.seas.harvard.edu]	DEPLOY ARCH
Harvard	11	Advanced agent design II: agent safety, self-evolving harness	[cs2680.seas.harvard.edu] [cs2680.seas.harvard.edu]	SAFE ARCH
Buffalo	1	What is agentic engineering?	–	ORG ARCH
Buffalo	6	Agent workflows	–	ARCH
UVA	3	⋆Topic 2: Agent Frameworks	[cs.virginia.edu]	ARCH MULTI
UVA	5	Topic 3: Agent Tool Use (online)	[cs.virginia.edu]	TOOL ARCH
UVA	7	Topic 4: Exploring Tools	[cs.virginia.edu] [GitHub]	TOOL RSN ARCH
UVA	17	Topic 11: Coding Agents	[cs.virginia.edu]	CODE ARCH
UW-Madison	2	⋆Building agents	[GitHub]	ARCH TOOL
UW-Madison	13	Optimizing agents I	–	ARCH TRAIN
UW-Madison	14	Optimizing agents II	–	ARCH TRAIN
UW-Madison	15	Self-improving agents I	–	ARCH WEB
UW-Madison	20	Multi-agent systems II	–	MULTI ARCH
UW-Madison	21	Memory I	–	MEM ARCH
JHU	7	Iterative Refinement and Agentic Framework	[PDF]	ARCH CODE
JHU	8	Agentic Frameworks: Model Context Protocol	[PDF]	TOOL ARCH
JHU	9	Agentic Frameworks: Model Context Protocol (2)	[PDF]	TOOL ARCH
JHU	10	Agentic Frameworks: Selected Topics	[PDF]	ARCH CODE

Course	#	Title (links to catalogue row)	Materials	Tags
CU Boulder	2	Survey of agents (Dig)	[PDF]	ARCH CODE
CU Boulder	3	Agentic SE roadmap (Dig)	[PDF]	CODE ARCH
CU Boulder	13	SWE-agent & AutoCodeRover (students)	–	CODE ARCH
CU Boulder	14	AutoGen & MetaGPT (students)	–	MULTI ARCH
CU Boulder	16	LangGraph tutorial & OpenHands	–	ARCH CODE
CU Boulder	18	RefAgent & Agentic Skills (students)	–	CODE ARCH TOOL
Mannheim	4	LLM Agents and Tool Use	[PDF] [notebook zip]	ARCH TOOL
Mannheim	6	Context Engineering	[PDF] [zip]	MEM ARCH
Stanford 329A	7	Open-Ended Evolution of Self-Improving Agents	–	ARCH CODE APP
Stanford 329A	13	Agentic Frameworks for Software Engineering	–	CODE ARCH
Berkeley '25	3	Evolution of system designs from an AI engineer perspective	[slides] [video]	ARCH DEPLOY
Berkeley Adv.	3	On Reasoning, Memory, and Planning of Language Agents	[slides] [video]	RSN MEM ARCH
Berkeley Adv.	11	Abstraction and Discovery with LLM Agents	[slides] [video]	RSN ARCH APP
Stanford 329T	1	Course introduction and overview	[slides]	ORG FND ARCH
Stanford 329T	3	Agentic AI	[slides]	ARCH EVAL
Stanford 329T	15	Multi-modal frontier models with agentic capabilities (guest: Deepak Ramachandran, Google DeepMind)	–	FND ARCH

Key readings

- Anthropic, *Building Effective Agents*
- Yao et al., *ReAct* (2022)
- Sumers et al., *CoALA: Cognitive Architectures for Language Agents* (2023)
- Chip Huyen, *Agents* (2025)
- Wang et al., *A Survey on LLM-based Autonomous Agents* (2024)

Hands-on

- **UW-Madison: Build a Coding Agent** — A CLI coding agent like Claude Code / Cursor CLI / Gemini CLI, using raw LLM API calls with no agent frameworks (LangChain, Strands, CrewAI banned; thin wrappers like litellm allowed). *Spec:* [GitHub]
- **CMU: A1: Build an Agent Harness** — A domain-agnostic ReAct `Agent` base class: prompt construction, tool-call loop, step limit, model-generated context compaction. *Starter code:* [GitHub] [GitHub] [GitHub]
- **UVA: Topic 2 portfolio: LangGraph agents** — Routed, parallel, multi-model LangGraph chat with history and crash recovery; optional web summarizer. *Starter code:* [cs.virginia.edu]
- **Buffalo: Lab 3: Workflow Graphs and State Machines** — Agentic workflow as a graph/state machine with explicit states, transitions, failure handling and logging. *(spec only)*
- **Buffalo: Lab 1: A Minimal Tool-Using Agent** — Python agent choosing among a calculator, a file reader and a search/lookup tool. *(spec only)*
- Supplement: Barkley local agent demos. Run `Local-Agent-Demo`, read `agent.py`, then compare it with the LangGraph version.
- **Stanford 329Z: HW1: Build an Agentic Harness** — An internal assistant for a fictional company (Cardinal Energy) from scratch, with no agent frameworks, over a real corporate email archive and company documents. *Starter code:* [GitHub]
- **Michigan: Lab05: Building on Codex** — Example products built on a harness (Slack chat bot, PR review bot, website builder); the Codex app-server (clients, threads, turns and items, approvals in the other direction, steering and stopping a turn); custom tools vs MCP; a toy chat bot (fixbot) on Codex. *Slides:* [GitHub]

Related supplements: Model Context Protocol docs (version 2026-07-28), OpenClaw documentation, MIT 6.566 guest lecture: AI Agent Security (Anish Athalye, April 2026), Jesse Barkley: local agent demos for the CMU executive Agentic AI program (2026).

Module 3: Tool Use and Function Calling

Tag: TOOL (MCP-focused lectures are in Module 4)

Learning objectives

- Explain how tool calls are encoded in chat templates and parsed by the harness.
- Design agent-ergonomic tools (names, schemas, errors, granularity).
- Implement manual tool dispatch, then framework tool nodes, and measure tool-selection accuracy.

Core concepts

Tools are how an agent acts. Three layers matter:

- **Encoding.** The model emits a structured call (name + JSON arguments) inside its chat template, the harness executes it, and the result returns as a tool message. Constrained decoding (e.g. XGrammar) can guarantee well-formed calls. Benchmarks such as BFCL measure function-calling accuracy.
- **Design.** Tools are an interface for a non-human user. Prefer a few high-level, composable tools with clear names, typed parameters and *informative errors* that tell the model how to recover. "Code as action" (CodeAct) replaces many tools with one interpreter. Stanford's readings include Anthropic's *Writing effective tools for agents* and "APIs don't make good MCP tools".
- **Selection at scale.** With dozens of tools, retrieval over tool descriptions, or a small classifier as in TinyAgent, keeps prompts short and accurate. For hundreds of tools see the code-execution pattern in Module 4 (MCP and Agent Interoperability).

The research lineage runs from Toolformer (self-taught API calls) to Gorilla (API retrieval), PAL (program-aided reasoning), MRKL and "What are tools anyway?".

Curated learning sequence

1. CMU 2 *Capabilities 1: Tool Use* — Tool use end to end: APIs, chat templates, CodeAct, constrained decoding, BFCL (slides + video).
2. UVA 5 *Topic 3: Agent Tool Use (online)* — Lab: manual tool handling with a calculator, the OpenAI tool format, tool forcing.
3. UVA 7 *Topic 4: Exploring Tools* — ToolNode versus `create_react_agent`, parallel tool dispatch, a 2-hour mini project.
4. Northeastern 5 *Tool-Calling* — ReAct support agent, tools-versus-model design rules, deploy to HF Spaces.
5. CU Boulder 11 *Reflexion & Toolformer (students)* and CU Boulder 12 *Gorilla & PAL (students)* — Research reading: Reflexion/Toolformer, then Gorilla/PAL.
6. Supplement: TinyAgent — Tool RAG and small-model function calling at the edge.
7. Stanford 329Z 4 *Tool Use & Function Calling* — From REST APIs to function calling to MCP, measured with BFCL and τ-bench.
8. Michigan 7 *Inside the Call: Messages, Streams, and Tool Calls* — A Chat Completions request on the wire: messages, streaming and tool calls (slides with speaker notes).

Lecture map (24 lectures)

Course	#	Title (links to catalogue row)	Materials	Tags
CMU	2	⋆Capabilities 1: Tool Use	[slides] [video]	TOOL
Stanford '26	5	All about agent skills (incl. web skills)	[course site]	MEM TOOL
Stanford '25	3	Building a coding agent from scratch	[slides] [drive.google.com]	ARCH TOOL CODE
Michigan	6	From pair-coding to agent orchestration	[slides]	CODE TOOL ARCH
Michigan	7	⋆Inside the Call: Messages, Streams, and Tool Calls	[slides]	TOOL FND ARCH
Michigan	10	Skills and the Elephant-Goldfish Model	[slides] [EGM skills]	TOOL CODE MEM
Northeastern	4	Simple Agent Architecture	[agenticllms.github.io] [GitHub]	ARCH TOOL MEM
Northeastern	5	⋆Tool-Calling	[agenticllms.github.io] [agenticllms.github.io] [GitHub]	TOOL RSN DEPLOY

Course	#	Title (links to catalogue row)	Materials	Tags
Harvard	4	Agents design I: context management and tool design	[PDF] [cs2680.seas.harvard.edu]	ARCH TOOL MEM
Buffalo	3	Tool/function calling and ReAct	–	TOOL RSN
UVA	5	⋆Topic 3: Agent Tool Use (online)	[cs.virginia.edu]	TOOL ARCH
UVA	6	Topic 3 lab continued (online)	–	TOOL
UVA	7	⋆Topic 4: Exploring Tools	[cs.virginia.edu] [GitHub]	TOOL RSN ARCH
UW-Madison	2	Building agents	[GitHub]	ARCH TOOL
UW-Madison	4	Reasoning and acting II	–	TOOL RSN
CU Boulder	9	ReAct & Web API code gen (students)	–	RSN TOOL EVAL
CU Boulder	11	⋆Reflexion & Toolformer (students)	–	RSN TOOL TRAIN
CU Boulder	12	⋆Gorilla & PAL (students)	–	TOOL RSN
CU Boulder	18	RefAgent & Agentic Skills (students)	–	CODE ARCH TOOL
CU Boulder	23	SafePro & ToolDreamer (students)	–	SAFE TOOL
Stanford 329A	4	Learning from Feedback with Tools/Code	[video]	TOOL CODE TRAIN
Berkeley Adv.	5	Coding Agents and AI for Vulnerability Detection	[slides] [video]	CODE TOOL SAFE
Berkeley Adv.	9	Language models for autoformalization and theorem proving	[slides] [video]	RSN APP TOOL
Stanford 329T	10	Data curation and RL to optimize agents (guest: Trung Vu, Bespoke Labs)	[slides]	TRAIN TOOL EVAL

Key readings

- Schick et al., *Toolformer* (2023)
- Patil et al., *Gorilla* (2023)
- Wang et al., *CodeAct: Executable Code Actions* (2024)
- Gao et al., *PAL* (2022)
- *What Are Tools Anyway?* (2024)
- Anthropic, *Writing effective tools for agents*

Hands-on

- **UVA: Topic 3 portfolio: Tool use** — Ollama parallel timing; manual tool calling with a calculator; LangGraph multi-tool agent with tool_map; persistent graph with Mermaid diagram. *Starter code:* [cs.virginia.edu]
- **Buffalo: Lab 1: A Minimal Tool-Using Agent** — Python agent choosing among a calculator, a file reader and a search/lookup tool. *(spec only)*
- **Northeastern: DADA1: The Trip Planner Agent** — A conversational day-trip planner with tools: get_weather (Open-Meteo), find_attractions/restaurants/transit (Tavily), find_events (Ticketmaster). *Starter code:* [GitHub]
- **UVA: Topic 4: 2-Hour Agent Project** — Partner-built mini agent using LangGraph/third-party tools. *Starter code:* [cs.virginia.edu]
- **Michigan: Lab04: Reaching the Computer: Shell, MCP and the Browser** — The shell tool (inputs, code mode, long-running and interactive commands, deadlines, background runs); MCP (servers, calls, local vs remote, MCP vs CLI, loading tools on demand); browser tools (pages, pixels and accessibility trees, click by reference); prompt injection and the lethal trifecta. *Slides:* [GitHub]

Related supplements: Anthropic Engineering: Code execution with MCP, Berkeley BAIR: TinyAgent, Function Calling at the Edge, MIT 6.566 guest lecture: AI Agent Security (Anish Athalye, April 2026), Anthropic: Building effective agent automations (Martin and Avilla, October 2026), Jesse Barkley: local agent demos for the CMU executive Agentic AI program (2026).

Module 4: MCP and Agent Interoperability

Tag: TOOL (lectures tagged TOOL whose title or topics mention MCP or A2A)

Learning objectives

- Describe MCP hosts, clients and servers, its primitives and transports, and what changed in the 2026-07-28 revision.
- Build and test an MCP server with authentication.
- Know when to switch from direct tool calls to code execution against MCP, and what A2A adds.

Core concepts

The **Model Context Protocol** standardizes how tools and context reach an agent: write one server, use it in any host. Read the MCP supplement first, because **the 2026-07-28 revision made the protocol stateless**, and lectures from before mid-2026 teach the older handshake.

- **Primitives:** tools, resources and prompts on the server side, and elicitation on the client side. Transports are stdio (local) and Streamable HTTP (remote, OAuth).
- **Server craft:** typed, composable tools; dry-run modes for write tools; `readOnlyHint`; structured errors; rate-limit handling; protocol-level tests (Stanford '26 week 2).
- **Security:** an MCP server is a trust boundary. See also Module 12 (Safety, Security and Oversight).
- **Scale:** with many servers, loading every tool schema is wasteful. Anthropic's code-execution pattern and CU Boulder's MCP design-choice paper argue for progressive discovery and code-based orchestration.
- **Agent-to-agent:** A2A (agent cards, registries) connects whole agents rather than tools. UVA runs an A2A trivia tournament.

Curated learning sequence

1. Supplement: MCP docs — Architecture, primitives and the 2026-07-28 changes.
2. Stanford '26 4 *Full introduction to MCP and tool-calling* — Full introduction to MCP and tool calling (theory, setup, advanced tool design).
3. Stanford '25 4 *Building a custom MCP server* — Building a custom MCP server (slides + completed exercise).
4. JHU 8 *Agentic Frameworks: Model Context Protocol* and JHU 9 *Agentic Frameworks: Model Context Protocol (2)* — Two MCP lectures in a PL/synthesis context (slides).
5. UVA 12 *Topic 7: MCP and A2A* — MCP clients for AI2 Asta tools, then A2A agents, a registry and ngrok.
6. Supplement: Code execution with MCP — Then CU Boulder 24 *Swiss Cheese safety & MCP design (students)* for the empirical study.
7. Supplement: Barkley local agent demos — A 32-line stdio server on version 2 of the Python SDK, and the same question asked with and without it.

Lecture map (13 lectures)

Course	#	Title (links to catalogue row)	Materials	Tags
Stanford '26	4	⋆Full introduction to MCP and tool-calling (theory, setup, advanced tool design)	[course site]	TOOL
Stanford '25	4	⋆Building a custom MCP server	[slides] [drive.google.com]	TOOL
Michigan	13	Browser Extensions and Assistants You Can Text	[slides]	WEB TOOL DEPLOY
Stanford 329Z	4	Tool Use & Function Calling	[slides]	TOOL EVAL
Harvard	2	Agents use I: How Claude Code Works	[PDF] [cs2680.seas.harvard.edu]	CODE MEM TOOL
Harvard	6	Agents design II: multi-agent architecture, coordination, and communication	[PDF] [cs2680.seas.harvard.edu]	MULTI ARCH TOOL
UVA	12	⋆Topic 7: MCP and A2A	[cs.virginia.edu]	TOOL MULTI
JHU	8	⋆Agentic Frameworks: Model Context Protocol	[PDF]	TOOL ARCH
JHU	9	⋆Agentic Frameworks: Model Context Protocol (2)	[PDF]	TOOL ARCH

Course	#	Title (links to catalogue row)	Materials	Tags
CU Boulder	24	Swiss Cheese safety & MCP design (students)	–	SAFE TOOL
Mannheim	4	LLM Agents and Tool Use	[PDF] [notebook zip]	ARCH TOOL
UChicago	5	Sampling, Fine Tuning, Benchmarking, and Tools	[GitHub]	TRAIN EVAL TOOL
Stanford 329T	8	MCP, effective tool descriptions, evals (guests: Yusuf Ozuysal, Ishita Gupta)	[slides (Ozuysal)] [slides (Gupta)]	TOOL EVAL

Key readings

- MCP specification and security best practices
- Anthropic, *Code execution with MCP* (2025)
- Felendler et al., *From Tool Orchestration to Code Execution: MCP Design Choices* (2026)
- *MCP Landscape & Security* (arXiv 2503.23278)

Hands-on

- **Stanford '26: Week 2 — Build an MCP Server for a Third-Party API** — FastMCP server over an OAuth 2.0 API with at least 3 composing tools, a write tool with dry-run, structured errors, readOnlyHint, full auth-code flow with refresh, and a protocol-level test. *Starter code:* [GitHub]
- **Stanford '25: Week 3 — Build a Custom MCP Server** — MCP server wrapping a real external API: at least 2 typed tools, robust error and rate-limit handling, STDIO or remote HTTP, optional OAuth2. *Starter code:* [GitHub]
- **UVA: Topic 7: MCP + A2A** — MCP client for AI2 Asta tools; A2A agents, registry and trivia tournament over ngrok. *Starter code:* [cs.virginia.edu]
- **Mannheim: Exercise: Agent Programming with LangChain + MCP** — Tool-using agents with LangChain and MCP. *Starter code:* [notebook zip]
- Supplement: Barkley local agent demos. Start with `EmeritusMCP`: add a second tool to the server and watch the agent discover it.
- **Michigan: Lab04: Reaching the Computer: Shell, MCP and the Browser** — The shell tool (inputs, code mode, long-running and interactive commands, deadlines, background runs); MCP (servers, calls, local vs remote, MCP vs CLI, loading tools on demand); browser tools (pages, pixels and accessibility trees, click by reference); prompt injection and the lethal trifecta. *Slides:* [GitHub]

Related supplements: Model Context Protocol docs (version 2026-07-28), Anthropic Engineering: Code execution with MCP, Berkeley BAIR: TinyAgent, Function Calling at the Edge, MIT 6.566 guest lecture: AI Agent Security (Anish Athalye, April 2026), Anthropic: Building effective agent automations (Martin and Avilla, October 2026), Jesse Barkley: local agent demos for the CMU executive Agentic AI program (2026).

Module 5: Memory, Retrieval and Context Engineering

Tag: MEM

Learning objectives

- Manage a finite context window: selection, compression/compaction, offloading and persistence.
- Build and evaluate a RAG pipeline (chunking, embeddings, top-K, reranking, unanswerable questions).
- Implement long-term memory and skills with progressive disclosure.

Core concepts

Apart from what the model learned in training, everything an agent can use at step t is in its context: the information supplied for that inference. **Context engineering** decides what goes in.

- **Limits.** Models degrade on long inputs ("lost in the middle", context rot), and every token costs money and latency. KV-cache reuse makes a stable prompt prefix cheap. CMU's long-context lecture uses real provider pricing and harness source code.
- **Compaction.** Summarize or fold old turns (model-generated compaction in CMU A1; AgentFold and context-folding in UVA and UW), and offload large observations to files.
- **Retrieval (RAG).** Chunk, embed, index, retrieve top-K, optionally rerank, then generate with citations. Treat chunk size, overlap, K, query rewriting and score thresholds as experimental variables (UVA Topic 5 runs 11 controlled experiments; Northeastern's DADA2 adds metadata filters, multi-query fusion, reranking and hybrid lexical retrieval one change at a time, scored by recall@k and MRR). Agentic RAG adds self-grading, correction and routing (Self-RAG, CRAG, Adaptive-RAG).
- **Long-term memory and skills.** OS-style memory tiers (MemGPT), reflection-based memories (Generative Agents, Reflexion), workflow and skill induction (AWM, ASI). *Skills* are SKILL.md bundles loaded on demand, the same progressive-disclosure idea Stanford '26 week 3 and CMU A1 teach.

Curated learning sequence

1. CMU 3 *Capabilities 2: Context Management for Long-Context Agents* — Context management for long-context agents: limits, compaction, KV-cache economics.
2. CMU 4 *Capabilities 3: Skills and Memory* — Skills and memory: MemGPT, AWM, ASI, ReasoningBank, SkillsBench.
3. Northeastern 7 *Context Management*–Northeastern 9 — Context engineering, then RAG from scratch to Pinecone, then a self-grading retrieval loop.
4. UVA 9 *Topic 5: Retrieval Augmented Generation* — The RAG experiment lab (chunking, top-K, unanswerable questions, small versus large model).
5. Mannheim 6 *Context Engineering* — Context-engineering deck and the deep-agent exercise.
6. Stanford '26 5 *All about agent skills (incl. web skills)* — Agent skills, including web skills (then author one in the Week 3 assignment).
7. UW-Madison 21 *Memory I* and UW-Madison 22 *Memory & other topics II* — Research reading: MemGPT, CoALA, A-Mem, context folding.
8. Supplement: Notion Data Scout — Curated, owned context for a production data agent.
9. Stanford 329Z 3 *Retrieval-Augmented Generation (RAG)* and Stanford 329Z 7 *Agent Memory Architectures* — Retrieval-augmented generation, then agent memory architectures.
10. Michigan 15 *Compaction and Prompt Caching*–Michigan 17 — Compaction and prompt caching, agent memory, and knowledge bases compared with RAG.

Lecture map (40 lectures)

Course	#	Title (links to catalogue row)	Materials	Tags
CMU	3	⋆Capabilities 2: Context Management for Long-Context Agents	[slides] [video]	MEM
CMU	4	⋆Capabilities 3: Skills and Memory	[slides] [video]	MEM
CMU	10	Domains 3: Deep Research Agents	[slides] [video]	APP MEM

Course	#	Title (links to catalogue row)	Materials	Tags
Stanford '26	3	Advanced prompting + agentic dev frameworks (RePPIT, spec-driven development)	[course site]	FND MEM
Stanford '26	5	⋆All about agent skills (incl. web skills)	[course site]	MEM TOOL
Stanford '25	5	From first prompt to optimal IDE setup	[slides] [drive.google.com]	MEM CODE
Michigan	10	Skills and the Elephant-Goldfish Model	[slides] [EGM skills]	TOOL CODE MEM
Michigan	14	Subagents	[slides]	MULTI ARCH MEM
Michigan	15	⋆Compaction and Prompt Caching	[slides]	MEM ARCH
Michigan	16	⋆Agent Memory	[slides]	MEM
Michigan	17	⋆Knowledge Bases	[slides]	MEM
Stanford 329Z	2	LLMs for Builders	[slides] [video]	FND MEM
Stanford 329Z	3	⋆Retrieval-Augmented Generation (RAG)	[slides]	MEM
Stanford 329Z	6	Agent Design Patterns & Scaffolds	[slides]	ARCH RSN MEM
Stanford 329Z	7	⋆Agent Memory Architectures	[slides]	MEM
Northeastern	4	Simple Agent Architecture	[agenticllms.github.io] [GitHub]	ARCH TOOL MEM
Northeastern	6	Short-term & Long-term Memory	[agenticllms.github.io] [agenticllms.github.io] [GitHub]	MEM ARCH DEPLOY
Northeastern	7	⋆Context Management	[agenticllms.github.io] [GitHub]	MEM ARCH
Northeastern	8	⋆Retrieval-Augmented Generation	[agenticllms.github.io] [GitHub]	MEM
Northeastern	9	⋆Reflection & Self-critique	[agenticllms.github.io] [GitHub]	RSN MEM
Northeastern	11	Agentic RAG	[agenticllms.github.io] [GitHub]	MEM RSN
Northeastern	16	Deep Agents I: The Agent Harness	[agenticllms.github.io] [GitHub]	ARCH MULTI MEM
Northeastern	17	Deep Agents II: Backends and Sandboxes	[agenticllms.github.io] [GitHub]	SAFE ARCH MEM
Harvard	2	Agents use I: How Claude Code Works	[PDF] [cs2680.seas.harvard.edu]	CODE MEM TOOL
Harvard	4	Agents design I: context management and tool design	[PDF] [cs2680.seas.harvard.edu]	ARCH TOOL MEM
Harvard	7	Agents design III: agent memory and reliability	[PDF] [cs2680.seas.harvard.edu]	MEM ARCH
Buffalo	4	Retrieval-augmented generation and context construction	–	MEM
Buffalo	5	Embeddings, vector search, reranking	–	MEM
Buffalo	8	Human-agent interaction and memory	–	MEM SAFE
UVA	9	⋆Topic 5: Retrieval Augmented Generation	[cs.virginia.edu] [GitHub]	MEM EVAL
UVA	15	Topic 9: Context Management	[cs.virginia.edu]	MEM
UW-Madison	8	Deep research agents II	–	APP MEM EVAL
UW-Madison	21	⋆Memory I	–	MEM ARCH
UW-Madison	22	⋆Memory & other topics II	–	MEM MULTI EVAL
CU Boulder	21	MemSkill & More with Less (students)	–	MEM DEPLOY
Mannheim	5	Retrieval Augmented Generation	[PDF] [zip]	MEM
Mannheim	6	⋆Context Engineering	[PDF] [zip]	MEM ARCH
UChicago	3	LLMs Prompted for Multi-Agent Simulation	[GitHub]	MULTI MEM APP
Berkeley Adv.	3	On Reasoning, Memory, and Planning of Language Agents	[slides] [video]	RSN MEM ARCH
Stanford 329T	2	Models, Prompting and RAG	[slides] [HW1]	FND MEM

Key readings

- Lewis et al., *Retrieval-Augmented Generation* (2020)
- Liu et al., *Lost in the Middle* (2023)
- Packer et al., *MemGPT* (2023)
- Park et al., *Generative Agents* (2023)
- Anthropic, *Effective context engineering for AI agents*
- Asai et al., *Self-RAG* (2023); Yan et al., *Corrective RAG* (2024)

Hands-on

- **UVA: Topic 5: RAG experiments** — 11 controlled RAG experiments (RAG vs no-RAG, top-K, chunking, prompts, unanswerable questions). *Starter code:* [cs.virginia.edu]
- **Northeastern: DADA2: Agentic RAG over SEC filings** — An ablation study in four notebooks (no separate spec). *Starter code:* [GitHub]
- **Buffalo: Lab 2: Retrieval and Memory** — Small RAG pipeline over a course-selected corpus. *(spec only)*

- **Stanford '26: Week 3 — Author a Skill** — A SKILL.md (Agent Skills standard) for a repeatable workflow on a real OSS repo (htmx, dayjs, yt-dlp, etc.), with trigger tests including a near-miss and an end-to-end run. *Starter code:* [GitHub]
- **Northeastern: Husky Tech Customer Support Agent** — A support agent with 4 tools and ticket filing. *Starter code:* [GitHub]
- **Mannheim: Exercise: RAG** — RAG pipeline notebook. *Starter code:* [zip]
- Supplement: Barkley local agent demos. In `Local-Agent-Demo`, change the short-term window and the similarity threshold in `config.py` and watch what the model is shown.
- **Stanford 329Z: HW1: Build an Agentic Harness** — An internal assistant for a fictional company (Cardinal Energy) from scratch, with no agent frameworks, over a real corporate email archive and company documents. *Starter code:* [GitHub]

Related supplements: Anthropic Engineering: Code execution with MCP, Notion: How we built a personal data scientist for every Notion employee (David Thomas, October 2026), Jesse Barkley: local agent demos for the CMU executive Agentic AI program (2026).

Module 6: Reasoning, Planning, Reflection and Search

Tag: RSN

Learning objectives

- Choose between ReAct-style interleaving, plan-then-execute and decomposition strategies.
- Add reflection/self-critique loops and measure whether they help.
- Use search (best-of-N, critics/verifiers, tree search) to trade compute for reliability.

Core concepts

- **Thinking in tokens.** CoT and self-consistency let the model compute before answering. UVA's CoT lecture looks at why this works and at latent (hidden) reasoning.
- **Planning.** Plan-and-execute, least-to-most and decomposed prompting split long tasks. Code-as-policies and DAG plans (LLMCompiler, used in TinyAgent) make plans executable and parallel. CMU's planning lecture asks *when* to plan ("Thinking vs. Doing", "Calibrate-Then-Act").
- **Reflection.** Reflexion and Self-Refine critique and retry. Northeastern's self-grading RAG loop is a concrete build. Reflection only helps when the feedback signal carries real information (tests, tools, retrieval scores), so measure it.
- **Search.** Sample many candidates, then pick with a verifier or critic (CMU Search 1), or explore a tree of states (ToT, LATS, CMU Search 2). AlphaCode, AlphaEvolve and ShinkaEvolve (UW) scale this to evolutionary program search. JHU starts from the classic version: enumerative program synthesis.

Curated learning sequence

1. CMU 5 *Capabilities 4: Planning, Task Decomposition, Multi-Agent Coordination* — Planning, task decomposition and multi-agent coordination (slides + video).
2. UW-Madison 3 *Reasoning and acting I* and UW-Madison 4 *Reasoning and acting II* — CoT/ReAct, then Toolformer/ToT, with supplementary papers.
3. Northeastern 9 *Reflection & Self-critique* and Northeastern 10 *Planning & Task Decomposition* — Reflection and self-critique, then planning with skill libraries (Voyager) and ToT.
4. UVA 16 *Topic 10: Understanding Chain of Thought* — Understanding chain of thought, including latent CoT.
5. CMU 20 *Search 1: Reranking and Critic Models* and CMU 21 *Search 2: Tree Search* — Reranking and critic models, then tree search.
6. UW-Madison 17 *Searching the solution space I* and UW-Madison 18 *Searching the solution space II* — Searching the solution space: AlphaCode, AlphaGeometry, evolutionary search.
7. JHU 2 *Syntax, Semantics, and Bottom-up Inductive Synthesis*–JHU 4 — Program synthesis as search (the foundation for A1).

Lecture map (49 lectures)

Course	#	Title (links to catalogue row)	Materials	Tags
CMU	5	⋆Capabilities 4: Planning, Task Decomposition, Multi-Agent Coordination	[slides] [video]	RSN MULTI
CMU	20	⋆Search 1: Reranking and Critic Models	–	RSN EVAL
CMU	21	⋆Search 2: Tree Search	–	RSN
Stanford '25	2	Power prompting for LLMs	[slides]	FND RSN
Michigan	4	Spec-Driven Development	[slides] [demo] [podcast]	CODE RSN
Stanford 329Z	6	Agent Design Patterns & Scaffolds	[slides]	ARCH RSN MEM
Northeastern	5	Tool-Calling	[agenticllms.github.io] [agenticllms.github.io] [GitHub]	TOOL RSN DEPLOY
Northeastern	9	⋆Reflection & Self-critique	[agenticllms.github.io] [GitHub]	RSN MEM
Northeastern	10	⋆Planning & Task Decomposition	[agenticllms.github.io] [GitHub]	RSN ARCH
Northeastern	11	Agentic RAG	[agenticllms.github.io] [GitHub]	MEM RSN
Buffalo	3	Tool/function calling and ReAct	–	TOOL RSN
Buffalo	7	Planning and decomposition	–	RSN
UVA	7	Topic 4: Exploring Tools	[cs.virginia.edu] [GitHub]	TOOL RSN ARCH

Course	#	Title (links to catalogue row)	Materials	Tags
UVA	16	⋆ Topic 10: Understanding Chain of Thought	[cs.virginia.edu]	RSN
UVA	20	Topic 14: Agentic AI for Mathematics and Verification (guest: Prof. Kevin Sullivan)	[cs.virginia.edu]	RSN APP
UVA	21	Topic 15: Visual Reasoning	[cs.virginia.edu] [video]	RSN EVAL
UW-Madison	3	⋆ Reasoning and acting I	–	RSN FND
UW-Madison	4	⋆ Reasoning and acting II	–	TOOL RSN
UW-Madison	16	Self-improving agents II	–	CODE RSN
UW-Madison	17	⋆ Searching the solution space I	–	RSN CODE
UW-Madison	18	⋆ Searching the solution space II	–	RSN
JHU	2	⋆ Syntax, Semantics, and Bottom-up Inductive Synthesis	[PDF]	RSN
JHU	3	⋆ Type Systems and Top-down Enumerative Synthesis	[PDF]	RSN
JHU	4	⋆ Functional Specifications and Synthesis	[PDF]	RSN
JHU	27	Special Topics on Neurosymbolic Programming	–	RSN APP
CU Boulder	9	ReAct & Web API code gen (students)	–	RSN TOOL EVAL
CU Boulder	11	Reflexion & Toolformer (students)	–	RSN TOOL TRAIN
CU Boulder	12	Gorilla & PAL (students)	–	TOOL RSN
CU Boulder	15	Multiagent debate & Agentless (students)	–	MULTI RSN CODE
Mannheim	3	Prompt Engineering	[PDF] [notebook zip]	FND RSN
UChicago	2	Text Learning, Transformers, and Diffusion Models	[GitHub]	FND RSN
UChicago	7	Reinforcement Learning to Optimize AI Agents and Institutions	[GitHub]	TRAIN RSN MULTI
UChicago	8	Multi-Modal and Embodied Agents	[GitHub]	WEB RSN APP
Stanford 329A	2	Test-time Compute Scaling	[video]	RSN EVAL
Stanford 329A	3	Robust Verification	[video]	RSN EVAL
Stanford 329A	5	Multi-step Reasoning/Planning	[video]	RSN TRAIN
Stanford 329A	6	Train Time Scaling/Scaling RL	[video]	TRAIN RSN
Stanford 329A	8	Self-Improvement with Search & Deep Research Agents	[video]	WEB RSN CODE
Berkeley '25	7	Multi-Agent AI	[slides] [video]	MULTI RSN
Berkeley Adv.	1	Inference-Time Techniques for LLM Reasoning	[slides] [intro] [video]	RSN ORG
Berkeley Adv.	2	Learning to reason with LLMs	[slides] [video]	TRAIN RSN
Berkeley Adv.	3	On Reasoning, Memory, and Planning of Language Agents	[slides] [video]	RSN MEM ARCH
Berkeley Adv.	4	Open Training Recipes for Reasoning in Language Models	[slides] [video]	TRAIN RSN
Berkeley Adv.	6	Multimodal Autonomous AI Agents	[slides] [video]	WEB EVAL RSN
Berkeley Adv.	8	AlphaProof: when reinforcement learning meets formal mathematics	[slides] [video]	RSN TRAIN APP
Berkeley Adv.	9	Language models for autoformalization and theorem proving	[slides] [video]	RSN APP TOOL
Berkeley Adv.	10	Bridging Informal and Formal Mathematical Reasoning (Berkeley page: "Advanced topics in theorem proving")	[slides] [video]	RSN APP
Berkeley Adv.	11	Abstraction and Discovery with LLM Agents	[slides] [video]	RSN ARCH APP
Stanford 329T	11	GEPA (guest: Lakshya Agrawal)	[slides]	TRAIN RSN

Key readings

- Wang et al., *Self-Consistency* (2022)
- Yao et al., *Tree of Thoughts* (2023)
- Shinn et al., *Reflexion* (2023); Madaan et al., *Self-Refine* (2023)
- Zhou et al., *Least-to-Most Prompting* (2022)
- Zhou et al., *LATS* (2023)
- Kim et al., *LLMCompiler* (2023)

Hands-on

- **JHU: A1: Inductive Synthesis** — A bottom-up enumerative synthesizer for a geometric-shape DSL (grow / eliminate observational equivalents / search); your own string DSL and synthesizer; then prompts plus output parsing so Claude Sonnet 4.6 (via Bedrock) solves at least 80% of string tasks. *Starter code:* [GitHub]
- **Stanford '25: Week 1 — Prompting Techniques** — Fill in prompts for K-shot, chain-of-thought, tool calling, self-consistency, RAG, and reflexion until the test scripts pass, using local Ollama models (mistral-nemo:12b, llama3.1:8b). *Starter code:* [GitHub]
- **Northeastern: Husky Tech Customer Support Agent** — A support agent with 4 tools and ticket filing. *Starter code:* [GitHub]
- Supplement: Barkley local agent demos. Wire a model into `tot_minimal.py`, then replace the model-scored evaluator with a code check and compare.

Related supplements: Jesse Barkley: local agent demos for the CMU executive Agentic AI program (2026).

Module 7: Multi-Agent Systems

Tag: MULTI

Learning objectives

- Implement handoffs, routers, orchestrator–worker and debate patterns.
- Diagnose multi-agent failure modes and decide when one agent is enough.

Core concepts

Multiple agents can parallelize work, isolate context (sub-agents with fresh windows) and specialize roles. They also multiply cost and add new failure modes: role confusion, lost information at handoffs, and endless loops.

- **Patterns:** router/handoff, orchestrator–workers (Anthropic's multi-agent research system), role-based SOPs (MetaGPT, CAMEL), conversation frameworks (AutoGen), debate, and planner/implementer/reviewer sub-agents in coding tools (Stanford '26 week 4).
- **Evidence:** *Why do multi-agent LLM systems fail?* (Northeastern reads it twice) and *Towards a Science of Scaling Agent Systems* (UW) set this against OpenHands' "don't sleep on single-agent systems". Start with one agent and add more only when a measurement says so.
- **Simulation:** populations of agents as social-science instruments (Generative Agents, Concordia, UChicago week 3).

Curated learning sequence

1. Northeastern 12 *Multi-Agent Orchestration I*–Northeastern 14 — Three sessions: failure modes and handoffs, role-based MAS, the LangGraph runtime and agent swarms.
2. CMU 18 *Interaction 1: Multi-Agent Interaction* — Multi-agent interaction (guest lecture).
3. UW-Madison 19 *Multi-agent systems I* and UW-Madison 20 *Multi-agent systems II* — Generative Agents/CAMEL, then AutoGen and the scaling science of agent systems.
4. CU Boulder 14 *AutoGen & MetaGPT (students)* and CU Boulder 15 *Multiagent debate & Agentless (students)* — AutoGen/MetaGPT, then debate and Agentless.
5. UChicago 3 *LLMs Prompted for Multi-Agent Simulation* — Prompted multi-agent social simulation (notebook with AutoGen and Concordia).
6. Supplements: Barkley local agent demos, CADSmith — One subagent with its own context and a briefing; then a five-role pipeline checked by a geometry kernel.
7. Stanford 329Z 8 *Multi-Agent Systems* — Multi-agent systems, including the A2A protocol.
8. Michigan 14 *Subagents* — Subagents as a way to isolate context.

Lecture map (33 lectures)

Course	#	Title (links to catalogue row)	Materials	Tags
CMU	5	Capabilities 4: Planning, Task Decomposition, Multi-Agent Coordination	[slides] [video]	RSN MULTI
CMU	18	⋆Interaction 1: Multi-Agent Interaction	[slides]	MULTI
Stanford '26	7	CLAUDE.md, AGENTS.md, hooks; subagent patterns (planner/implementer/reviewer)	[course site]	CODE MULTI
Stanford '25	7	How to be an agent manager	[slides]	CODE MULTI
Stanford '25	17	Incident response and DevOps	[slides]	DEPLOY MULTI
Michigan	14	⋆Subagents	[slides]	MULTI ARCH MEM
Michigan	19	Running Claude Code Projects on Your Own Machine	[slides]	MULTI DEPLOY CODE
Stanford 329Z	8	⋆Multi-Agent Systems	[slides]	MULTI
Northeastern	12	⋆Multi-Agent Orchestration I	[agenticllms.github.io] [GitHub]	MULTI ARCH WEB
Northeastern	13	⋆Multi-Agent Orchestration II	[agenticllms.github.io] [GitHub]	MULTI EVAL
Northeastern	14	⋆Multi-Agent Orchestration III	[agenticllms.github.io] [GitHub]	MULTI ARCH
Northeastern	16	Deep Agents I: The Agent Harness	[agenticllms.github.io] [GitHub]	ARCH MULTI MEM

Course	#	Title (links to catalogue row)	Materials	Tags
Harvard	3	Agents use II: 100x Engineer with Multi-agents	[PDF] [cs2680.seas.harvard.edu]	CODE MULTI
Harvard	6	Agents design II: multi-agent architecture, coordination, and communication	[PDF] [cs2680.seas.harvard.edu]	MULTI ARCH TOOL
Buffalo	11	Multi-agent systems and coordination	–	MULTI
UVA	3	Topic 2: Agent Frameworks	[cs.virginia.edu]	ARCH MULTI
UVA	12	Topic 7: MCP and A2A	[cs.virginia.edu]	TOOL MULTI
UVA	13	Topic 7 cont. (A2A) + Topic 8: LLM Fine-Tuning	[cs.virginia.edu]	TRAIN MULTI
UVA	19	Topic 13: Agentic AI for Science	[cs.virginia.edu] [GitHub]	APP MULTI
UW-Madison	7	Deep research agents I	–	APP MULTI
UW-Madison	10	Agents for science II	–	APP MULTI
UW-Madison	19	⋆Multi-agent systems I	–	MULTI
UW-Madison	20	⋆Multi-agent systems II	–	MULTI ARCH
UW-Madison	22	Memory & other topics II	–	MEM MULTI EVAL
CU Boulder	4	Refactoring agents (students)	–	CODE MULTI EVAL
CU Boulder	5	MM-Assist & MANTRA (Dig)	[PDF]	CODE MULTI
CU Boulder	14	⋆AutoGen & MetaGPT (students)	–	MULTI ARCH
CU Boulder	15	⋆Multiagent debate & Agentless (students)	–	MULTI RSN CODE
UChicago	3	⋆LLMs Prompted for Multi-Agent Simulation	[GitHub]	MULTI MEM APP
UChicago	7	Reinforcement Learning to Optimize AI Agents and Institutions	[GitHub]	TRAIN RSN MULTI
Berkeley '25	7	Multi-Agent AI	[slides] [video]	MULTI RSN
Berkeley '25	9	AI Agents to Automate Scientific Discoveries	[video]	APP MULTI
Berkeley '25	11	Multi-Agent Systems in the Era of LLMs	[video]	MULTI

Key readings

- Wu et al., *AutoGen* (2023)
- Hong et al., *MetaGPT* (2023)
- Li et al., *CAMEL* (2023)
- Du et al., *Multiagent Debate* (2023)
- Cemri et al., *Why Do Multi-Agent LLM Systems Fail?* (2025)
- Anthropic, *How we built our multi-agent research system*

Hands-on

- **Northeastern: DADA3: A Bug-Fixing Team for a GitHub Repository** — A team of agents that works through the ten open issues of a small pure-Python library (huskyutils; seven real bugs, two reports of documented behavior, one feature request; each docstring is the spec). *Starter code:* [GitHub]
- **UVA: Topic 7: MCP + A2A** — MCP client for AI2 Asta tools; A2A agents, registry and trivia tournament over ngrok. *Starter code:* [cs.virginia.edu]
- **UChicago: Week 3 notebook** — Prompting (0-shot, few-shot, chain-of-reasoning) via the OpenAI API; simulated survey and social-media respondents compared with human data; AutoGen multi-agent dialogues with reflection; Concordia situated simulation; RAG over 5-10 documents vs. a vanilla model. *Starter code:* [GitHub]
- **Mannheim: Project topic 1: Multi-Agent Gaming** — Agent teams that cooperate in, or compete at, games (Codenames worked well before). *Resources:* [GitHub] [GitHub]
- **Stanford '25: Week 5 — Agentic Development with Warp** — Warp Drive prompts, rules, or MCP integrations, plus multi-agent concurrent sessions working from a task list. *Starter code:* [GitHub]
- Supplement: Barkley local agent demos. In `Agent-with-Subagent`, untick delegation and compare the main agent's context with and without the subagent.

Related supplements: Jesse Barkley: local agent demos for the CMU executive Agentic AI program (2026), CADSmith: multi-agent CAD generation with programmatic geometric validation (Barkley, Loghmani and Barati Farimani, 2026).

Module 8: Coding Agents

Tag: CODE

Learning objectives

- Explain the architecture of production coding agents (system prompt, core tools, sandbox, context management, sub-agents).
- Build a coding agent and evaluate it on repository-level tasks.
- Make repositories and teams agent-ready (AGENTS.md/CLAUDE.md, hooks, tests, review).

Core concepts

Coding is the domain where agents work best today, because the environment provides dense, verifiable feedback (compilers, tests, linters).

- **Anatomy.** A long system prompt, a small core toolset (read, write, edit, bash, search), a sandbox, compaction, TODO/plan tools, and optionally sub-agents. Stanford '26 dissects real system prompts and traces. JHU A3 makes you implement path-validated file tools and TODO tools.
- **Interfaces matter.** SWE-agent showed that agent–computer interface design (viewer windows, linting edits) moves benchmark scores. Agentless showed that a fixed pipeline can be competitive, so keep a simple baseline.
- **Benchmarks.** SWE-bench (and SWE-Gym, R2E-Gym for training), Terminal-Bench, BaxBench and RedCode (security). See Module 10 (Evaluating Agents).
- **Practice.** Spec-driven development, context files (AGENTS.md/CLAUDE.md), hooks as lint/test gates, agentic code review, background agents, and "agent readiness" of a codebase (Stanford '26 weeks 2–8).
- **Research frontier.** Refactoring agents (CU Boulder), coding-agent configuration studies, and evolutionary coding agents (AlphaEvolve, Live-SWE-agent).

Curated learning sequence

1. CMU 6 *Domains 1: Coding Agents* — Agents for coding and software development (slides + video).
2. Stanford '26 1 *Course intro + build Claude Code in 200 lines* and Stanford '26 2 *How SOTA coding agents are designed: deep dive into system prompts* — Build a mini Claude Code, then a deep dive into production system prompts.
3. UW-Madison 5 *Coding agents I* and UW-Madison 6 *Coding agents II* — SWE-bench, SWE-agent, OpenHands, BaxBench.
4. CU Boulder 13 *SWE-agent & AutoCodeRover (students)* — SWE-agent and AutoCodeRover (agent–computer interfaces, repository repair).
5. JHU 7 *Iterative Refinement and Agentic Framework* and JHU 10 *Agentic Frameworks: Selected Topics* — Iterative refinement and agentic frameworks for code (slides).
6. Harvard 2 *Agents use I: How Claude Code Works* and Harvard 4 *Agents design I: context management and tool design* — How Claude Code works, then the design space of search, edit and shell tools and of context management (slides).
7. Harvard 16 *Assignments feedback and agent pitfalls, GPU programming (cont.)* — What students' agents actually did: why the agent reached for Bash over its native tools, and ten pitfalls of working with a coding agent (slides).
8. Stanford '26 7 *CLAUDE.md, AGENTS.md, hooks; subagent patterns*, Stanford '26 9, Stanford '26 11, Stanford '26 15 — Customizing the agent, agent-ready repos, code review, background agents.
9. UVA 17 *Topic 11: Coding Agents* — History, Claude Code architecture, skills files and evolutionary programming.
10. Michigan 5 *Modern agentic CLIs*, Michigan 6, Michigan 9–Michigan 11 — Modern agentic CLIs, from edit blocks to the tool loop, stop conditions and approvals, skills, guardrails and retries (slides with speaker notes).
11. Stanford 329Z 18 *Coding & Software Agents* — Coding and software agents.

Lecture map (62 lectures)

Course	#	Title (links to catalogue row)	Materials	Tags
CMU	6	⋆Domains 1: Coding Agents	[slides] [video]	CODE
CMU	14	Frameworks 1: OpenHands	[slides] [video]	ARCH CODE

Course	#	Title (links to catalogue row)	Materials	Tags
Stanford '26	1	⋆Course intro + build Claude Code in 200 lines	[course site]	CODE ARCH
Stanford '26	2	⋆How SOTA coding agents are designed: deep dive into system prompts	[course site]	CODE FND
Stanford '26	7	⋆CLAUDE.md, AGENTS.md, hooks; subagent patterns (planner/implementer/reviewer)	[course site]	CODE MULTI
Stanford '26	9	⋆Agent readiness in your repos: structure, docs, tests, checks; scoring and auditing readiness	[course site]	CODE EVAL
Stanford '26	11	⋆Agentic code review: best practices and architectures	[course site]	CODE EVAL
Stanford '25	3	Building a coding agent from scratch	[slides] [drive.google.com]	ARCH TOOL CODE
Stanford '25	5	From first prompt to optimal IDE setup	[slides] [drive.google.com]	MEM CODE
Stanford '25	7	How to be an agent manager	[slides]	CODE MULTI
Stanford '25	9	How to build a breakthrough AI developer product	[slides]	CODE DEPLOY
Stanford '25	11	AI QA, SAST, DAST, and beyond	[slides]	SAFE EVAL CODE
Stanford '25	13	AI code review	[slides]	CODE EVAL
Stanford '25	15	End-to-end apps from a single prompt	[slides]	CODE APP
Stanford '25	19	Software development in 10 years	–	CODE ORG
Michigan	1	Course intro + the AI coding landscape	[slides] [demo] [podcast]	ORG CODE
Michigan	3	The Big Three: Context, Model, Prompt	[slides] [demo] [podcast]	CODE FND
Michigan	4	Spec-Driven Development	[slides] [demo] [podcast]	CODE RSN
Michigan	5	⋆Modern agentic CLIs	[slides] [podcast]	CODE ARCH SAFE
Michigan	6	⋆From pair-coding to agent orchestration	[slides]	CODE TOOL ARCH
Michigan	10	⋆Skills and the Elephant-Goldfish Model	[slides] [EGM skills]	TOOL CODE MEM
Michigan	19	Running Claude Code Projects on Your Own Machine	[slides]	MULTI DEPLOY CODE
Stanford 329Z	18	⋆Coding & Software Agents	–	CODE ARCH
Harvard	2	⋆Agents use I: How Claude Code Works	[PDF] [cs2680.seas.harvard.edu]	CODE MEM TOOL
Harvard	3	Agents use II: 100x Engineer with Multi-agents	[PDF] [cs2680.seas.harvard.edu]	CODE MULTI
Harvard	16	⋆Assignments feedback and agent pitfalls, GPU programming (cont.)	[PDF] [PDF] [how-i-used-claude.cs2680.com]	CODE DEPLOY
Buffalo	12	Coding agents, testing, CI, observability	–	CODE DEPLOY
UVA	17	⋆Topic 11: Coding Agents	[cs.virginia.edu]	CODE ARCH
UW-Madison	5	⋆Coding agents I	–	CODE EVAL
UW-Madison	6	⋆Coding agents II	–	CODE SAFE EVAL
UW-Madison	16	Self-improving agents II	–	CODE RSN
UW-Madison	17	Searching the solution space I	–	RSN CODE
JHU	5	Language Modeling for Synthesis	[PDF]	FND CODE
JHU	6	Decoding and Prompting for Synthesis	[PDF]	FND CODE
JHU	7	⋆Iterative Refinement and Agentic Framework	[PDF]	ARCH CODE
JHU	10	⋆Agentic Frameworks: Selected Topics	[PDF]	ARCH CODE
JHU	11	Pre-training of Coding Language Models (1)	[PDF]	TRAIN CODE
JHU	12	Pre-training of Coding Language Models (2)	[PDF]	TRAIN CODE
JHU	13	Post-training of Coding Language Models: SFT & RL	[PDF]	TRAIN CODE
JHU	14	Special Topics on Coding Language Models	[PDF]	TRAIN CODE
JHU	15	PL Kaleidoscope I: Formal Verification for Mathematics	[PDF]	APP CODE
JHU	16	PL Kaleidoscope II: Formal Program Verification	[PDF] [machine-programming.github.io]	APP CODE
JHU	17	PL Kaleidoscope III: Software Security and Static Analysis	[PDF]	SAFE CODE APP
JHU	18	PL Kaleidoscope IV: Software Security and Dynamic Analysis	[PDF]	SAFE CODE APP
JHU	19	PL Kaleidoscope V: Accelerated Computing	[PDF] [machine-programming.github.io]	APP CODE
CU Boulder	2	Survey of agents (Dig)	[PDF]	ARCH CODE
CU Boulder	3	Agentic SE roadmap (Dig)	[PDF]	CODE ARCH
CU Boulder	4	Refactoring agents (students)	–	CODE MULTI EVAL
CU Boulder	5	MM-Assist & MANTRA (Dig)	[PDF]	CODE MULTI
CU Boulder	13	⋆SWE-agent & AutoCodeRover (students)	–	CODE ARCH
CU Boulder	15	Multiagent debate & Agentless (students)	–	MULTI RSN CODE
CU Boulder	16	LangGraph tutorial & OpenHands	–	ARCH CODE

Course	#	Title (links to catalogue row)	Materials	Tags
CU Boulder	18	RefAgent & Agentic Skills (students)	–	CODE ARCH TOOL
CU Boulder	19	AppAgent & coding-agent configuration (students)	–	WEB CODE
CU Boulder	22	MetaClaw & code smells (students)	–	CODE EVAL
UChicago	0	Introduction / Review	–	ORG CODE
Stanford 329A	4	Learning from Feedback with Tools/Code	[video]	TOOL CODE TRAIN
Stanford 329A	7	Open-Ended Evolution of Self-Improving Agents	–	ARCH CODE APP
Stanford 329A	8	Self-Improvement with Search & Deep Research Agents	[video]	WEB RSN CODE
Stanford 329A	13	Agentic Frameworks for Software Engineering	–	CODE ARCH
Berkeley '25	4	Post-Training Verifiable Agents	[slides] [video]	TRAIN EVAL CODE WEB
Berkeley Adv.	5	Coding Agents and AI for Vulnerability Detection	[slides] [video]	CODE TOOL SAFE

Key readings

- Jimenez et al., *SWE-bench* (2023)
- Yang et al., *SWE-agent* (2024)
- Wang et al., *OpenHands* (2024)
- Xia et al., *Agentless* (2024)
- Zhang et al., *AutoCodeRover* (2024)
- Hassan et al., *Agentic Software Engineering* roadmap (2025)
- Anthropic, *Claude Code best practices*

Hands-on

- **UW-Madison: Build a Coding Agent** — A CLI coding agent like Claude Code / Cursor CLI / Gemini CLI, using raw LLM API calls with no agent frameworks (LangChain, Strands, CrewAI banned; thin wrappers like litellm allowed). *Spec:* [GitHub]
- **JHU: A3: Coding LLM Agents** — "WAA" (Web-App Agent): a Python tool-calling loop (tool registry, system prompt, LLM query, parse, execute, terminate or hit the turn limit), file-system tools with path validation and protected files, TODO tools; the agent then builds a chat room, a Tetris variant in three milestones (mechanics, rules/scoring, leaderboard) and a creative app, plus a report; runs on Bedrock Claude Sonnet 4.6, max_turns 75; an optional extension adds a database plus DB tools. *Starter code:* [GitHub]
- **CMU: A1: Build an Agent Harness** — A domain-agnostic ReAct `Agent` base class: prompt construction, tool-call loop, step limit, model-generated context compaction. *Starter code:* [GitHub] [GitHub] [GitHub]
- **Harvard: A2 Mads-Loop: Build the loop** — `madsLoop`, a minimal coding-agent harness around an OpenAI-compatible model API (the handout pins `qwen3.6-35b-a3b`): tool calling with schemas, context management, retries, termination, a required log schema. *Starter code:* [cs2680.seas.harvard.edu] [cs2680.seas.harvard.edu]
- **Stanford '26: Week 1 — Trace Dissection of a Real Claude Code Session** — Capture a Claude Code session through a mitmproxy reverse proxy, then annotate the system prompt, `<system-reminder>`s, tool inventory and schemas, error recovery, planning, subagents, and context management. *Starter code:* [GitHub]
- **Stanford '25: Week 4 — The Autonomous Coding Agent IRL** — At least 2 Claude Code automations (slash commands, CLAUDE.md, subagents, MCP) used to extend a FastAPI app. *Starter code:* [GitHub]
- **Stanford '25: Week 7 — Exploring AI Code Review Using Graphite** — Implement 4 tasks with one-shot agent prompts, open PRs, and compare manual review with Graphite Diamond AI review. *Starter code:* [GitHub]
- **Michigan: P2: Aider as an agent** — Extend a private copy of Aider in four parts using Codex: a skills loader, a loop in which the model chooses tools, a shell tool behind approval rules, and a stdio MCP client. *Spec:* [GitHub]
- **Michigan: Lab01: How Aider works** — Take Aider apart as a harness: edit-shaped replies, what the model never touches, who chooses context, the repo map, the shared token budget, how the prompt is rebuilt each turn, architect mode; start designing the pair-programmer. *Slides:* [GitHub]
- **Stanford 329Z: HW1: Build an Agentic Harness** — An internal assistant for a fictional company (Cardinal Energy) from scratch, with no agent frameworks, over a real corporate email archive and company documents. *Starter code:* [GitHub]

Module 9: Web, GUI, Computer-Use and Multimodal Agents

Tag: WEB

Learning objectives

- Explain observation and action spaces for browser/GUI agents (DOM, accessibility tree, screenshots, coordinates).
- Build a vision-language agent and know the main web/OS benchmarks.

Core concepts

When there is no API, the agent operates the interface a human would. Observations are HTML/DOM, accessibility trees or screenshots (set-of-marks). Actions are clicks, typing and navigation, or full OS control. These tasks have long horizons, partial observability and irreversible actions, which makes them hard and makes safety and human confirmation central. Benchmarks progress from MiniWoB and WebShop through Mind2Web and WebArena/VisualWebArena to OSWorld. Training environments include CUA-Gym and OpenCUA. Mobile agents (AppAgent) and embodied agents (SayCan, Voyager) extend the same loop. Personal agents such as the OpenClaw family control real accounts, so read UVA's risk discussion together with Module 12 (Safety, Security and Oversight).

Curated learning sequence

1. CMU 7 *Computer Use Agents* — Computer-use agents: WebArena, VisualWebArena, OSWorld, OpenCUA (slides + video).
2. UVA 10 *Topic 6: Vision-Language Models* — Vision-language models and a VLM chat agent and video-surveillance agent lab.
3. CU Boulder 19 *AppAgent & coding-agent configuration (students)* — AppAgent (smartphone GUI) and coding-agent configuration.
4. UVA 18 *Topic 12: Personal Agents and Distilling* — Personal agents (OpenClaw family, Claude Cowork) and their risks; distillation.
5. UChicago 8 *Multi-Modal and Embodied Agents* — Multimodal and embodied agents (notebook).

Lecture map (13 lectures)

Course	#	Title (links to catalogue row)	Materials	Tags
CMU	7	⋆Computer Use Agents	[slides] [video]	WEB
Michigan	13	Browser Extensions and Assistants You Can Text	[slides]	WEB TOOL DEPLOY
Northeastern	12	Multi-Agent Orchestration I	[agenticllms.github.io] [GitHub]	MULTI ARCH WEB
UVA	10	⋆Topic 6: Vision-Language Models	[cs.virginia.edu] [GitHub]	WEB APP FND
UVA	18	⋆Topic 12: Personal Agents and Distilling	[cs.virginia.edu]	WEB SAFE TRAIN
UW-Madison	15	Self-improving agents I	–	ARCH WEB
CU Boulder	19	⋆AppAgent & coding-agent configuration (students)	–	WEB CODE
UChicago	8	⋆Multi-Modal and Embodied Agents	[GitHub]	WEB RSN APP
Stanford 329A	8	Self-Improvement with Search & Deep Research Agents	[video]	WEB RSN CODE
Berkeley '25	4	Post-Training Verifiable Agents	[slides] [video]	TRAIN EVAL CODE WEB
Berkeley Adv.	6	Multimodal Autonomous AI Agents	[slides] [video]	WEB EVAL RSN
Berkeley Adv.	7	Multimodal Agents – From Perception to Action	[slides] [video]	WEB EVAL
Stanford 329T	4	Recap, further discussion, and planning for projects	[slides]	ORG WEB

Key readings

- Zhou et al., *WebArena* (2023); Koh et al., *VisualWebArena* (2024)

- Xie et al., *OSWorld* (2024)
- Deng et al., *Mind2Web* (2023)
- Yao et al., *WebShop* (2022)
- Zhang et al., *AppAgent* (2023)
- Wang et al., *Voyager* (2023)

Hands-on

- **UVA: Topic 6: VLM agents** — Image-chat LangGraph agent; video-surveillance agent with OpenCV + LLaVA (optional TTS intruder alert). *Starter code:* [cs.virginia.edu]
- **Mannheim: Project topic 4: Online Shopping Assistant** — Product search/recommendation agent with structured feature comparison. *Resources:* [webarena.dev] [webshop-pnlp.github.io]
- **Mannheim: Project topic 2: QA Agents** — RAG (multi-)agent QA over web search that explains its evidence. *Resources:* [openai.com] [aicrowd.com]
- **Michigan: Lab04: Reaching the Computer: Shell, MCP and the Browser** — The shell tool (inputs, code mode, long-running and interactive commands, deadlines, background runs); MCP (servers, calls, local vs remote, MCP vs CLI, loading tools on demand); browser tools (pages, pixels and accessibility trees, click by reference); prompt injection and the lethal trifecta. *Slides:* [GitHub]

Module 10: Evaluating Agents

Tag: EVAL

Learning objectives

- Design task-level evals with success criteria, failure taxonomies and reproducible scripts.
- Build and validate an LLM-as-judge against human labels.
- Report results honestly (pass@k, variance, cost) and avoid benchmark pitfalls.

Core concepts

Evaluation separates agent engineering from demo-building, and every serious course in this collection weights it heavily.

- **Levels.** Unit tests for tools, trajectory and intermediate-step checks, end-to-end task success, and human judgments of usefulness.
- **Verifiers.** Deterministic tests where possible (SWE-bench, Harbor tasks with reference solutions and *mutants* in CMU A2). LLM-as-judge where not, *validated against human labels* (CMU A2 scores judges by MCC).
- **Statistics and cost.** Agents are stochastic: report pass@k over repeated runs (JHU A2), confidence intervals and dollars per task. *AI Agents That Matter* criticizes leaderboards that ignore cost and overfitting.
- **Error analysis.** A failure taxonomy (Buffalo Lab 4; CMU A2's four error families) turns logs into engineering decisions.
- **Benchmarks:** SWE-bench, τ-bench, AgentBench, WebArena, AgentDojo (security), BrowseComp and DeepResearch Bench (research agents), and METR's time-horizon measurements.

Curated learning sequence

1. **CMU: A2: Evaluating a Data-Visualization Agent** — The single best exercise here: author tasks, label runs, build and score a judge.
2. Northeastern 18 *Agentic AI Evaluation* — SWE-bench, τ-bench, LLM-as-a-judge, LangSmith evaluation.
3. Buffalo 9 *Evaluation of agents* — From single-prompt to task-level evals (then Lab 4).
4. Harvard 8 *Agent design IV: agent evaluation* — Benchmark by benchmark: what each one tests and what its grader inspects, plus leakage and saturation (slides and a reference note).
5. **JHU: A2: Evaluating Coding LLMs** — A rigorous pass@k study across languages, models and prompting strategies.
6. CMU 20 *Search 1: Reranking and Critic Models* — Critic models: evaluation used at inference time.
7. UW-Madison 22 *Memory & other topics II* — Includes METR's *Measuring AI Ability to Complete Long Tasks*.
8. Stanford 329Z 13 *Evaluation Fundamentals & Benchmark Design* and Stanford 329Z 15 *LLM-as-Judge & Evaluation Infrastructure* — Evaluation fundamentals and benchmark design, then LLM-as-judge and evaluation infrastructure.

Lecture map (44 lectures)

Course	#	Title (links to catalogue row)	Materials	Tags
CMU	17	Agents and the Future of Work	[slides]	APP EVAL
CMU	19	Interaction 2: Human-Agent Interaction	–	APP EVAL
CMU	20	⋆Search 1: Reranking and Critic Models	–	RSN EVAL
Stanford '26	9	Agent readiness in your repos: structure, docs, tests, checks; scoring and auditing readiness	[course site]	CODE EVAL
Stanford '26	11	Agentic code review: best practices and architectures	[course site]	CODE EVAL
Stanford '25	11	AI QA, SAST, DAST, and beyond	[slides]	SAFE EVAL CODE
Stanford '25	13	AI code review	[slides]	CODE EVAL
Stanford 329Z	4	Tool Use & Function Calling	[slides]	TOOL EVAL
Stanford 329Z	11	What Data Do Agents Need?	[slides]	TRAIN EVAL

Course	#	Title (links to catalogue row)	Materials	Tags
Stanford 329Z	12	Data Selection & Quality	[slides]	TRAIN EVAL
Stanford 329Z	13	⋆Evaluation Fundamentals & Benchmark Design	[slides]	EVAL
Stanford 329Z	15	⋆LLM-as-Judge & Evaluation Infrastructure	–	EVAL
Northeastern	13	Multi-Agent Orchestration II	[agenticllms.github.io] [GitHub]	MULTI EVAL
Northeastern	18	⋆Agentic AI Evaluation	[agenticllms.github.io] [GitHub]	EVAL
Harvard	8	⋆Agent design IV: agent evaluation	[PDF] [cs2680.seas.harvard.edu]	EVAL
Buffalo	9	⋆Evaluation of agents	–	EVAL
UVA	1	Topic 1: Running an LLM	[cs.virginia.edu]	FND EVAL DEPLOY
UVA	9	Topic 5: Retrieval Augmented Generation	[cs.virginia.edu] [GitHub]	MEM EVAL
UVA	21	Topic 15: Visual Reasoning	[cs.virginia.edu] [video]	RSN EVAL
UW-Madison	5	Coding agents I	–	CODE EVAL
UW-Madison	6	Coding agents II	–	CODE SAFE EVAL
UW-Madison	8	Deep research agents II	–	APP MEM EVAL
UW-Madison	11	Security & prompt injection I	–	SAFE EVAL
UW-Madison	22	⋆Memory & other topics II	–	MEM MULTI EVAL
CU Boulder	4	Refactoring agents (students)	–	CODE MULTI EVAL
CU Boulder	9	ReAct & Web API code gen (students)	–	RSN TOOL EVAL
CU Boulder	22	MetaClaw & code smells (students)	–	CODE EVAL
Mannheim	x1	Evaluation of LLMs and LLM agents	–	EVAL
UChicago	4	Experimental Designs with AI Agent and Human Interactions	[GitHub]	APP EVAL
UChicago	5	Sampling, Fine Tuning, Benchmarking, and Tools	[GitHub]	TRAIN EVAL TOOL
Stanford 329A	2	Test-time Compute Scaling	[video]	RSN EVAL
Stanford 329A	3	Robust Verification	[video]	RSN EVAL
Stanford 329A	17	Agentic Evaluations & Long-Horizon Tasks	[video]	EVAL
Berkeley '25	4	Post-Training Verifiable Agents	[slides] [video]	TRAIN EVAL CODE WEB
Berkeley '25	5	Agent Evaluation & Project Overview	[slides] [video]	EVAL ORG
Berkeley '25	8	Predictable Noise in LLM	[slides] [video]	EVAL
Berkeley '25	10	Practical Lessons from Deploying Real-World AI Agents	[slides] [video]	DEPLOY EVAL APP
Berkeley Adv.	6	Multimodal Autonomous AI Agents	[slides] [video]	WEB EVAL RSN
Berkeley Adv.	7	Multimodal Agents – From Perception to Action	[slides] [video]	WEB EVAL
Stanford 329T	3	Agentic AI	[slides]	ARCH EVAL
Stanford 329T	7	Systematic evaluation of agentic systems (guest: Allison Jia)	[slides]	EVAL
Stanford 329T	8	MCP, effective tool descriptions, evals (guests: Yusuf Ozuysal, Ishita Gupta)	[slides (Ozuysal)] [slides (Gupta)]	TOOL EVAL
Stanford 329T	10	Data curation and RL to optimize agents (guest: Trung Vu, Bespoke Labs)	[slides]	TRAIN TOOL EVAL
Stanford 329T	12	Security for agents: adversarial attacks and guardrails (guests: Matt Fredrikson, Andy Zou)	[slides]	SAFE EVAL

Key readings

- Zheng et al., *Judging LLM-as-a-Judge* (2023)
- Yao et al., *τ-bench* (2024)
- Liu et al., *AgentBench* (2023)
- Kapoor et al., *AI Agents That Matter* (2024)
- Mohammadi et al., *Evaluation and Benchmarking of LLM Agents* (KDD 2025)
- METR, *Measuring AI Ability to Complete Long Tasks* (2025)

Hands-on

- **CMU: A2: Evaluating a Data-Visualization Agent** — A `validate(run)` LLM-as-judge validator that flags four error families (execution_failure, wrong_data, wrong_chart, hard_to_read), scored by per-family and

macro MCC against human labels. *Starter code:* [GitHub]

- **JHU: A2: Evaluating Coding LLMs** — Execution harnesses for Python/Rust/OCaml; 4 prompting strategies (zero-shot, two-step CoT, iterative refinement with feedback up to 3 rounds, a custom method of about 5 calls or fewer); pass@1/2/3 pipeline over 24 experiments (Claude Haiku vs Sonnet x 3 languages x 4 strategies) on a 16-item xCodeEval/MBPP set; plots. *Starter code:* [GitHub]
- **Buffalo: Lab 4: Evaluation Harness** — Eval suite for a bounded task: test cases, success criteria, metrics, failure taxonomy, reproducible script. *(spec only)*
- **UVA: Topic 5: RAG experiments** — 11 controlled RAG experiments (RAG vs no-RAG, top-K, chunking, prompts, unanswerable questions). *Starter code:* [cs.virginia.edu]
- **Harvard: A2 Mads-Loop: Build the loop** — `madsLoop`, a minimal coding-agent harness around an OpenAI-compatible model API (the handout pins `qwen3.6-35b-a3b`): tool calling with schemas, context management, retries, termination, a required log schema. *Starter code:* [cs2680.seas.harvard.edu] [cs2680.seas.harvard.edu]
- **Stanford 329Z: HW2: Evaluate an Agent** — In the Harbor framework (Docker environments): run existing tasks from HumanEval, SWE-bench Verified and Terminal-Bench 2.1 and analyze agent trajectories; then author one original multi-turn task (problem statement, environment, verifier giving a reward in [0, 1], oracle solution), validate it against wrong and partial solutions, run five attempts of mini-SWE-agent with Gemini 3.8 Flash and report on validity, realism, reproducibility and difficulty. *Starter code:* [GitHub]

Related supplements: Notion: How we built a personal data scientist for every Notion employee (David Thomas, October 2026), CADSmith: multi-agent CAD generation with programmatic geometric validation (Barkley, Loghmani and Barati Farimani, 2026).

Module 11: Training and Optimizing Agents

Tag: TRAIN

Learning objectives

- Fine-tune a model on agent trajectories (SFT, LoRA/QLoRA) and know when fine-tuning beats prompting or RAG.
- Implement policy-gradient RL (REINFORCE, GRPO) for multi-turn agents, including async rollouts.
- Optimize prompts and programs automatically (DSPy, TextGrad, GEPA).

Core concepts

- **SFT and distillation.** Collect or synthesize high-quality trajectories from a strong model, filter them with verifiers, and fine-tune a smaller model (TinyAgent, CMU Training 1, UVA's Tinker text-to-SQL lab). LoRA and QLoRA make this cheap.
- **RL from verifiable rewards.** The policy generates trajectories in an environment and receives a reward (tests passed, task solved). Advantages are computed by REINFORCE baselines, PPO with GAE, or GRPO's group-relative normalization (DeepSeekMath, DeepSeek-R1, Dr. GRPO). Multi-turn agent RL adds credit assignment across tool calls and requires masking tool-output tokens.
- **RL systems.** Rollouts dominate cost, so asynchronous producer–consumer training with off-policy corrections (importance weighting) is standard. CMU A3 makes you build both sync and async GRPO on Tinker with a SkyRL server.
- **Optimizing without gradients.** DSPy compiles LM programs. TextGrad, GEPA, ADAS and Promptbreeder evolve prompts or whole agent designs (UW weeks on optimizing and self-improving agents).

Curated learning sequence

1. CMU 8 *Training 1: SFT* — Supervised fine-tuning for agents (slides + video).
2. CMU 9 *Training 2: RL Basics* — RL basics: ReST, ReST-EM, DeepSeekMath/GRPO, Dr. GRPO (slides + video).
3. CMU 11 *Training 3: Advanced RL Algorithms* and CMU 12 *Training 4: RL Systems* — Advanced RL algorithms, then RL systems (slides).
4. UVA 13 *Topic 7 cont. (A2A) + Topic 8: LLM Fine-Tuning* — Fine-tuning versus RAG, QLoRA, the Tinker pipeline, text-to-SQL.
5. JHU 11 *Pre-training of Coding Language Models (1)*–JHU 13 — Pre- and post-training of code LLMs.
6. UW-Madison 13 *Optimizing agents I* and UW-Madison 14 *Optimizing agents II* — DSPy/TextGrad, then ADAS/GEPA.
7. Supplement: TinyAgent — An end-to-end distillation case study.
8. Stanford 329Z 9 *Optimization*, Stanford 329Z 11, Stanford 329Z 12 — Prompt and weight optimization (GEPA, MIPROv2, SFT, PPO, GRPO), then what data agents need and how to select it.

Lecture map (33 lectures)

Course	#	Title (links to catalogue row)	Materials	Tags
CMU	8	⋆ Training 1: SFT	[slides] [video]	TRAIN
CMU	9	⋆ Training 2: RL Basics	[slides] [video]	TRAIN
CMU	11	⋆ Training 3: Advanced RL Algorithms	[slides] [video]	TRAIN
CMU	12	⋆ Training 4: RL Systems	[slides] [video]	TRAIN DEPLOY
Stanford 329Z	9	⋆ Optimization	[slides]	TRAIN
Stanford 329Z	11	⋆ What Data Do Agents Need?	[slides]	TRAIN EVAL
Stanford 329Z	12	⋆ Data Selection & Quality	[slides]	TRAIN EVAL
Northeastern	3	LLM Foundations (II)	[agenticllms.github.io] [GitHub]	FND TRAIN
UVA	13	⋆ Topic 7 cont. (A2A) + Topic 8: LLM Fine-Tuning	[cs.virginia.edu]	TRAIN MULTI
UVA	18	Topic 12: Personal Agents and Distilling	[cs.virginia.edu]	WEB SAFE TRAIN

Course	#	Title (links to catalogue row)	Materials	Tags
UW-Madison	13	⋆Optimizing agents I	–	ARCH TRAIN
UW-Madison	14	⋆Optimizing agents II	–	ARCH TRAIN
JHU	11	⋆Pre-training of Coding Language Models (1)	[PDF]	TRAIN CODE
JHU	12	⋆Pre-training of Coding Language Models (2)	[PDF]	TRAIN CODE
JHU	13	⋆Post-training of Coding Language Models: SFT & RL	[PDF]	TRAIN CODE
JHU	14	Special Topics on Coding Language Models	[PDF]	TRAIN CODE
CU Boulder	11	Reflexion & Toolformer (students)	–	RSN TOOL TRAIN
Mannheim	2	Post-training and Efficient Adaptation	[PDF] [notebook zip]	TRAIN FND
UChicago	5	Sampling, Fine Tuning, Benchmarking, and Tools	[GitHub]	TRAIN EVAL TOOL
UChicago	7	Reinforcement Learning to Optimize AI Agents and Institutions	[GitHub]	TRAIN RSN MULTI
Stanford 329A	4	Learning from Feedback with Tools/Code	[video]	TOOL CODE TRAIN
Stanford 329A	5	Multi-step Reasoning/Planning	[video]	RSN TRAIN
Stanford 329A	6	Train Time Scaling/Scaling RL	[video]	TRAIN RSN
Berkeley '25	2	LLM Agents Overview	[slides] [video]	FND TRAIN
Berkeley '25	4	Post-Training Verifiable Agents	[slides] [video]	TRAIN EVAL CODE WEB
Berkeley '25	6	Some Challenges and Lessons from Training Agentic Models	[slides] [video]	TRAIN
Berkeley '25	12	Autonomous Agents: Embodiment, Interaction, and Learning	[video]	APP TRAIN
Berkeley Adv.	2	Learning to reason with LLMs	[slides] [video]	TRAIN RSN
Berkeley Adv.	4	Open Training Recipes for Reasoning in Language Models	[slides] [video]	TRAIN RSN
Berkeley Adv.	8	AlphaProof: when reinforcement learning meets formal mathematics	[slides] [video]	RSN TRAIN APP
Stanford 329T	9	Deep dive into LLMs	[slides]	FND TRAIN
Stanford 329T	10	Data curation and RL to optimize agents (guest: Trung Vu, Bespoke Labs)	[slides]	TRAIN TOOL EVAL
Stanford 329T	11	GEPA (guest: Lakshya Agrawal)	[slides]	TRAIN RSN

Key readings

- Ouyang et al., *InstructGPT* (2022)
- Shao et al., *DeepSeekMath (GRPO)* (2024)
- DeepSeek-AI, *DeepSeek-R1* (2025)
- Singh et al., *ReST-EM* (2023)
- Hu et al., *LoRA* (2021); Dettmers et al., *QLoRA* (2023)
- Khattab et al., *DSPy* (2023); Agrawal et al., *GEPA* (2025)

Hands-on

- **CMU: A3: Agentic RL Training, "scottyrl"** — Fill in TODOs for REINFORCE/GRPO advantages, trajectory to masked token sequences, grouped rollouts, a sync trainer, and an async producer-consumer GRPO trainer. *Starter code:* [GitHub]
- **UVA: Topic 8: Text-to-SQL fine-tuning** — Fine-tune Llama on text-to-SQL with Tinker and score semantic SQL matches. *Starter code:* [cs.virginia.edu]
- **UChicago: Week 5 notebook** — 3 of 4 modules: sampling; LoRA/QLoRA fine-tuning with varied ranks plus CKA comparison; designing an agent benchmark; a tool-using agent and an MCP server. *Starter code:* [GitHub]
- **UChicago: Week 7 notebook** — Bandits (epsilon-greedy, annealing, UCB), RL for alignment, then RL for reasoning or multi-agent RL; memo pilot. *Starter code:* [GitHub]

Related supplements: Berkeley BAIR: TinyAgent, Function Calling at the Edge.

Module 12: Safety, Security and Oversight

Tag: SAFE

Learning objectives

- Threat-model an agent: prompt injection, tool misuse, data exfiltration, supply chain.
- Apply defenses: sandboxing, least privilege, capability separation, human approval, monitoring.
- Evaluate security with benchmarks such as AgentDojo.

Core concepts

An agent with tools is a confused-deputy risk. Simon Willison's **lethal trifecta** (UW reading) names the dangerous combination: access to private data, exposure to untrusted content, and an exfiltration channel.

- **Attacks.** Indirect prompt injection through web pages, documents, tool outputs or MCP servers. Jailbreaks. Malicious or compromised tools and packages. CMU's safety lecture works through real incidents: the OpenClaw incident report, a database-deletion incident, CVE-2025-32711, and the Cline supply-chain compromise.
- **Defenses.** Sandboxed execution and credential isolation; least-privilege tools and permission modes; human-in-the-loop approval for irreversible actions (Buffalo Lab 5); capability-based designs that separate control flow from untrusted data (CaMeL); guardrails; layered "Swiss cheese" safety.
- **Oversight.** Tracing, monitoring and anomaly detection (CMU Safety 2). Static analysis of agent-written code (Semgrep, Stanford weeks on security).
- **Alignment.** Sleeper agents, misalignment after narrow fine-tuning, representation engineering (UChicago week 9, UVA project ideas).

Curated learning sequence

1. CMU 13 *Safety 1: Agent Safety* — Agent safety through real incidents (slides).
2. UW-Madison 11 *Security & prompt injection I* and UW-Madison 12 *Security & prompt injection II* — Indirect prompt injection and AgentDojo, then CaMeL and browser-agent security.
3. Buffalo 10 *Security and privacy* — Prompt injection, tool misuse, sandboxing, privacy (then Lab 5).
4. Stanford '25 11 *AI QA, SAST, DAST, and beyond* — AI QA, SAST/DAST and prompt-injection RCE case studies (slides).
5. CMU 16 *Safety 2: Observability and Monitoring* — Observability and monitoring.
6. Northeastern 17 *Deep Agents II: Backends and Sandboxes* — Where a deep agent's files live, permission rules on file tools, and sandboxes (Daytona, then Docker).
7. Mannheim 9 *LLM Security and Safety (+ project coaching)* — LLM security and safety (full-stack safety survey).
8. Stanford 329Z 17 *Agent Safety & Guardrails* and Michigan 11 *Guardrails: Errors, Retries and Approval Gates* — Agent safety and guardrails, then errors, retries and approval gates inside a coding harness.

Lecture map (27 lectures)

Course	#	Title (links to catalogue row)	Materials	Tags
CMU	13	⋆Safety 1: Agent Safety	[slides] [video]	SAFE
CMU	16	⋆Safety 2: Observability and Monitoring	–	SAFE DEPLOY
Stanford '25	11	⋆AI QA, SAST, DAST, and beyond	[slides]	SAFE EVAL CODE
Michigan	5	Modern agentic CLIs	[slides] [podcast]	CODE ARCH SAFE
Michigan	9	Stop conditions and the approval layer	[slides]	ARCH SAFE
Michigan	11	⋆Guardrails: Errors, Retries and Approval Gates	[slides]	SAFE ARCH
Stanford 329Z	17	⋆Agent Safety & Guardrails	–	SAFE DEPLOY
Stanford 329Z	19	Proactive Agents	–	ARCH APP SAFE
Northeastern	17	⋆Deep Agents II: Backends and Sandboxes	[agenticllms.github.io] [GitHub]	SAFE ARCH MEM
Harvard	11	Advanced agent design II: agent safety, self-evolving harness	[cs2680.seas.harvard.edu] [cs2680.seas.harvard.edu]	SAFE ARCH
Buffalo	8	Human-agent interaction and memory	–	MEM SAFE
Buffalo	10	⋆Security and privacy	–	SAFE

Course	#	Title (links to catalogue row)	Materials	Tags
UVA	18	Topic 12: Personal Agents and Distilling	[cs.virginia.edu]	WEB SAFE TRAIN
UW-Madison	6	Coding agents II	–	CODE SAFE EVAL
UW-Madison	11	⋆Security & prompt injection I	–	SAFE EVAL
UW-Madison	12	⋆Security & prompt injection II	–	SAFE
JHU	17	PL Kaleidoscope III: Software Security and Static Analysis	[PDF]	SAFE CODE APP
JHU	18	PL Kaleidoscope IV: Software Security and Dynamic Analysis	[PDF]	SAFE CODE APP
CU Boulder	23	SafePro & ToolDreamer (students)	–	SAFE TOOL
CU Boulder	24	Swiss Cheese safety & MCP design (students)	–	SAFE TOOL
Mannheim	9	⋆LLM Security and Safety (+ project coaching)	[PDF]	SAFE
UChicago	6	Auto-Encoders, Interpretability, and Steering Agents	[GitHub]	SAFE FND
UChicago	9	Alignment, Ethics, Safety, and Novelty	[GitHub]	SAFE APP
Berkeley '25	13	Agentic AI Safety & Security	[video]	SAFE
Berkeley Adv.	5	Coding Agents and AI for Vulnerability Detection	[slides] [video]	CODE TOOL SAFE
Berkeley Adv.	12	Towards building safe and secure agentic AI	[slides] [video]	SAFE DEPLOY
Stanford 329T	12	Security for agents: adversarial attacks and guardrails (guests: Matt Fredrikson, Andy Zou)	[slides]	SAFE EVAL

Key readings

- Greshake et al., *Indirect Prompt Injection* (2023)
- Debenedetti et al., *AgentDojo* (2024)
- Debenedetti et al., *CaMeL: Defeating Prompt Injections by Design* (2025)
- Willison, *The lethal trifecta*
- Wang et al., *Full Stack Safety survey* (2025)
- Hubinger et al., *Sleeper Agents* (2024)

Hands-on

- **Buffalo: Lab 5: Human-in-the-Loop and Safety Boundaries** — Add human approval, structured outputs, guardrails or permission boundaries. *(spec only)*
- **Stanford '25: Week 6 — Scan and Fix Vulnerabilities with Semgrep** — Run Semgrep on the app, triage findings, fix at least 3 issues (SQLi, unsafe DOM, CORS, etc.). *Starter code:* [GitHub]
- **UChicago: Week 9 notebook** — Any 4 of 7 modules: safety judge, Constitutional AI, deception detection, moral geometry, automated red-teaming, AGrail guardrails. *Starter code:* [GitHub]
- **Stanford '26: Week 2 — Build an MCP Server for a Third-Party API** — FastMCP server over an OAuth 2.0 API with at least 3 composing tools, a write tool with dry-run, structured errors, readOnlyHint, full auth-code flow with refresh, and a protocol-level test. *Starter code:* [GitHub]
- **Michigan: Lab04: Reaching the Computer: Shell, MCP and the Browser** — The shell tool (inputs, code mode, long-running and interactive commands, deadlines, background runs); MCP (servers, calls, local vs remote, MCP vs CLI, loading tools on demand); browser tools (pages, pixels and accessibility trees, click by reference); prompt injection and the lethal trifecta. *Slides:* [GitHub]

Related supplements: OpenClaw documentation, MIT 6.566 guest lecture: AI Agent Security (Anish Athalye, April 2026), Anthropic: Building effective agent automations (Martin and Avilla, October 2026).

Module 13: Deployment, Efficiency and Operations

Tag: DEPLOY

Learning objectives

- Deploy an agent as a service and operate it (logging, tracing, cost tracking, fallbacks).
- Reduce cost and latency: caching, model routing, small or quantized models, code execution.
- Understand long-running and background agent architectures.

Core concepts

- **Shipping.** Northeastern deploys agents to Hugging Face Spaces and Render. Buffalo adds CI and observability. Stanford '25 covers SRE and incident response with agents.
- **Cost and latency.** Prompt and KV caching (keep a stable prefix), model routing and gateways (Stanford '26 week 10), fallbacks and spend policies (Northeastern's optimization session), shorter contexts through tool retrieval or code execution, and small or quantized models on device (TinyAgent, UVA Topic 1). Harvard CS 2680 breaks an agent's cost into turns per task, steps per turn, tokens per step and price per token, and treats completion time as a critical path (Harvard 10); its second half goes down into the serving stack (batching, scheduling, speculative decoding, prefix caching, quantization, routing).
- **Long-running agents.** Background and asynchronous agents (Stanford '26 week 8) and always-on gateways with channels, cron jobs and hooks (OpenClaw) need durable state, idempotent tools and resumability (checkpointing in Module 2 (The Agent Loop and Agent Architectures)).
- **Observability.** Traces of every step are the raw material for evaluation (Module 10 (Evaluating Agents)), debugging and safety monitoring (Module 12 (Safety, Security and Oversight)).

Curated learning sequence

1. Northeastern 5 *Tool-Calling* and Northeastern 6 *Short-term & Long-term Memory* — Deploy to Hugging Face Spaces, then Render.
2. Northeastern 19 *Agentic AI Optimization* — DSPy plus cost tracking, caching, fallbacks and spend policies.
3. Harvard 10 *Advanced agent design I: Cost and performance* — Where an agent's cost and latency come from and how to cut each part (slides and the cost-effective agents note).
4. Harvard 12–Harvard 24 — LLM serving, from LLM and GPU basics through batching and paging, speculative decoding, KV-cache, quantization and prefix caching to routing and load balancing. Slides through November 4 at the last update; later sessions have reading lists.
5. Stanford '26 15 *Background agents: launching tasks asynchronously* and Stanford '26 19 *Coding agents in big teams* — Background agents, then coding agents in big teams (gateways, routing, cost).
6. Stanford '25 17 *Incident response and DevOps* — Incident response and DevOps with agents (slides).
7. CMU 12 *Training 4: RL Systems* — RL systems: the systems side of agent training.
8. Supplements: OpenClaw, TinyAgent — Always-on gateway architecture; edge deployment.
9. Supplements: Anthropic agent automations, Notion Data Scout — Durable state and guardrails for a scheduled agent; operating a production data agent.

Lecture map (33 lectures)

Course	#	Title (links to catalogue row)	Materials	Tags
CMU	12	⋆ Training 4: RL Systems	[slides] [video]	TRAIN DEPLOY
CMU	15	Frameworks 2: Orchestration and Monitoring	[slides]	ARCH DEPLOY
CMU	16	Safety 2: Observability and Monitoring	–	SAFE DEPLOY
Stanford '26	15	⋆ Background agents: launching tasks asynchronously	[course site]	DEPLOY ARCH
Stanford '26	19	⋆ Coding agents in big teams (MCP portals, LLM gateways, org patterns, cost optimization, model routing)	[course site]	DEPLOY
Stanford '26	20	The Software Factory: self-running, self-improving software systems	[course site]	ARCH DEPLOY

Course	#	Title (links to catalogue row)	Materials	Tags
Stanford '25	9	How to build a breakthrough AI developer product	[slides]	CODE DEPLOY
Stanford '25	17	⋆Incident response and DevOps	[slides]	DEPLOY MULTI
Michigan	13	Browser Extensions and Assistants You Can Text	[slides]	WEB TOOL DEPLOY
Michigan	19	Running Claude Code Projects on Your Own Machine	[slides]	MULTI DEPLOY CODE
Stanford 329Z	5	Frameworks & Orchestration	[slides] [code]	ARCH DEPLOY
Stanford 329Z	17	Agent Safety & Guardrails	–	SAFE DEPLOY
Northeastern	2	LLM Foundations (I)	[agenticllms.github.io] [agenticllms.github.io] [video] [video]	FND DEPLOY
Northeastern	5	⋆Tool-Calling	[agenticllms.github.io] [agenticllms.github.io] [GitHub]	TOOL RSN DEPLOY
Northeastern	6	⋆Short-term & Long-term Memory	[agenticllms.github.io] [agenticllms.github.io] [GitHub]	MEM ARCH DEPLOY
Northeastern	19	⋆Agentic AI Optimization	–	DEPLOY FND
Harvard	10	⋆Advanced agent design I: Cost and performance	[PDF] [PDF] [cs2680.seas.harvard.edu]	DEPLOY ARCH
Harvard	12	⋆Efficient LLM serving basics I: intro to LLMs	[PDF] [cs2680.seas.harvard.edu]	FND DEPLOY
Harvard	13	⋆Efficient LLM serving basics II: intro to GPUs	[PDF] [cs2680.seas.harvard.edu]	DEPLOY FND
Harvard	16	⋆Assignments feedback and agent pitfalls, GPU programming (cont.)	[PDF] [PDF] [how-i-used-claude.cs2680.com]	CODE DEPLOY
Harvard	17	⋆Efficient LLM serving basics III: Batching, paging, and parallelism	[PDF] [cs2680.seas.harvard.edu]	DEPLOY
Harvard	19	⋆Efficient LLM serving: speculative decoding	[cs2680.seas.harvard.edu]	DEPLOY
Harvard	20	⋆Efficient LLM serving: KV-cache optimization and model architectures	[cs2680.seas.harvard.edu] [cs2680.seas.harvard.edu]	DEPLOY
Harvard	22	⋆Efficient LLM serving: pruning and quantization	[cs2680.seas.harvard.edu]	DEPLOY
Harvard	23	⋆Efficient agentic systems: scheduling and prefix caching	[cs2680.seas.harvard.edu]	DEPLOY
Harvard	24	⋆Efficient agentic systems: routing and load balancing	[cs2680.seas.harvard.edu]	DEPLOY
Buffalo	12	Coding agents, testing, CI, observability	–	CODE DEPLOY
Buffalo	13	Applied agents and responsible deployment	–	APP DEPLOY
UVA	1	Topic 1: Running an LLM	[cs.virginia.edu]	FND EVAL DEPLOY
CU Boulder	21	MemSkill & More with Less (students)	–	MEM DEPLOY
Berkeley '25	3	Evolution of system designs from an AI engineer perspective	[slides] [video]	ARCH DEPLOY
Berkeley '25	10	Practical Lessons from Deploying Real-World AI Agents	[slides] [video]	DEPLOY EVAL APP
Berkeley Adv.	12	Towards building safe and secure agentic AI	[slides] [video]	SAFE DEPLOY

Key readings

- Google, *SRE book, introduction*
- Anthropic, *Code execution with MCP* (2025)
- Erdogan et al., *TinyAgent* (2024)
- Gao & Peng, *More with Less* (2025)
- Anthropic, *Building effective agent automations* (2026)
- Notion, *How we built a personal data scientist for every Notion employee* (2026)

Hands-on

- **Northeastern: DADA1: The Trip Planner Agent** — A conversational day-trip planner with tools: get_weather (Open-Meteo), find_attractions/restaurants/transit (Tavily), find_events (Ticketmaster). *Starter code:* [GitHub]
- **UVA: Topic 1 portfolio: Running an LLM** — MMLU timing/accuracy study across models, quantization and hardware; chat agent with bounded context and restartability; optional MT-Bench. *Starter code:* [cs.virginia.edu]

- **Harvard: A3 Mads-Opt: Make every token count** — A harness competition. *Starter code:* [cs2680.seas.harvard.edu] [cs2680.seas.harvard.edu]
- **Michigan: Lab05: Building on Codex** — Example products built on a harness (Slack chat bot, PR review bot, website builder); the Codex app-server (clients, threads, turns and items, approvals in the other direction, steering and stopping a turn); custom tools vs MCP; a toy chat bot (fixbot) on Codex. *Slides:* [GitHub]

Related supplements: Anthropic Engineering: Code execution with MCP, OpenClaw documentation, Berkeley BAIR: TinyAgent, Function Calling at the Edge, Anthropic: Building effective agent automations (Martin and Avilla, October 2026), Notion: How we built a personal data scientist for every Notion employee (David Thomas, October 2026), Google Colab.

Module 14: Applications: Deep Research, Science and Society

Tag: APP

Learning objectives

- Understand how deep-research and scientific-discovery agents are built and evaluated.
- Scope an application agent for a bounded domain with a measurable outcome.

Core concepts

- **Deep research agents** combine search, browsing, reading and synthesis over many steps (STORM, PaperQA, OpenScholar, Tongyi DeepResearch, DR Tulu). They are evaluated on BrowseComp, DeepResearch Bench and LiveResearchBench.
- **Agents for science** range from AI Scientist and autonomous chemistry to the AI co-scientist, sequential falsification, HEPTAPOD for particle physics, and Lean-based mathematics (UVA's guest lecture).
- **Society.** Simulated subjects and digital doubles, mixed human–AI experiments, persuasion, the future of work (CMU), and human–agent interaction design (CMU Interaction 2).
- **Scoping your own.** Mannheim's nine project briefs and Buffalo's "useful bounded agent" spec are good templates: a narrow domain, explicit tools, a benchmark and a risk assessment.

Curated learning sequence

1. CMU 10 *Domains 3: Deep Research Agents* — Deep research agents (slides).
2. UW-Madison 7 *Deep research agents I*–UW-Madison 10 — Deep research, then agents for science (four sessions).
3. UVA 19 *Topic 13: Agentic AI for Science* and UVA 20 *Topic 14: Agentic AI for Mathematics and Verification* — Agentic AI for science, then mathematics and verification (Lean).
4. CMU 17 *Agents and the Future of Work* and CMU 19 *Interaction 2: Human-Agent Interaction* — The future of work; human–agent interaction.
5. UChicago AI Agents for Social Science and Society — The whole course, for agents as research instruments.

Lecture map (34 lectures)

Course	#	Title (links to catalogue row)	Materials	Tags
CMU	10	⋆Domains 3: Deep Research Agents	[slides] [video]	APP MEM
CMU	17	⋆Agents and the Future of Work	[slides]	APP EVAL
CMU	19	⋆Interaction 2: Human-Agent Interaction	–	APP EVAL
Stanford '25	15	End-to-end apps from a single prompt	[slides]	CODE APP
Stanford 329Z	19	Proactive Agents	–	ARCH APP SAFE
Buffalo	13	Applied agents and responsible deployment	–	APP DEPLOY
UVA	10	Topic 6: Vision-Language Models	[cs.virginia.edu] [GitHub]	WEB APP FND
UVA	19	⋆Topic 13: Agentic AI for Science	[cs.virginia.edu] [GitHub]	APP MULTI
UVA	20	⋆Topic 14: Agentic AI for Mathematics and Verification (guest: Prof. Kevin Sullivan)	[cs.virginia.edu]	RSN APP
UW-Madison	7	⋆Deep research agents I	–	APP MULTI
UW-Madison	8	⋆Deep research agents II	–	APP MEM EVAL
UW-Madison	9	⋆Agents for science I	–	APP
UW-Madison	10	⋆Agents for science II	–	APP MULTI
JHU	15	PL Kaleidoscope I: Formal Verification for Mathematics	[PDF]	APP CODE
JHU	16	PL Kaleidoscope II: Formal Program Verification	[PDF] [machine-programming.github.io]	APP CODE
JHU	17	PL Kaleidoscope III: Software Security and Static Analysis	[PDF]	SAFE CODE APP
JHU	18	PL Kaleidoscope IV: Software Security and Dynamic Analysis	[PDF]	SAFE CODE APP
JHU	19	PL Kaleidoscope V: Accelerated Computing	[PDF] [machine-programming.github.io]	APP CODE

Course	#	Title (links to catalogue row)	Materials	Tags
JHU	26	PL Kaleidoscope VI: Visual Programming	–	APP
JHU	27	Special Topics on Neurosymbolic Programming	–	RSN APP
UChicago	1	Deep Learning and Social Agents	[GitHub]	FND APP
UChicago	3	LLMs Prompted for Multi-Agent Simulation	[GitHub]	MULTI MEM APP
UChicago	4	Experimental Designs with AI Agent and Human Interactions	[GitHub]	APP EVAL
UChicago	8	Multi-Modal and Embodied Agents	[GitHub]	WEB RSN APP
UChicago	9	Alignment, Ethics, Safety, and Novelty	[GitHub]	SAFE APP
Stanford 329A	7	Open-Ended Evolution of Self-Improving Agents	–	ARCH CODE APP
Berkeley '25	9	AI Agents to Automate Scientific Discoveries	[video]	APP MULTI
Berkeley '25	10	Practical Lessons from Deploying Real-World AI Agents	[slides] [video]	DEPLOY EVAL APP
Berkeley '25	12	Autonomous Agents: Embodiment, Interaction, and Learning	[video]	APP TRAIN
Berkeley Adv.	8	AlphaProof: when reinforcement learning meets formal mathematics	[slides] [video]	RSN TRAIN APP
Berkeley Adv.	9	Language models for autoformalization and theorem proving	[slides] [video]	RSN APP TOOL
Berkeley Adv.	10	Bridging Informal and Formal Mathematical Reasoning (Berkeley page: "Advanced topics in theorem proving")	[slides] [video]	RSN APP
Berkeley Adv.	11	Abstraction and Discovery with LLM Agents	[slides] [video]	RSN ARCH APP
Stanford 329T	18	Final Project Fair	–	ORG APP

Key readings

- Shao et al., *STORM* (2024)
- Lála et al., *PaperQA* (2023)
- Asai et al., *OpenScholar* (2024)
- Lu et al., *The AI Scientist* (2024)
- Gottweis et al., *Towards an AI Co-Scientist* (2025)
- Park et al., *Generative Agents* (2023)

Hands-on

- **Mannheim: Project topic 2: QA Agents** — RAG (multi-)agent QA over web search that explains its evidence. *Resources:* [openai.com] [aicrowd.com]
- **Mannheim: Project topic 8: Wikipedia Article Generation** — RAG/agent system to generate Wikipedia articles, evaluated against real articles with rubrics. *(spec only)*
- **Buffalo: Final Team Project: Useful Bounded Agent** — Working agent for a bounded task. *(spec only)*
- **UChicago: Final project** — Agent project for social insight: at least 3 data types and 3 model types from 3 weeks, qualitatively validated, with an open repo. *(spec only)*

Related supplements: Notion: How we built a personal data scientist for every Notion employee (David Thomas, October 2026), CADSmith: multi-agent CAD generation with programmatic geometric validation (Barkley, Loghmani and Barati Farimani, 2026).

Course Catalogue

CMU: 11-768 AI Agents

Institution	Carnegie Mellon University (LTI) (Graham Neubig, Daniel Fried)
Term	Fall 2026
Website	https://www.cmu-agents.com/#/schedule
Level	Graduate; assumes you have trained neural LMs
Archetype	Systems + training (build, evaluate, RL-train)
Public materials	Slides: L1–14, L17–18; video: L1–14; code: A1–A3 public
Best for	The backbone course: harness engineering, evals, SFT/RL, safety, search.

The most complete public agents course in the collection, and the one this guide is built around. It goes from capabilities (tool use, long-context management, skills and memory, planning) to domains (coding, computer use, deep research), then to training (SFT, RL basics, advanced RL, RL systems), safety and observability, frameworks (OpenHands, LangGraph), interaction and search. The readings are taken from production systems: harness source code, provider KV-cache pricing pages and the MCP spec. The three assignments form one pipeline: build a harness (A1), evaluate an agent with an LLM judge (A2), then RL-train an agentic policy (A3). Slides for lectures 1–14, 17 and 18 are public on the schedule page, with recordings for lectures 1–9 there and for lectures 10–14 on YouTube.

Lectures

#	Title	Key topics	Readings	Materials	Tags
1	**Course Overview: What Is an Agent?**	Module: Intro & Capabilities	Toolformer; ReAct; Mini-SWE-Agent	slides video	ORG ARCH
2	**Capabilities 1: Tool Use**	Module: Intro & Capabilities	What Are Tools Anyway?; CodeAct; Toolformer; XGrammar (refs: OpenAI Chat Completions, HF chat templates, BFCL V4)	slides video	TOOL
3	**Capabilities 2: Context Management for Long-Context Agents**	Module: Intro & Capabilities	(refs: DistServe, Needle in a Haystack, Longformer)	slides video	MEM
4	**Capabilities 3: Skills and Memory**	Module: Intro & Capabilities	OpenHands Agent Skills; SkillsBench; MemGPT; AWM; ASI; ReasoningBank (refs: Mem0, Reflexion, ExpeL)	slides video	MEM
5	**Capabilities 4: Planning, Task Decomposition, Multi-Agent Coordination**	Module: Intro & Capabilities	Cursor Plan Mode; Least-to-Most; Decomposed Prompting; Code as Policies; SayCan; Plan-and-Act; Thinking vs. Doing; Calibrate-Then-Act; RAO; MACU	slides video	RSN MULTI
6	**Domains 1: Coding Agents**	Module: Domains	(refs: OLMo, StarCoder, Code Llama)	slides video	CODE
7	**Computer Use Agents**	Module: Domains; lecturer: JY Koh	WebArena; VisualWebArena; OSWorld; OpenCUA; CUA-Gym; MACU	slides video	WEB
8	**Training 1: SFT**	Module: Training; lecturer: Yueqi Song	(refs: DeepSeek-R1, Kimi, Nemotron)	slides video	TRAIN
9	**Training 2: RL Basics**	Module: Training	ReST; ReST-EM; DeepSeekMath; DrGRPO	slides video	TRAIN
10	**Domains 3: Deep Research Agents**	Module: Domains; lecturer: Akari Asai	OpenScholar; BrowseComp; DeepResearch Bench; Tongyi DeepResearch; DR Tulu; AgentIR	slides video	APP MEM
11	**Training 3: Advanced RL Algorithms**	Module: Training	(refs: InstructGPT, GAE, Let's Verify Step by Step, AReaL)	slides video	TRAIN
12	**Training 4: RL Systems**	Module: Training; lecturer: Apurva Gandhi	–	slides video	TRAIN DEPLOY

#	Title	Key topics	Readings	Materials	Tags
13	**Safety 1: Agent Safety**	Module: Agent Safety	(refs: OpenClaw incident report, CVE-2025-32711, Cline supply-chain postmortem)	slides video	SAFE
14	**Frameworks 1: OpenHands**	Module: Agent Frameworks; lecturer: Graham Neubig	OpenHands Software Agent SDK report; SWE-agent; OpenDevin CodeAct 1.0; Don't Sleep on Single-agent Systems; OpenHands-Versa (refs: HumanEval, SWE-bench, CodeR, MetaGPT)	slides video	ARCH CODE
15	**Frameworks 2: Orchestration and Monitoring**	Module: Agent Frameworks	Building effective agents (Anthropic); Thinking in LangGraph; Diagnosing a performance regression on Terminal-Bench with Docent; Hodoscope (refs: LangGraph workflows/persistence/interrupts/testing docs, LangSmith evaluation, OpenAI Agents SDK, AG2, DSPy, OpenHands SDK, Agent Skills spec, Embedding Atlas)	slides	ARCH DEPLOY
16	**Safety 2: Observability and Monitoring**	Module: Frameworks & Safety; lecturer: Eric Wallace	Securing our frontier model training environment (OpenAI)	–	SAFE DEPLOY
17	**Agents and the Future of Work**	Module: Interaction & Projects; lecturer: Zora Wang	How Well Does Agent Development Reflect Real-World Work?; GDPval; CollabSkill; Efficient Test-Time Adaptation through Human-AI Interaction; Future of Work with AI Agents (refs: How AI is transforming work at Anthropic, METR long-task horizon, Remote Labor Index, TheAgentCompany, Gym-Anything, Humans are Missing from AI Coding Agent Research, GPTs are GPTs, Canaries in the Coal Mine; 31 in all)	slides	APP EVAL
18	**Interaction 1: Multi-Agent Interaction**	Module: Interaction & Projects; lecturer: Saujas Vaduguru	Multiagent Debate; Multi-Agent Computer Use; Scaling Discovery through Test-Time Communication; SWEET-RL; Don't lie to your friends (collaborative self-play); Debating with More Persuasive LLMs (refs: MARL book, ChatDev, LangChain subagents, Recursive Agent Optimization, Asynchronous SWE Agents, Blackboard multi-agent system, GitSwarm, Generative Agents, Magentic Marketplace; 17 in all)	slides	MULTI
19	**Interaction 2: Human-Agent Interaction**	Module: Interaction & Projects; lecturer: Valerie Chen	–	–	APP EVAL
20	**Search 1: Reranking and Critic Models**	Module: Search & Inference	–	–	RSN EVAL
21	**Search 2: Tree Search**	Module: Search & Inference; lecturer: JY Koh	–	–	RSN

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
A1: Build an Agent Harness	A domain-agnostic ReAct `Agent` base class: prompt construction, tool-call loop, step limit, model-generated context compaction. On top of it, a `CodeAgent` (terminal/message tools, skill loader with progressive disclosure via `invoke_skill`) that fixes a chess-app bug and solves SWE-bench `django__django-15368` with and without compaction. Then a `ChessAgent` (play_move, stateless simulate_move, sandboxed run_python, chess skill) plus an observation A/B experiment. Uses Modal sandboxes, an OpenAI-compatible LLM (default DeepSeek-V4-Flash), uv, and pytest.	ReAct loops, tool dispatch, sandboxing, skills, context compaction, token accounting	[GitHub] (handout: [GitHub]); chess app: [GitHub]	ARCH TOOL CODE
A2: Evaluating a Data-Visualization Agent	A `validate(run)` LLM-as-judge validator that flags four error families (execution_failure, wrong_data, wrong_chart, hard_to_read), scored by per-family and macro MCC against human labels. Students author 5+ new tasks, run 3 fixed agents (15+ runs), human-label the runs, and package the tasks as Harbor environments with deterministic tests, a reference solution, and two mutants. The judge is fixed to Qwen3-VL-30B on Modal.	Eval design, LLM-as-judge, human labeling, verifier robustness, benchmark packaging	[GitHub]	EVAL
A3: Agentic RL Training, "scottyrl"	Fill in TODOs for REINFORCE/GRPO advantages, trajectory to masked token sequences, grouped rollouts, a sync trainer, and an async producer-consumer GRPO trainer. Tasks: Number Search (sync and async), ablations (group size 2, stale-4 async with and without importance weighting), and async GRPO on TextCraft-Synth. The final adapter is evaluated against the base model (Qwen/Qwen3.5-4B) on medium and hard validation splits through a standalone inference-only vLLM server on Modal (`serve_evaluation.py`, LoRA adapter, added Oct 26), then exported to Hugging Face. Uses Tinker 0.24.0 with a SkyRL Tinker server on Modal and vLLM/SGLang.	Policy-gradient RL for multi-turn agents, async RL systems, ablation methodology	[GitHub]	TRAIN

Stanford '26: CS146S The Modern Software Developer

Institution	Stanford University (Mihail Eric)
Term	Fall 2026 (current)
Website	https://themodernsoftware.dev/
Level	Undergrad/grad; CS111/CS161-level programming
Archetype	Coding-agent practice (practitioner)
Public materials	Slides: Weeks 1–3 and most instructor sessions of weeks 4–8; video: none; code: Weeks 1–5 assignments
Best for	How production coding agents work and how to set up a repo and a team around them.

A practitioner course about building software with coding agents, with an industry guest in most weeks. The Fall 2026 version focuses on agents. Week 1 builds "Claude Code in 200 lines" and dissects production system prompts. Later weeks cover context engineering and MCP, agent skills, CLAUDE.md/AGENTS.md and hooks, agent-ready codebases, agentic code review, security, background agents, and the "software factory". The assignments are short and practical: dissect a real Claude Code trace, build an OAuth MCP server, and author an Agent Skill.

Lectures

#	Title	Key topics	Readings	Materials	Tags
1	**Course intro + build Claude Code in 200 lines**	Week 1: The Internals of Coding Agents	Building a Coding Agent; Lessons from Building Claude Code; Dive into Claude Code	course site	CODE ARCH
2	**How SOTA coding agents are designed: deep dive into system prompts**	Week 1: The Internals of Coding Agents	–	course site	CODE FND
3	**Advanced prompting + agentic dev frameworks (RePPIT, spec-driven development)**	Week 2: Advanced Context Engineering	The Context Tax: When to Compact; OpenSpec; Superpowers; Enterprise MCP (Cloudflare)	course site	FND MEM
4	**Full introduction to MCP and tool-calling (theory, setup, advanced tool design)**	Week 2: Advanced Context Engineering	–	course site	TOOL
5	**All about agent skills (incl. web skills)**	Week 3: Agent Skills and CLI	pstack; Matt Pocock's Skills; browse.sh	course site	MEM TOOL
6	**Guest: Lee Robinson**	Week 3: Agent Skills and CLI	–	course site	APP
7	**CLAUDE.md, AGENTS.md, hooks; subagent patterns (planner/implementer/reviewer)**	Week 4: Customizing Your Agent and Repository	I Still Prefer MCP Over Skills; AGENTS.md (Factory Docs); Steering Claude Code: Skills, Hooks, Rules, Subagents, and More; Writing a Good CLAUDE.md	course site	CODE MULTI
8	**Guest: Boris Cherny (Claude Code, Anthropic)**	Week 4: Customizing Your Agent and Repository	–	course site	CODE
9	**Agent readiness in your repos: structure, docs, tests, checks; scoring and auditing readiness**	Week 5: Agent-Ready Codebases	Making Codebases Agent Ready; Making Ergonomic Repositories	course site	CODE EVAL
10	**Guest: Eno Reyes (Factory)**	Week 5: Agent-Ready Codebases	–	course site	CODE
11	**Agentic code review: best practices and architectures**	Week 6: Agentic Code Review	Layered Code Review and Risk-Based Human Oversight; Coordinating Specialist Review Agents at Scale; Beyond the Diff: Context, Testing, and Model Selection	course site	CODE EVAL
12	**Guest: Silas Alberti (Cognition)**	Week 6: Agentic Code Review	–	course site	CODE

#	Title	Key topics	Readings	Materials	Tags
14	**Guest: Isaac Evans (Semgrep)**	Week 7: Security	–	course site	SAFE
15	**Background agents: launching tasks asynchronously**	Week 8: Background Agents	Spotify: From Migration Scripts to Agent Fleets; Stripe: From Slack Message to Review-Ready PR; Ramp: Giving Agents the Tools to Prove Their Work; LangChain: Inside an Open-Source Software Factory	course site	DEPLOY ARCH
16	**Guest: Rajesh Bhatia (Cloudflare)**	Week 8: Background Agents	–	course site	DEPLOY
17	**Guest: Elad Gil**	Week 9: Building an AI-Native Team	–	course site	APP
18	**Guest: Amjad Masad (Replit)**	Week 9: Building an AI-Native Team	–	course site	APP
19	**Coding agents in big teams (MCP portals, LLM gateways, org patterns, cost optimization, model routing)**	Week 10: Software Factory + Future	–	course site	DEPLOY
20	**The Software Factory: self-running, self-improving software systems**	Week 10: Software Factory + Future	–	course site	ARCH DEPLOY

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
Week 1 — Trace Dissection of a Real Claude Code Session	Capture a Claude Code session through a mitmproxy reverse proxy, then annotate the system prompt, `<system-reminder>`s, tool inventory and schemas, error recovery, planning, subagents, and context management	Reverse-engineering a production coding agent, prompt and tool design analysis	[GitHub]	CODE ARCH MEM
Week 2 — Build an MCP Server for a Third-Party API	FastMCP server over an OAuth 2.0 API with at least 3 composing tools, a write tool with dry-run, structured errors, readOnlyHint, full auth-code flow with refresh, and a protocol-level test	Agent-ergonomic tool design, OAuth, MCP	[GitHub]	TOOL SAFE
Week 3 — Author a Skill	A SKILL.md (Agent Skills standard) for a repeatable workflow on a real OSS repo (htmx, dayjs, yt-dlp, etc.), with trigger tests including a near-miss and an end-to-end run	Skill authoring, trigger description design, eval of triggering	[GitHub]	CODE MEM EVAL
Week 4 — Make a Messy Repo Agent-Ready	Write the missing AGENTS.md or CLAUDE.md for an unfamiliar OSS repo (htmx, dayjs, yt-dlp, etc.): what the project is, architecture, exact build/test/lint commands, conventions and guardrails, every line specific to the repo and about a page long; then run two small agent tasks with it and report one change	Repo orientation, agent context files, judging what belongs in always-loaded context	[GitHub]	CODE MEM
Week 5 — Agent-Readiness Report	Run Factory's Droid `/readiness-report` (eight pillars, five maturity levels) on three different repos, record the results in a comparable format, explain which pillars fail and why, pick the cheapest high-leverage fix per repo, and critique the framework	Assessing repos against a readiness framework, comparative analysis, scoping claims to a small sample	[GitHub]	CODE EVAL

Stanford '25: CS146S The Modern Software Developer (archive)

Institution	Stanford University (Mihail Eric)
Term	Fall 2025
Website	https://themodernsoftware.dev/fall2025
Level	Undergrad/grad
Archetype	Coding-agent practice (practitioner)
Public materials	Slides: Most lectures (Google Slides); video: none; code: 8 weekly assignments (branch fall2025)
Best for	Slide decks on prompting, building a coding agent, MCP servers, AI testing/security, code review, SRE.

The previous offering of CS146S. Its value here is that nearly every lecture has a public slide deck, and the eight assignments (prompting playground, AI-IDE workflow, custom MCP server, Claude Code automations, Warp, Semgrep, Graphite review, multi-stack build) live on the `fall2025` branch of the assignments repository. The lecture dates come from an authorized translation and may be off by a day, so treat them as approximate.

Lectures

#	Title	Key topics	Readings	Materials	Tags
1	**Introduction and how an LLM is made**	What LLMs are	Karpathy "Deep Dive into LLMs" [video] ; "How OpenAI Uses Codex" [PDF]	Slides [slides]	ORG FND
2	**Power prompting for LLMs**	Effective prompting techniques	Google "What is prompt engineering" [cloud.google.com] ; Prompting Guide [promptingguide.ai] ; "AI Prompt Engineering: A Deep Dive" [video]	Slides [slides]	FND RSN
3	**Building a coding agent from scratch**	Agent architecture and components; tool use / function calling	Week 2 readings (MCP, below)	Slides [slides] ; completed exercise [drive.google.com]	ARCH TOOL CODE
4	**Building a custom MCP server**	Model Context Protocol	MCP intro [stytch.com] ; MCP servers [GitHub] ; Cloudflare remote MCP auth [developers.cloudflare.com] ; MCP TS SDK [GitHub] ; MCP Registry [blog.modelcontextprotocol.io] ; "APIs don't make good MCP tools" [reillywood.com]	Slides [slides] ; completed exercise [drive.google.com]	TOOL
5	**From first prompt to optimal IDE setup**	Context management and code understanding; PRDs/specs for agents; IDE integrations	"Specs Are the New Source Code" [blog.ravi-mehta.com] ; "How Long Contexts Fail" [dbreunig.com] ; Devin Agents 101 [devin.ai] ; HumanLayer ACE-FCA [GitHub] ; "How FAANG Vibe Codes" [x.com] ; Anthropic "Writing effective tools for agents" [anthropic.com]	Slides [slides] ; design doc template [drive.google.com]	MEM CODE
6	**Guest: Silas Alberti (Cognition)**	Coding agents in industry (Devin)	(week 3 readings)	Slides [slides]	CODE
7	**How to be an agent manager**	Managing levels of agent autonomy; human-agent collaboration patterns	"How Anthropic Uses Claude Code" [PDF] ; Claude Code best practices [anthropic.com] ; awesome-claude-agents [GitHub] ; SuperClaude [GitHub] ; "Good Context Good Code" [blog.stockapp.com] ; "Peeking Under the Hood of Claude Code" [medium.com]	Slides [slides]	CODE MULTI
8	**Guest: Boris Cherny (creator of Claude Code, Anthropic)**	Claude Code	(week 4 readings)	Slides [slides]	CODE

#	Title	Key topics	Readings	Materials	Tags
9	**How to build a breakthrough AI developer product**	AI-enhanced CLIs; terminal automation and scripting	Warp University [warp.dev] ; Warp vs Claude Code [warp.dev] ; "How Warp uses Warp to build Warp" [notion.warp.dev]	Slides [slides]	CODE DEPLOY
10	**Guest: Zach Lloyd (CEO, Warp)**	The agentic terminal	(week 5 readings)	Slides (Figma) [figma.com]	CODE
11	**AI QA, SAST, DAST, and beyond**	Secure vibe coding; history of vulnerability detection; AI-generated test suites	SAST vs DAST [splunk.com] ; Copilot RCE via prompt injection [embracethered.com] ; Semgrep, finding vulns with Claude Code and Codex [semgrep.dev] ; Unit42 agentic AI threats [unit42.paloaltonetworks.com] ; OWASP Top Ten [owasp.org] ; Chroma "Context Rot" [research.trychroma.com] ; o3 CVE-2025-37899 prompt [GitHub]	Slides [slides]	SAFE EVAL CODE
12	**Guest: Isaac Evans (CEO, Semgrep)**	Static analysis and AI security	(week 6 readings)	n/a (no slides listed)	SAFE
13	**AI code review**	Which AI code systems to trust; debugging and diagnosis; generated documentation	"Code Reviews: Just Do It" [blog.codinghorror.com] ; GitHub "How to review code effectively" [github.blog] ; "AI-Assisted Assessment of Coding Practices in Modern Code Review" [arXiv:2405.13565] ; Graphite AI code review best practices [graphite.dev] ; "Code Review Essentials" [blakesmith.me] ; "Lessons from millions of AI code reviews" [video]	Slides [slides]	CODE EVAL
14	**Guest: Tomas Reimers (CPO, Graphite)**	AI code review in practice	(week 7 readings)	Slides [drive.google.com]	CODE
15	**End-to-end apps from a single prompt**	Design and frontend for everyone; rapid UI/UX prototyping	n/a (none listed)	Slides [slides]	CODE APP
16	**Guest: Gaspar Garcia (Head of AI Research, Vercel)**	AI app generation (v0)	–	Slides [slides]	CODE APP
17	**Incident response and DevOps**	Monitoring and observability for AI systems; automated incident response; triage and debugging	Google SRE book intro [sre.google] ; observability basics [last9.io] ; Resolve.ai posts [resolve.ai] , [resolve.ai] , [resolve.ai] , [resolve.ai]	Slides [slides]	DEPLOY MULTI
18	**Guests: Mayank Agarwal (CTO) and Milind Ganjoo, Resolve.ai**	Agentic on-call/SRE	(week 9 readings)	Slides [drive.google.com]	DEPLOY
19	**Software development in 10 years**	The future of developer roles; emerging AI coding paradigms	–	–	CODE ORG
20	**Guest: Martin Casado (General Partner, a16z)**	Industry trends and predictions	–	–	APP ORG

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
Week 1 — Prompting Techniques ("LLM Prompting Playground")	Fill in prompts for K-shot, chain-of-thought, tool calling, self-consistency, RAG, and reflexion until the test scripts pass, using local Ollama models (mistral-nemo:12b, llama3.1:8b)	Prompt design, iterative refinement	[GitHub]	FND RSN TOOL

Name	What you build	Skills	Starter code / resources	Tags
Week 2 — Action Item Extractor ("First Steps in the AI IDE")	Extend a FastAPI and SQLite notes app with LLM structured-output extraction, tests, a refactor, frontend endpoints, and a generated README, using Cursor	AI-IDE workflow, LLM integration, testing	[GitHub]	CODE FND
Week 3 — Build a Custom MCP Server	MCP server wrapping a real external API: at least 2 typed tools, robust error and rate-limit handling, STDIO or remote HTTP, optional OAuth2	MCP tools/resources/prompts, transport, auth	[GitHub]	TOOL
Week 4 — The Autonomous Coding Agent IRL ("Coding with Claude Code")	At least 2 Claude Code automations (slash commands, CLAUDE.md, subagents, MCP) used to extend a FastAPI app	Claude Code customization, subagents	[GitHub]	CODE MULTI
Week 5 — Agentic Development with Warp	Warp Drive prompts, rules, or MCP integrations, plus multi-agent concurrent sessions working from a task list	Agentic terminal workflows, parallel agents	[GitHub]	CODE MULTI
Week 6 — Scan and Fix Vulnerabilities with Semgrep ("Writing Secure AI Code")	Run Semgrep on the app, triage findings, fix at least 3 issues (SQLi, unsafe DOM, CORS, etc.)	SAST, secrets and dependency scanning, remediation	[GitHub]	SAFE CODE
Week 7 — Exploring AI Code Review Using Graphite ("Code Review Reps")	Implement 4 tasks with one-shot agent prompts, open PRs, and compare manual review with Graphite Diamond AI review	AI code review, PR hygiene	[GitHub]	CODE EVAL
Week 8 — Multi-Stack AI-Accelerated Web App Build	Build the same CRUD app in 3 stacks, at least one with bolt.new and one with a non-JS language	AI app generation, stack comparison	[GitHub]	CODE APP

Michigan: EECS 498-016 Applied Agentic Software Engineering

Institution	University of Michigan (Marcus Darden)
Term	Fall 2026 (in progress)
Website	https://eecs498-aase.github.io/
Level	Upper-level undergrad; EECS 281 and 201 (no ML background assumed)
Archetype	Coding-agent internals (use, dissect, extend a harness)
Public materials	Slides: L1–11, L13–19 (PDF, PPTX, Slidev); video: none public; code: Lecture demos and EGM skills; project repos private
Best for	Taking real coding-agent harnesses (Aider, Codex, Claude Code) apart, then extending one.

Teaches software engineering with coding agents in three phases: use one, take it apart, build on one. *Apply* drives Aider with a small local model and has students specify and build their own terminal pair-programmer, diagnosing sessions through context, model and prompt. *Analyze* opens production harnesses (Aider, Codex, Claude Code): the request and stream on the wire, the tool loop and stop conditions, approvals and guardrails, skills, shell, MCP and browser tools, prompt injection and subagents. The second project extends a copy of Aider with a skills loader, a model-driven tool loop, an approval-gated shell tool and an MCP client, each designed first in a design document reviewed by a fresh-context session. *Create* builds on an existing harness: compaction and prompt caching, agent memory, knowledge bases, multiple model providers, and coordinator-and-thread setups, with a lab on the Codex app-server. When this guide was last updated (November 11, 2026), lectures 20–28 had no published topics.

Lectures

#	Title	Key topics	Readings	Materials	Tags
1	**Course intro + the AI coding landscape**	Course shape (Apply, Analyze, Create); a brief history of AI coding; "code curator" vs typist; first look at the Big Three; why Aider first; AI pair-coding loop	–	slides demo podcast	ORG CODE
2	**How LLMs Code (Under the Hood)**	Tokenization; the context window and its costs; attention; context drift; hallucination as statistical priors; capability vs knowledge; matching model to task; a diagnostic for wrong output	Optional: "Let's build the GPT Tokenizer" [video] ; "Attention in transformers, step-by-step" [video]	slides podcast	FND
3	**The Big Three: Context, Model, Prompt**	Three-lever diagnostic for AI coding sessions; surgical context and where tokens go; choosing a model; specifying intent; live demo of context as an engineering decision	–	slides demo podcast	CODE FND
4	**Spec-Driven Development**	Information-dense keywords; high/mid/low prompt levels; plan = prompt; spec anatomy (five sections); Aider architect mode; common spec mistakes; the seven pair-programmer features	–	slides demo podcast	CODE RSN
5	**Modern agentic CLIs**	How Aider, the model and the repo interact; the five-rung autonomy gradient; four primitives that make a tool agentic; one request run one rung up, live; what the harness still decides (tool menu, permissions, stop reasons, traces)	–	slides podcast	CODE ARCH SAFE

#	Title	Key topics	Readings	Materials	Tags
6	**From pair-coding to agent orchestration**	The pair-programmer project; understanding what the model writes; decision codes and naming in sessions; Claude Code in a repository; from edit blocks to tool calls: READ and RUN commands, a dispatcher, the tool-call shape, the loop that picks its step	–	slides	CODE TOOL ARCH
7	**Inside the Call: Messages, Streams, and Tool Calls**	The Chat Completions request field by field; what "compatible" promises; finish reasons and truncation; streaming events and failed streams; the tool definition and its token cost; the tool round trip; native calls vs a text protocol; the loop on the wire	–	slides	TOOL FND ARCH
9	**Stop conditions and the approval layer**	How a run ends; done signals; six stop conditions (step limit, token budget, wall clock, repeated identical call, consecutive tool errors); where each check sits; the approval step in the harness; four approval decisions; approval fatigue; rules from observed use	–	slides	ARCH SAFE
10	**Skills and the Elephant-Goldfish Model**	What a skill is (SKILL.md, frontmatter, body); four homes for instructions; skill vs tool; three levels of loading; one skill across harnesses; the Elephant (long design session and four-section design doc) and the Goldfish (fresh-context reviewers); the EGM skill set	Dave Rensin, "Elephants, Goldfish and the New Golden Age of Software Engineering" [drensin.medium.com]	slides EGM skills	TOOL CODE MEM
11	**Guardrails: Errors, Retries and Approval Gates**	Three kinds of failure; retry loops and timeouts; what an approval prompt does not show; Codex vs Aider approvals; approval gates: prefix rules, auto mode and auto-review, a typed classifier (Jev); safety categories; thresholds as code	Anthropic, "How we built Claude Code auto mode" (Mar 25, 2026); OpenAI, "Auto-review of agent actions" (Apr 30, 2026); Codex auto-review docs; TypeSafe's Jev docs (URLs not given in the deck)	slides	SAFE ARCH
13	**Browser Extensions and Assistants You Can Text**	One browser extension across harnesses (Claude in Chrome, the ChatGPT extension); permissions and signed-in risk; uses of browser automation; always-on assistants (OpenClaw, Hermes Agent): driving a harness from a program, channels as MCP servers, triggers, heartbeat, lasting memory, approving from a phone	code.claude.com/docs/en/chrome ; learn.chatgpt.com/docs/chrome-extension ; playwright.dev/docs/test-agents	slides	WEB TOOL DEPLOY
14	**Subagents**	What a long conversation costs; a task split into parts; what a subagent is and starts with; the Agent tool, the brief and the report; parallel subagents; fresh vs forked; reviewers defined in files; subagents vs agent teams, handoffs and swarms; a worktree per subagent	The Sources slide is an image; the notes say the links are in a content.md that is not published	slides	MULTI ARCH MEM
15	**Compaction and Prompt Caching**	A window that fills and four ways to handle it; how compaction runs; what a summary keeps and loses; compaction as a handoff; prompt caching: what it keeps, how it matches, what it costs, request order, what resets it; compaction through the cache	Claude Code docs on prompt caching and on what survives compaction; Anthropic, "Lessons from building Claude Code"; Anthropic and OpenAI prompt caching docs; Codex compaction prompt and Aider chat summarizer (source); Anthropic on effective context engineering (named in speaker notes; no URLs in text)	slides	MEM ARCH

#	Title	Key topics	Readings	Materials	Tags
16	**Agent Memory**	Creating and storing memories: what to keep, four kinds of note, modifying notes, when they are written; loading and retrieving: three loading levels, on-demand reads, searching older notes; notes that go wrong; facts vs methods	Claude Code memory docs; Claude API memory tool; Anthropic on effective context engineering; MemGPT (2023); Hermes Agent, OpenClaw and Codex memory source (named in speaker notes; no URLs in text)	slides	MEM
17	**Knowledge Bases**	A wiki the model keeps: three layers, one page from many sources, adding sources, asking, checking; finding relevant text; limits of vector search; retrieve vs compile; an open format; trust and freshness	Karpathy, LLM Wiki; Open Knowledge Format v0.2; Anthropic, contextual retrieval; Lewis et al., RAG (2020) (named in speaker notes; no URLs in text)	slides	MEM
18	**Many Models, One Harness**	One conversation in two provider formats; a common format with per-provider adapters; provider-specific reasoning; model profiles; choosing a model per job (planner and editor); effort as a dial; who picks the model; switching mid-task; session cost	Anthropic Messages API; OpenAI on prompt caching; RouteLLM paper; Aider source (named in speaker notes; no URLs in text)	slides	FND ARCH
19	**Running Claude Code Projects on Your Own Machine**	Limits of one terminal session; a coordinator that starts a thread (separate harness session) per job; a thread's life and what outlives it; Remote Control and the staff's rct tool on an always-on Mac mini; a thread's pull request; settings; limits of the beta	Claude Code docs on Projects and on Remote Control; the rct and excalive tools (named in speaker notes; no URLs in text)	slides	MULTI DEPLOY CODE

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
Aider lessons (aider-practice)	Sixteen guided Aider lessons in two parts, each a failing test: build a task manager's command line (taskr) on a small local model, then put a web front end on the same data	Driving a coding agent; context, model and prompt levers	n/a (private repository per student)	CODE
Apply build: pair-programmer (ai-assistant)	A terminal coding assistant built with Aider from a specification: F1 conversation, budget handling and streamed replies; F2 add/drop/list/clear context; F3 file rendering; F4 a tested edit-format prompt; F5 parsing with useful errors; F6 exact edits with preview and explicit approval; F7 one commit per accepted edit set with guarded undo; plus YAML configuration (API base, model ID, credential variable, budget, timeout, temperature) against an OpenAI-compatible Chat Completions contract. Deliverables include a system spec, increment specs, three diagrams (components, sequence, lifecycle), student-written tests and an AI_LOG.md	Spec-driven development, harness design, endpoint client, testing	n/a (private template supplying only documentation and development tooling)	CODE ARCH TOOL
Hackathon 1	One evening build: an AI extension to the supplied Taskr application, with a stronger hosted model (gpt-5.6-luna)	Building with a stronger model under time limits	n/a (event packet, private)	CODE APP
P2: Aider as an agent	Extend a private copy of Aider in four parts using Codex: a skills loader, a loop in which the model chooses tools, a shell tool behind approval rules, and a stdio MCP client. Each part gets an Elephant design document and a Goldfish review ledger before implementation	Skills, tool loops, approvals, MCP, design review in a fresh context	n/a (private aider-<uniqname> repository); EGM skills: [GitHub]	TOOL ARCH SAFE CODE
Hackathon 2	A chat app driven through the Codex app-server, after Lab05 (details TBD)	Building on an existing harness	–	CODE ARCH
Create project: build on an existing harness	Build on an existing harness; requirements are TBD on the site	Harness extension	–	CODE ARCH

Name	What you build	Skills	Starter code / resources	Tags
Lab00: Setup Lab	Serve a local model with Ollama (laptop, CAEN GPU or Great Lakes GPU) and point Aider at it	Local model hosting, Aider setup	[GitHub]	ORG CODE
Lab01: How Aider works	Take Aider apart as a harness: edit-shaped replies, what the model never touches, who chooses context, the repo map, the shared token budget, how the prompt is rebuilt each turn, architect mode; start designing the pair-programmer	Harness anatomy, context construction	n/a (lab slides: [GitHub])	CODE ARCH MEM
Lab02: Hackathon 1 post-mortem	Post-mortem of Hackathon 1 (published as PDF only; not read in detail)	Reflection on agent-assisted builds	–	CODE
Lab03: Where we are, and starting on Aider	Set up Codex on the U-M GPT Toolkit and a private copy of Aider; learn the codebase with the five EGM skills and a "peanuts" map; use Codex as a teacher; design skills for Aider (start of P2)	Reading a large codebase with an agent, skill design	[GitHub]; lab slides: [GitHub]	CODE TOOL
Lab04: Reaching the Computer: Shell, MCP and the Browser	The shell tool (inputs, code mode, long-running and interactive commands, deadlines, background runs); MCP (servers, calls, local vs remote, MCP vs CLI, loading tools on demand); browser tools (pages, pixels and accessibility trees, click by reference); prompt injection and the lethal trifecta	Tool design, MCP, browser agents, injection risk	n/a (lab slides: [GitHub])	TOOL WEB SAFE
Lab05: Building on Codex	Example products built on a harness (Slack chat bot, PR review bot, website builder); the Codex app-server (clients, threads, turns and items, approvals in the other direction, steering and stopping a turn); custom tools vs MCP; a toy chat bot (fixbot) on Codex	Building on an existing harness, app-server protocol	n/a (lab slides: [GitHub])	ARCH TOOL DEPLOY

Stanford 329Z: CS 329Z Engineering AI Agents

Institution	Stanford University (Diyi Yang, Michael Ryan, John Yang)
Term	Fall 2026 (in progress)
Website	https://cs329z.stanford.edu/
Level	Prereq CS224N/U/V or CS336, or NLP background
Archetype	Engineering discipline (build from scratch, then frameworks; eval-first)
Public materials	Slides: L1–13 so far; video: L1–2 (YouTube); code: HW1, HW2, framework notebook (MIT)
Best for	A harness built with no framework, then authoring and validating a benchmark task.

Teaches how to engineer agentic systems along the whole range from simple LLM pipelines to compound systems (models, retrievers, tools, optimizers) to autonomous agents: choosing and decomposing problems, selecting components, curating data, building evaluations and reasoning about trade-offs. Core components (RAG, tool use, the agent loop) are built from scratch first, then shown as abstractions in DSPy and LangGraph. The schedule runs from foundations and LLM internals through RAG, tool use and MCP, frameworks, design patterns, memory, multi-agent systems and optimization, then data for agents, evaluation and LLM-as-judge, safety and guardrails, and coding and proactive agents, with guest sessions on Terminal-Bench and SWE-bench. The first homework builds a harness with no framework (tools, a terminal sandbox, compaction, memory, human approval, guardrails, simulated-user evaluation). The second authors and validates a benchmark task in the Harbor framework and reviews classmates' tasks. When this guide was last updated (November 11, 2026), slides were posted through lecture 13 and the schedule for the remaining sessions was marked tentative.

Lectures

#	Title	Key topics	Readings	Materials	Tags
1	**Introduction — What Are Agentic Systems?**	Brief history of agents; anatomy of an agent (LLM core, reasoning, memory, tools, environment); compound AI systems; workflows vs. agents; challenges of reliability, training, safety, evaluation	Zaharia et al., "The Shift from Models to Compound AI Systems" (BAIR 2024); Weng, "LLM Powered Autonomous Agents" (2023); Kapoor et al., "AI Agents That Matter" (TMLR 2025)	slides, video	FND ARCH
2	**LLMs for Builders**	Decoding; attention (incl. linear and hybrid) and context length; pretraining to post-training; inference and test-time scaling; structured I/O and constrained decoding; context engineering	Vaswani et al., "Attention Is All You Need" (2017); Katharopoulos et al., "Transformers are RNNs" (ICML 2020); Rajasekaran et al., "Effective Context Engineering for AI Agents" (Anthropic 2025)	slides, video	FND MEM
3	**Retrieval-Augmented Generation (RAG)**	Indexing (chunking, contextual retrieval, RAPTOR, HNSW); BM25, DPR, hybrid search; reranking; ColBERT; agentic RAG (Self-RAG, Search-R1); failure modes; course project overview	Lewis et al., RAG (NeurIPS 2020); Karpukhin et al., DPR (EMNLP 2020); Khattab & Zaharia, ColBERT (SIGIR 2020)	slides	MEM
4	**Tool Use & Function Calling**	Toolformer; Gorilla and ToolLLM; REST to function calling to MCP; evaluating tool use (BFCL, ToolBench, τ-bench); failure modes	Schick et al., Toolformer (NeurIPS 2023); Patil et al., Gorilla (NeurIPS 2024); Model Context Protocol specification, rev. 2025-06-18	slides	TOOL EVAL
5	**Frameworks & Orchestration**	Data, system and evals pillars; live demos of DSPy (signatures, modules, optimizers) and LangGraph (state, nodes, edges); open-source harnesses (mini-SWE-agent, OpenHands, Pi); durable orchestration with Temporal; observability	Khattab et al., DSPy (ICLR 2024); LangChain, "LangGraph Overview"; Arena.ai, "HarnessTax" (2026)	slides, code	ARCH DEPLOY

#	Title	Key topics	Readings	Materials	Tags
6	**Agent Design Patterns & Scaffolds**	Workflows vs. agents; five composable workflow patterns; ReAct, Reflexion, CodeAct, ReWOO; context management (event logs, compaction, observation masking, context isolation); Pi harness case study	Yao et al., ReAct (ICLR 2023); Schluntz & Zhang, "Building Effective AI Agents" (Anthropic 2024); LangChain, "Context Engineering" (2025). Additional: Shinn et al., Reflexion (NeurIPS 2023); Wang et al., CodeAct (ICML 2024)	slides	ARCH RSN MEM
7	**Agent Memory Architectures**	Working, episodic, semantic, procedural memory; skill libraries, tiered hierarchies (MemGPT), reflective consolidation, memory maintenance; memory benchmarks; forgetting, privacy, over-personalization	Sumers et al., CoALA (TMLR 2024); Packer et al., MemGPT (2023); Park et al., Generative Agents (UIST 2023)	slides	MEM
8	**Multi-Agent Systems**	Single vs. multi-agent; orchestrator, swarm, hybrid; handoffs, message boards, A2A protocol; task ledgers, shared memory, worktree isolation; why multi-agent systems fail	Wu et al., AutoGen (COLM 2024); Cemri et al., "Why Do Multi-Agent LLM Systems Fail?" (NeurIPS D&B 2025); Kim et al., "Towards a Science of Scaling Agent Systems" (2025)	slides	MULTI
9	**Optimization**	Context optimization (bootstrapped few-shot, MIPROv2, OPRO, TextGrad, GEPA); weight optimization (SFT on agent rollouts, PPO, GRPO, on-policy distillation); agents as optimizers (Darwin Gödel Machine, autoresearch); prompts vs. weights	Agrawal et al., GEPA (ICLR 2026); Opsahl-Ong et al., MIPROv2 (EMNLP 2024); Shao et al., DeepSeekMath (2024)	slides	TRAIN
10	**Guest Lecture: Terminal-Bench (Ryan Marten, Alex Shaw, Laude Institute)**	Benchmarking how well agents accomplish real-world tasks in a terminal environment	Terminal-Bench project site (tbench.ai); Merrill et al., "Terminal-Bench" (ICLR 2026)	slides	EVAL CODE
11	**What Data Do Agents Need?**	Traces, demonstrations, feedback; data for optimization vs. evaluation; data flywheels; synthetic data; data from human-agent interaction	Shankar, "Data Flywheels for LLM Applications" (2024). Additional: Tan et al., LLMs for data annotation and synthesis survey (EMNLP 2024)	slides	TRAIN EVAL
12	**Data Selection & Quality**	Informative data; filtering and selection; tiny targeted benchmarks; annotation practice; quality assessment; datasets from agent traces	Yang et al., SWE-smith (NeurIPS D&B 2025); Shankar et al., "Who Validates the Validators?" (UIST 2024). Additional: Zhou et al., LIMA (NeurIPS 2023)	slides	TRAIN EVAL
13	**Evaluation Fundamentals & Benchmark Design**	Why evals are hard; 4-tuple framework (request, environment, stopping criteria, scorer); properties of good benchmarks; realistic scaffolding; reliability	Zhu et al., "Establishing Best Practices for Building Rigorous Agentic Benchmarks" (2025). Additional: Press, "How to Build Good Language Modeling Benchmarks" (2024); Polo et al., tinyBenchmarks (ICML 2024)	slides	EVAL
14	**Guest: Fireside Chat with Andrew Ng (hosted by Diyi Yang)**	Fireside chat; topics not stated on the site	–	–	FND
15	**LLM-as-Judge & Evaluation Infrastructure**	Three grader types; judge prompts; known biases; pairwise vs. pointwise; non-determinism (pass@k vs. pass^k); harness design; Anthropic's 8-step roadmap	Grace et al., "Demystifying Evals for AI Agents" (Anthropic 2026); Zheng et al., MT-Bench and Chatbot Arena (NeurIPS 2023); Ryan et al., AutoMetrics (ICLR 2026). Additional: Zhu et al., AutoLibra (ICLR 2026)	n/a (not yet posted)	EVAL
16	**Guest Lecture: SWE-bench (Carlos Jimenez, Alexander Wettig, Anthropic)**	Evolution and future of evaluations for coding capabilities and beyond	Jimenez et al., SWE-bench (ICLR 2024)	n/a (upcoming)	EVAL CODE

#	Title	Key topics	Readings	Materials	Tags
17	**Agent Safety & Guardrails**	Privacy risks of tool access; prompt injection incl. indirect; red-teaming; sandboxing and permissions; output guardrails; liability; responsible deployment; human-in-the-loop	Shao et al., PrivacyLens (NeurIPS D&B 2024); Zhang & Yang, privacy risks in LLM agents via simulation (2025); Li, "Agentic LLMs as Powerful Deanonymizers" (2026). Additional: Wen et al., contextualized privacy defense (2026); OpenAI, prompt injections (2025); Anthropic, Responsible Scaling Policy (2023)	n/a (upcoming)	SAFE DEPLOY
18	**Coding & Software Agents**	SWE-agent, Claude Code and OpenHands architectures; scaffolds as design decisions; SWE-bench and the 4-tuple framework in practice; future of software development with agents	Yang et al., SWE-agent (NeurIPS 2024); Wang et al., OpenHands (ICLR 2025). Additional: Anthropic, "Claude Code: Best Practices for Agentic Coding" (2025); Young, "Effective Harnesses for Long-Running Agents" (Anthropic 2025)	n/a (upcoming)	CODE ARCH
19	**Proactive Agents**	Reactive to proactive agents; General User Models (GUM); next action prediction; open-source proactive personal agents; privacy and trust; mixed initiative (when to initiate vs. wait)	Shaikh et al., "Creating General User Models from Computer Use" (UIST 2025). Additional: Shaikh et al., next action predictors (2026); Steinberger, OpenClaw (2025)	n/a (upcoming)	ARCH APP SAFE

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
HW1: Build an Agentic Harness	An internal assistant for a fictional company (Cardinal Energy) from scratch, with no agent frameworks, over a real corporate email archive and company documents. Part 1, LLM pipelines: single-call email prioritizer, daily digest, BM25 retriever from scratch, multi-hop RAG QA. Part 2, the agent: tool registry, text tool-call protocol, agent loop, pipelines and document retrieval as tools, sandboxed terminal, compaction to a context budget, memory across sessions, an approve/deny/redirect layer, guardrails for auto mode, and evaluation on LLM-simulated user personas scored by checks and a judge model. Deterministic and live test suites; a design memo on each "design decision" and field notes from using the agent	Workflows vs. agents, retrieval, tool calling, context management, memory, sandboxing, human-in-the-loop, simulated-user evaluation	[GitHub]	ARCH TOOL MEM SAFE EVAL
HW2: Evaluate an Agent (Building an Evaluation Task)	In the Harbor framework (Docker environments): run existing tasks from HumanEval, SWE-bench Verified and Terminal-Bench 2.1 and analyze agent trajectories; then author one original multi-turn task (problem statement, environment, verifier giving a reward in [0, 1], oracle solution), validate it against wrong and partial solutions, run five attempts of mini-SWE-agent with Gemini 3.8 Flash and report on validity, realism, reproducibility and difficulty. Peer review: review three classmates' tasks (instructions, environment, verifier, oracle, trajectories) with a template	Benchmark design, verifiers, reproducible environments, trajectory analysis, critical review	[GitHub] (linked from the site as assignment2-harness, which redirects)	EVAL CODE
Paper Video	A video presenting a recent agent paper: problem, how the system works, evidence, a critical take, and added value beyond the paper (run the code, walk through an example or a small experiment). Peer review of two classmates' videos	Reading and critiquing agent research	n/a (handout: [PDF])	ARCH EVAL

Name	What you build	Skills	Starter code / resources	Tags
Course Project	Design, build and evaluation of an agentic system tied to a course topic (retrieval, tool use, memory, multi-agent, optimization, data, evaluation, safety): an environment and data, methods, results against baselines, error analysis, a safety and ethics section, a reproducible repository and a live demo. Peer review of the demos	End-to-end agent engineering, evaluation design, reproducibility	–	ARCH EVAL SAFE

Northeastern: Agentic AI

Institution	Northeastern University (Khoury) (Caglar Yildirim)
Term	Fall 2026 (in progress)
Website	https://agenticllms.github.io/
Level	Gentle on-ramp (Python primer provided)
Archetype	Builder (LangChain/LangGraph stack)
Public materials	Slides: 3 HTML decks; video: none; code: Notebooks W02b–W10a, DADA1–DADA3
Best for	Best guided, notebook-by-notebook path from an LLM API call to a deployed LangGraph agent.

Pairs one seminal paper with the matching LangChain/LangGraph documentation in every session. One running example, the "Husky Tech" customer-support agent, grows week by week: a notebook prototype, then a project skeleton, then a Hugging Face Spaces deployment, a LangGraph rewrite, RAG, a self-grading retrieval loop, and then a plan-search-review-replan loop and agentic RAG behind its policy-search tool. Later sessions cover planning, agentic RAG, three multi-agent sessions (ending with handoffs and agent swarms), two on deep agents (a long-running harness built by hand and then with the `deepagents` library, then file backends, permissions and sandboxes), evaluation, and optimization with DSPy and cost controls. The third take-home (DADA3) builds a bug-fixing team for a GitHub repository (triage, then a fixer and a sandboxed verifier per bug, in parallel) and compares it with a single deep agent on hidden tests. The second (DADA2) is an ablation study of agentic RAG over SEC filings: retrieval strategies measured one at a time with recall@k and MRR, then a routed agent with failure diagnosis compared against a baseline agent on accuracy, calls and time. Sessions after F 11/13 were not posted when this guide was compiled. Which notebooks belong to which session is inferred: Tuesday notebooks may run one session ahead of or behind the lecture topic.

Lectures

#	Title	Key topics	Readings	Materials	Tags
1	**Introduction to Agentic AI**	Workflows vs agents; course setup (VS Code, Python primer)	Building Effective Agents (Anthropic) [core]; Rise and Potential of LLM Agents (arXiv 2309.07864); LangGraph "Workflows and agents"; LangChain "Philosophy"	[agenticllms.github.io] ; Python primer [colab.research.google.com]	ORG ARCH
2	**LLM Foundations (I)**	Transformers, autoregressive generation; run a local open-weight model (Ollama, Qwen 3.5); base vs instruction-tuned comparison	Attention Is All You Need [core]; GPT-1 "Improving Language Understanding"; videos: 3Blue1Brown LLMs visualized, Karpathy "Let's build GPT"	[agenticllms.github.io] ; walkthrough slides (sampling knobs: temperature, top_p, max_tokens) [agenticllms.github.io] ; [video] ; [video]	FND DEPLOY
3	**LLM Foundations (II)**	Instruction tuning/RLHF, CoT; LLM API basics; environment setup (uv, .env keys)	InstructGPT [core]; Chain-of-Thought (2201.11903); LangChain Models, Messages docs	[agenticllms.github.io] ; code W02b (W02b.1_LLM_API_Basics.ipynb, W02b.2_AgentBasics.ipynb) [GitHub]	FND TRAIN
4	**Simple Agent Architecture**	Tools and agent loop; short-term memory, structured output; agentui.py chat UI	Toolformer [core]; MRKL (2205.00445); LangChain Tools, Agents docs	[agenticllms.github.io] ; code W03a (W03a.1_ShortTerm_Memory.ipynb, W03a.2_Structured_Output.ipynb, agentui.py) [GitHub]	ARCH TOOL MEM
5	**Tool-Calling**	ReAct; Husky Tech customer support agent; notebook to project skeleton; deploy to HF Spaces (Gradio); tools-vs-model design rules	ReAct [core]; What Are Tools Anyway? (2403.15452); LangChain short-term memory, structured output docs	[agenticllms.github.io] ; slides [agenticllms.github.io] ; code W03b [GitHub]	TOOL RSN DEPLOY
6	**Short-term & Long-term Memory**	Memory in agents; LangGraph basics (state, nodes, edges, reducers, cycles); deploy to Render	Generative Agents [core]; MemGPT (2310.08560); Thinking in LangGraph, Graph API, Persistence docs	[agenticllms.github.io] ; slides "Why LangGraph?" [agenticllms.github.io] ; code W04a [GitHub]	MEM ARCH DEPLOY
7	**Context Management**	Lost-in-the-middle, context engineering, middleware; LangGraph messages/tools/state; support agent in LangGraph	Lost in the Middle [core]; Effective Context Engineering (Anthropic); LangChain context engineering, middleware docs	[agenticllms.github.io] ; code W04b [GitHub]	MEM ARCH

#	Title	Key topics	Readings	Materials	Tags
8	**Retrieval-Augmented Generation**	Chunking, embeddings, cosine similarity, vector store (dict, then Pinecone)	RAG (2005.11401) [core]; RAG Survey sec. 2-3 (2312.10997); LangChain Retrieval, Semantic search docs	[agenticllms.github.io] ; code W05a [GitHub]	MEM
9	**Reflection & Self-critique**	Self-grading retrieval loop, query rewriting; RAG inside the support agent; rate limits/cost	Reflexion [core]; Self-RAG; Self-Refine; LangGraph Graph API loops, unit testing docs	[agenticllms.github.io] ; code W05b [GitHub]	RSN MEM
10	**Planning & Task Decomposition**	Skill libraries, ToT, task decomposition; subgraphs, skills; planner -> per-step search -> review -> replan loop (exits: covered, out of replans, or no new searches), then put behind the support agent's search_ policy tool	Voyager [core]; Tree of Thoughts; HuggingGPT (skim); LangGraph Subgraphs, Skills docs	[agenticllms.github.io] ; code W06a (W06a.1_ Planning.ipynb, husky_ policy.md, support_tools.py) [GitHub]	RSN ARCH
11	**Agentic RAG**	Corrective/adaptive RAG, multi-source routing; per-question assess -> search -> grade (some add plan/replan); small-model comparison (Ollama qwen3.5:4b) on a hard question	Corrective RAG (2401.15884) [core]; Adaptive-RAG (2403.14403); Custom RAG agent, router knowledge base docs	[agenticllms.github.io] ; code W06b (W06b.1_Agentic_ RAG.ipynb, husky_ policy.md, support_tools.py) [GitHub]	MEM RSN
12	**Multi-Agent Orchestration I**	MAS failure modes; handoffs, routers; parallel search (Tavily + Wikipedia branches), static vs dynamic fan-out (LangGraph Send), parallel/dynamic/orchestration shapes ranked by coordination cost; deep research agent over a 244-paper file: orchestrator writes sub-questions, parallel workers, fan-in synthesis (map-reduce)	Why Do Multi-Agent LLM Systems Fail? (2503.13657) [core]; AutoGen; Anthropic multi-agent research system; LangChain Customer support with handoffs, Handoffs, Router docs	[agenticllms.github.io] ; code W07a (W07a.1_Parallel_ Search.ipynb, W07a.2_ Research_Agent.ipynb, research_tools.py, papers.json) [GitHub]	MULTI ARCH WEB
13	**Multi-Agent Orchestration II**	Role-based MAS; subagents, custom workflows; supervisor pattern: supervisor routes among a researcher (W07a team), a writer with a sentence-level citation check, and a reviewer (ReAct, accept/research/rewrite verdict); three requests, three paths; label two failures in your own trace with the MAST taxonomy	MetaGPT [core]; CAMEL (skim); LangChain Subagents, Subagents tutorial (personal assistant), Custom workflow docs	[agenticllms.github.io] ; code W07b (W07b.1_Supervisor_ Agents.ipynb, research_ team.py, research_tools.py, papers.json) [GitHub]	MULTI EVAL
14	**Multi-Agent Orchestration III**	MAS failures revisited; LangGraph runtime (Pregel), Studio; agent swarms (session page "Agent Swarms"): multi-agent patterns on slides, then the Husky Tech desk split into an orders agent and a returns agent, each its own graph, handing the customer over by tool call (tools returning a string, then a Command), then rebuilt with langgraph-swarm; midterm review	Why MAS Fail sec. 4-5 (re-read) [core]; Agents Are Not Enough (2412.16241, optional); LangGraph runtime, Studio docs	[agenticllms.github.io] ; code W08a (W08a.1_Support_ Swarm.ipynb, support_ tools.py) [GitHub]	MULTI ARCH
16	**Deep Agents I: The Agent Harness**	A harness built by hand around the Week 4 loop: a to-do list, files kept in graph state, a task tool that hands work to a subagent with a fresh context; then the same agent with LangChain's deepagents library (create_deep_agent), matching each part to the hand-written code; a lead starts three research subagents on the Week 7 literature-review task	Effective Harnesses for Long-Running Agents (Anthropic) [core]; Measuring AI Ability to Complete Long Tasks (2503.14499, skim); Deep Agents overview, Subagents docs; Deep Agents blog post	[agenticllms.github.io] ; code W09a (W09a.1_Agent_ Harness.ipynb, W09a.2_ Deep_Agents_ Library.ipynb, research_ team.py, research_tools.py, papers.json) [GitHub]	ARCH MULTI MEM
17	**Deep Agents II: Backends and Sandboxes**	Where a deep agent's files live (graph state, a store shared across threads, a folder on disk) and permission rules on the file tools; running the agent's code in a sandbox, first on Daytona, then in a local Docker container (optional)	Making Claude Code Safer with Sandboxing (Anthropic) [core]; Design Patterns for Securing LLM Agents against Prompt Injections (2506.08837, skim); Deep Agents Backends, Sandboxes docs	[agenticllms.github.io] ; code W09b (W09b.1_Backends_ and_Permissions.ipynb, W09b.2_Sandboxes.ipynb, docker_sandbox.py, papers.json) [GitHub]	SAFE ARCH MEM

#	Title	Key topics	Readings	Materials	Tags
18	**Agentic AI Evaluation**	Benchmarks, trajectory/intermediate-step eval, LLM-as-judge; the Husky Tech agent run on 16 fixed tasks and checked at three levels: outcome (code compares refunds and escalations with the expected ones), trajectory (code checks the tool calls) and reply (a model as judge, from a different model family); the judge is first checked against 18 human-labelled replies	SWE-bench [core]; tau-bench (2406.12045); LLM-as-a-Judge (2306.05685); AgentBench (skim); LangSmith Evaluation concepts, Evaluate intermediate steps, Complex agent, LLM-as-judge docs	[agenticllms.github.io] ; code W10a (W10a.1_Evaluating_Agents.ipynb, eval_harness.py, husky_env.py, eval_tasks.json, judge_calibration.json) [GitHub]	EVAL
19	**Agentic AI Optimization**	Prompt/program optimization (DSPy); cost tracking, caching, fallbacks, spend policies	DSPy (2310.03714) [core]; LangSmith cost tracking, caching, fallbacks, spend policies, insights docs	–	DEPLOY FND

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
DADA1: The Trip Planner Agent	A conversational day-trip planner with tools: get_weather (Open-Meteo), find_attractions/restaurants/transit (Tavily), find_events (Ticketmaster). It has memory-based plan revision, a structured Itinerary/DayPlan output, 8 test prompts, and a Gradio app deployed on Render. Judged on tool routing, honesty at data limits, sources and memory. Optional extra credit: a find_performer_events chained tool	Tool design, prompting, memory, structured output, deployment	[GitHub]	TOOL MEM DEPLOY
DADA2: Agentic RAG over SEC filings	An ablation study in four notebooks (no separate spec). (1) Index the Business, Risk Factors and MD&A sections of nine 10-K filings (NVIDIA, AMD, Micron; three fiscal years each; 27 files) into a Pinecone index with filterable metadata. (2) Measure retrieval strategies one at a time with recall@k and MRR on 14 specific and 10 comparative labelled questions: a similarity baseline, a metadata filter, multi-query with reciprocal rank fusion, cross-encoder reranking (Cohere rerank-v3.5), and a given BM25 hybrid. (3) Build a LangGraph agent with a router by question kind, a structured-data path for figures (XBRL financials instead of prose chunks), a diagnosis step that labels a failed search absent, wrong_words or incomplete, and two recovery paths (a stronger strategy, or a plan of per-company sub-questions). Routing and diagnosis use a "decision model" that returns a probability per option instead of text (Cloudflare Workers AI). (4) Compare a chat-model router with the decision-model router, and the student's agent with a given baseline "ladder" agent, on accuracy (chat-model judge), chat calls and seconds per question; a five-part write-up; a Gradio app deployed on Render. DADA2_00_Start_Here.ipynb says to change one thing per row and to report the conditions of each number	Retrieval evaluation (recall@k, MRR), metadata filtering, RRF, reranking, hybrid lexical retrieval, query routing, failure diagnosis, ablation design, cost accounting	[GitHub]	MEM EVAL RSN

Name	What you build	Skills	Starter code / resources	Tags
DADA3: A Bug-Fixing Team for a GitHub Repository	A team of agents that works through the ten open issues of a small pure-Python library (huskyutils; seven real bugs, two reports of documented behavior, one feature request; each docstring is the spec). Triage labels each issue with one structured-output call; each bug goes by Send to its own fixer-verifier pair, run in parallel. The fixer (the W04b loop, three tools, cannot run code) writes a fix and hands it over with transfer_to_verifier; the verifier is a deep agent with a sandbox backend (Daytona, or Docker) that sees the before and after copies but not the diff, writes its own test from the issue and docstring, and ends with accept_fix (with evidence) or transfer_to_fixer (with the failing test), up to MAX_ROUNDS. Code writes the maintainer report; a model writes only its summary. Notebook 03 compares the team with a given single-deep-agent baseline on hidden tests: issues resolved, triage accuracy, bugs fixed, false "verified" claims, non-bugs left unchanged, and cost (calls, tokens, time). A Gradio app deployed on Render. The spec says a baseline that matches the team at lower cost, shown with numbers, is a full result	Multi-agent handoffs, independent verification, sandboxed code execution, parallel fan-out, hidden-test evaluation, cost accounting	[GitHub]	MULTI CODE EVAL
Husky Tech Customer Support Agent (running in-class build, W03b-W10a)	A support agent with 4 tools and ticket filing. It is built as a notebook, then a project skeleton, then deployed to HF Spaces, then rewritten in LangGraph, then given RAG over husky_policy.md, then a reflection/self-grading loop, then a planning/replan loop and agentic RAG behind the search_policy tool (W06a-W06b), then split into an orders agent and a returns agent that hand off to each other (W08a), then evaluated on 16 fixed tasks with outcome, trajectory and judge checks (W10a)	Tool calling, LangGraph, RAG, reflection, planning, handoffs, evaluation, deployment	[GitHub] (also W04b, W05a, W05b, W06a, W06b, W08a, W10a)	TOOL ARCH MEM
Deep research agent (running in-class build, W07a-W09b)	A research agent over a file of 244 papers (papers.json; PAPER_SOURCE=live switches the same search_papers/get_paper tools to arXiv, optional Semantic Scholar key). Built as one agent, then an orchestrator-workers team with parallel fan-out and synthesis (W07a), then a supervisor coordinating researcher, citation-checking writer and gap-searching reviewer (W07b), then a deep agent with a to-do list, files and research subagents, by hand and with deepagents (W09a), then with file backends, permissions and sandboxed code (W09b)	Multi-agent orchestration, parallel fan-out/fan-in, supervisor routing, citation checking, agent harnesses, failure analysis	[GitHub] (also W07b, W09a, W09b)	MULTI ARCH EVAL
Final Project (research)	A research contribution: agentic mechanism -> experimental manipulation -> measured behavior (e.g., memory policy -> stale-action rate). Suggested scope: controlled experiments on benchmarks, LoRA fine-tuning of 4-9B open models, trajectory analysis, fixed-token-budget architecture comparisons. Anchor papers cover eval, fine-tuning, safety, tool use/MCP and memory	Research design, experimentation, evaluation	n/a (resources: [agenticllms.github.io])	EVAL TRAIN ORG

Harvard: CS 2680 Modern AI Systems: Agents and System Optimizations

Institution	Harvard University (SEAS, MaDSys) (Juncheng Yang)
Term	Fall 2026 (in progress)
Website	https://cs2680.seas.harvard.edu/
Level	Graduate systems course; CS61 plus a graduate systems course
Archetype	Agent systems and serving (use, build, optimize, serve)
Public materials	Slides: Part I and Part II through Nov 4; video: none (not recorded); code: A2–A4 starter archives (no license); A5 unreleased
Best for	Treating an agent as a system to measure: harness design, evaluation, and cost per solved task, then the serving stack underneath.

Follows one agentic system from the agent loop down through the serving stack. Part I uses a coding agent (Claude Code), then designs one: context management and tool design, multi-agent architecture and communication, memory and reliability, evaluation, and cost and performance. Part II, from October 19, covers LLM serving: LLM and GPU basics, batching, paging and parallelism, speculative decoding, KV-cache optimization, pruning and quantization, scheduling and prefix caching, and routing and load balancing. The annotated reading pages and the two reference notes (agent evaluation and benchmarks, cost-effective agents) are worth reading on their own. The five assignments take one system apart in steps: watch the agent, build the loop, make every token count, serve the model yourself, optimize the full stack. **Access:** assignments 2 and 3 call a course model API whose keys go to enrolled students only. Outside the course you can run assignment 2 only against your own OpenAI-compatible endpoint, and assignment 3's leaderboard scoring is not available. When this guide was last updated (November 11, 2026), slides were posted for Part I and for Part II through November 4; the later serving lectures, which run to December 2, had reading lists but no slides, and the assignment 5 spec had not been released. The Part II schedule has been reordered several times.

Lectures

#	Title	Key topics	Readings	Materials	Tags
1	**Course overview and logistics**	What an AI system is and how it differs (compute, non-determinism, memory, fast-moving models); what an agent is; agents as a systems problem; the agentic systems stack; what to optimize; observability	–	Slides [PDF]	ORG ARCH
2	**Agents use I: How Claude Code Works**	The agentic loop (model proposes, harness executes); permissions; context growth and compaction; memory (CLAUDE.md, rules, auto memory); core tools; extension points: skills, MCP, hooks, plugins	How I use LLMs (Karpathy) [video] ; Best practices for Claude Code [code.claude.com] ; Agentic Engineering Patterns [simonwillison.net] ; Agent Skills [anthropic.com] ; Code execution with MCP [anthropic.com] ; Guardrails Beat Guidance [arXiv:2604.11088] ; The Scaffolding Matters More Than the Interface [arXiv:2608.08654]	Slides [PDF] ; readings (10 entries) [cs2680.seas.harvard.edu]	CODE MEM TOOL
3	**Agents use II: 100x Engineer with Multi-agents**	Skills and plugin case studies (Superpowers, Caveman token compression); parallel agents and git worktrees; subagents, agent teams, dynamic workflows; long-running agents; "Software 3.0" and living with uncertainty; the delegation ladder	Claude Code agent teams [code.claude.com] ; Claude Code subagents [code.claude.com] ; Software is changing (again) [video] ; Building a C compiler with a team of parallel Claudes [anthropic.com] ; Research acceleration: the view inside OpenAI [openai.com] ; The Shift to Agentic AI [arXiv:2606.26959]	Slides [PDF] ; readings (9 entries) [cs2680.seas.harvard.edu]	CODE MULTI

#	Title	Key topics	Readings	Materials	Tags
4	**Agents design I: context management and tool design**	The ReAct loop in code; components of a coding agent; search and read tools (glob/grep, semantic search, LSP); edit tools (rewrite, diff, search-and-replace, AST); shell execution; tool design practices; prompting techniques; context engineering, compaction, five ways context fails; agent = model x harness	ReAct [arXiv:2210.03629] ; Building effective agents [anthropic.com] ; A practical guide to building agents (OpenAI) [openai.com] ; LLM Powered Autonomous Agents [lilianweng.github.io] ; Effective context engineering [anthropic.com] ; Lost in the Middle [arXiv:2307.03172] ; context rot [arXiv:2606.29718] ; Writing effective tools for agents [anthropic.com] ; Toolformer [arXiv:2302.04761]	Slides [PDF] ; readings (15 entries) [cs2680.seas.harvard.edu]	ARCH TOOL MEM
6	**Agents design II: multi-agent architecture, coordination, and communication**	Eight architectures (supervisor-worker, hierarchical, pipeline/DAG, blackboard, debate, market, swarm, dynamic topology); task decomposition and its failures; communication medium, channels and message format; multi-agent as a distributed-systems problem; A2A agent cards and task lifecycle; MCP vs A2A	AutoGen [arXiv:2308.08155] ; MetaGPT [arXiv:2308.00352] ; How we built our multi-agent research system (Anthropic) [anthropic.com] ; Towards a Science of Scaling Agent Systems [arXiv:2512.08296] ; Don't Build Multi-Agents (Cognition) [cognition.com]	Slides [PDF] ; readings (5 entries) [cs2680.seas.harvard.edu]	MULTI ARCH TOOL
7	**Agents design III: agent memory and reliability**	Memory design questions: when to write, when to use, how to retrieve, how to generalize, forgetting, valid time and provenance, ownership; evaluating memory; reliability: where a fault originates (task specification, harness, model, tool, environment); designing for failure	RAG [arXiv:2005.11401] ; MemGPT [arXiv:2310.08560] ; Where agents fail [arXiv:2509.25370] ; Why Do Multi-Agent LLM Systems Fail? [arXiv:2503.13657] ; agent failure taxonomy [arXiv:2607.05775] ; overthinking [arXiv:2502.08235]	Slides [PDF] ; readings (6 entries) [cs2680.seas.harvard.edu]	MEM ARCH
8	**Agent design IV: agent evaluation**	Evaluation objectives and metrics; the benchmark evaluation loop; GAIA, BFCL, WebArena, OSWorld, SWE-bench, tau-bench, Terminal-Bench, TheAgentCompany; data leakage across the agent stack; saturation	Course note: Agent evaluation and benchmarks [cs2680.seas.harvard.edu]	Slides [PDF] ; readings, not linked from the home page (14 entries) [cs2680.seas.harvard.edu]	EVAL
9	**Guest lecture: Developing and Evaluating Agents in Enterprise Setting (Harsh Nilesh Pathak, Snowflake)**	Guest lecture; no materials posted	–	–	EVAL DEPLOY
10	**Advanced agent design I: Cost and performance**	Where cost comes from (turns per task, steps per turn, tokens per step, price per token); lean context, shorter output, caching and batching, model routing; budget caps; "do you need an LLM at all?"; latency as a critical path: parallel and speculative actions; three harness architectures (monolithic, modular, microkernel) across eight open harnesses	Course note: Cost-effective agents [cs2680.seas.harvard.edu]	Slides [PDF] ; Assignment 3 slides [PDF] ; readings, not linked from the home page (4 entries) [cs2680.seas.harvard.edu]	DEPLOY ARCH

#	Title	Key topics	Readings	Materials	Tags
11	**Advanced agent design II: agent safety, self-evolving harness**	Student-led paper discussion (held). Reading page groups: agent safety I: attacks (OWASP agentic top 10, indirect prompt injection, AgentPoison, multi-agent code execution, adaptive attacks); agent safety II: defenses (instruction hierarchy, CaMeL, securing agents against prompt injection, AI control, AgentDojo); self-evolving harnesses I: improving the agent from its own experience (Reflexion, Self-Refine, ADAS, Darwin Gödel Machine, ACE); self-evolving harnesses II: automated harness engineering (Meta-Harness, agentic harness engineering, Live-SWE-agent)	OWASP agentic top 10 (linked from the home page) [genai.owasp.org] ; Indirect prompt injection [arXiv:2302.12173] ; AgentDojo [arXiv:2406.13352] ; Design Patterns for Securing LLM Agents against Prompt Injections [arXiv:2506.08837] ; CaMeL [arXiv:2503.18813] ; Reflexion [arXiv:2303.11366] ; Self-Refine [arXiv:2303.17651]	Discussion guide [cs2680.seas.harvard.edu] ; readings (18 entries; linked from the home page by 2026-10-18) [cs2680.seas.harvard.edu]	SAFE ARCH
12	**Efficient LLM serving basics I: intro to LLMs**	The life of a request (tokenize, schedule and batch, prefill, decode and sampling, detokenize and stream; time to first token vs tokens per second); the decoder block (embedding, RMSNorm, self-attention and its O(n^2) cost, multi-head attention, MHA vs GQA vs MQA and KV-cache size, FFN, LM head); Llama 3 70B sized end to end (parameters, 141 GB of weights, FLOPs per token, 320 KB of KV cache per token); mixture of experts (memory scales with total parameters, compute with active ones); advanced, to skim: sliding-window attention, MLA, linear attention, state-space models, sparse attention, looped transformers, multi-token prediction, diffusion LMs	Transformer [arXiv:1706.03762] ; Llama 3 [arXiv:2407.21783] ; Mixtral [arXiv:2401.04088] ; BPE [arXiv:1508.07909] ; Transformer Inference Arithmetic [kipp.ly] ; DeepSeek-V3 [arXiv:2412.19437]	Slides [PDF] ; readings (15 entries) [cs2680.seas.harvard.edu]	FND DEPLOY
13	**Efficient LLM serving basics II: intro to GPUs**	The GPU as a throughput machine (warps, SIMT, branch divergence, latency hidden by switching warps); the three limits (peak compute, HBM bandwidth, memory capacity); the roofline: batch-1 decode is memory-bound (about 0.5% of H200 peak), prefill compute-bound, batching helps weight reads but not KV-cache reads; inside an H200 (SMs, tensor cores, memory hierarchy, bytes in flight, TMA, Blackwell tensor memory); scale-up and scale-out (NVLink, NVL72); other accelerators (AMD MI355X, TPU, Trainium, SRAM-based chips). Reading page groups: GPU programming model; performance model; FlashAttention; attention kernels inside a serving engine; kernel fusion; compilers and mega-kernels; the runtime between scheduler and kernel; measurements	CUDA programming guide [docs.nvidia.com] ; Roofline [doi.org] ; Making Deep Learning Go Brrrr [horace.io] ; FlashAttention [arXiv:2205.14135] ; FlashInfer [arXiv:2501.01005] ; A Systematic Characterization of LLM Inference on GPUs [arXiv:2512.01644]	Slides [PDF] ; readings (17 entries) [cs2680.seas.harvard.edu]	DEPLOY FND

#	Title	Key topics	Readings	Materials	Tags
16	**Assignments feedback and agent pitfalls, GPU programming (cont.)**	Added by 2026-11-11 (speculative decoding, planned for this date, moved to Nov 11). Feedback: what students learned from and wanted of Claude Code in Assignment 1 (asynchronous event streams, checkable requirements over broad prompts, poorly split delegation); why the agent used Bash over its native tools (Claude Code's system prompt); repeated gradings of 18 leaderboard submissions, whose run-to-run variance depends on the harness; how Claude Code use grew across the three assignments (prompts, hours, cost and tool calls up 4.5–7x); ten pitfalls of working with an agent (trusting its claims unchecked, losing track of what it built, long sessions losing context to compaction, stacking changes without controlled experiments, parallel sessions colliding, destructive commands, leaked secrets, reading one noisy run as an effect, venting instead of steering, platform issues) and a good iteration loop. GPU programming, the second part of the Oct 21 deck: asynchronous host-device execution, SIMT kernels, blocks and warps on SMs; graph compilers (torch.compile) and tile languages (Triton, TileLang, CuTe/CUTLASS, ThunderKittens; to skim); MFU and MBU, why MFU is low, profiling with Nsight Systems, reading GPU counters beyond nvidia-smi; kernel advances (fusion, FlashAttention 1–4, persistent kernels, split-KV, CUDA Graphs, programmatic dependent launch, megakernels, fused collectives, MoE all-to-all overlap, agents writing kernels); hardware challenges	–	Slides (feedback) [PDF] ; slides (GPU, part 2) [PDF] ; Claude Code usage-feedback site (enrolled students; live Oct 28) [how-i-used-claude.cs2680.com]	CODE DEPLOY

#	Title	Key topics	Readings	Materials	Tags
17	**Efficient LLM serving basics III: Batching, paging, and parallelism**	Moved from Oct 28 by 2026-11-11. Why serving differs (prefill vs decode bottlenecks, unknown output length); metrics and evaluation (TTFT and TBT percentiles, goodput, GPU vs user throughput; four benchmarking pitfalls: short transient runs, misleading metrics, unrealistic workloads, unfair comparison); continuous and selective batching; request scheduling (FCFS, prefill-first vs decode-first, chunked prefill, prefill-decode disaggregation and KV-cache transfer, SJF); PagedAttention; prefix caching (block hashing, SGLang RadixAttention); parallelism (collectives; data, tensor (column- and row-parallel, the Megatron block), pipeline, expert and context parallelism; TP and EP need NVLink). Reading page groups: measuring serving performance; iteration-level batching; separating prefill from decode; choosing which request runs next; paging and memory management; prefix caching; collective communication; splitting the model across devices	Orca [usenix.org] ; vLLM / PagedAttention [arXiv:2309.06180] ; On Evaluating Performance of LLM Inference Serving Systems [arXiv:2507.09019] ; Sarathi-Serve [arXiv:2403.02310] ; DistServe [arXiv:2401.09670] ; SGLang / RadixAttention [arXiv:2312.07104] ; Megatron-LM [arXiv:1909.08053]	Slides [PDF] ; readings (17 entries) [cs2680.seas.harvard.edu]	DEPLOY
18	**Guest lecture: Why Infrastructure sets the pace of AI (Xianzheng Dou and Haythem Memmi, Meta)**	Guest lecture; no materials posted	–	–	DEPLOY
19	**Efficient LLM serving: speculative decoding**	Scheduled; no slides yet (moved from Nov 2 by 2026-11-11). Reading page groups: guess ahead, verify in parallel; where the guesses come from; drafting a whole block in one pass; drafting from text already seen; verification trees and acceptance rules; inside a serving system; training drafters	–	readings, not linked from the home page (45 entries) [cs2680.seas.harvard.edu]	DEPLOY
20	**Efficient LLM serving: KV-cache optimization and model architectures**	Student-led paper discussion (scheduled). Reading page groups: storing fewer keys and values (MQA, GQA, MLA); fixed sparsity; removing the sequence-growing state; sparsity built into the model	–	Discussion guide [cs2680.seas.harvard.edu] ; readings, not linked from the home page (13 entries) [cs2680.seas.harvard.edu]	DEPLOY
22	**Efficient LLM serving: pruning and quantization**	Scheduled; no slides yet (moved from Nov 11 by 2026-11-11). Reading page groups: number formats; outliers and INT8; post-training weight quantization; KV-cache quantization; serving a quantized model at speed; quantized fine-tuning; 4-bit, 2-bit and ternary; what a bit costs beyond perplexity; rotations; pruning, sparsity and cheaper architectures	–	readings, not linked from the home page (36 entries) [cs2680.seas.harvard.edu]	DEPLOY

#	Title	Key topics	Readings	Materials	Tags
23	**Efficient agentic systems: scheduling and prefix caching**	Scheduled; no slides yet (moved from Nov 4 by 2026-11-11; the site's "earlier because of Assignment 4" note is gone). Reading page groups: workloads, traces and evaluation; exposing workflows to the serving layer; scheduling programs rather than requests; optimizing across the workflow; inference scaling and agentic RL; indexing shared prefixes; prefix cache for hybrid attention; memory pool; approximate reuse of non-prefix spans; eviction policies; cache engines and simulators	–	readings, not linked from the home page (49 entries) [cs2680.seas.harvard.edu]	DEPLOY
24	**Efficient agentic systems: routing and load balancing**	Scheduled; no slides yet (moved from Nov 23 by 2026-11-11). Reading page groups: load balancing; model routing	–	readings, not linked from the home page (9 entries) [cs2680.seas.harvard.edu]	DEPLOY

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
A1 Mads-Lens: Watch the agent work	A web front end that drives Claude Code in headless mode (its JSON event stream): submit a prompt, show the trajectory live with each tool call and its result, show status per call and per run, resume a session, show cost, duration and turns, and nest subagent events as a collapsible tree with an outline	Driving an agent programmatically; reading an event stream; trajectory visualization	None. Spec [cs2680.seas.harvard.edu] ; six student implementations with recorded demo runs [GitHub]	CODE ARCH
A2 Mads-Loop: Build the loop	`madsLoop`, a minimal coding-agent harness around an OpenAI-compatible model API (the handout pins `qwen3.6-35b-a3b`): tool calling with schemas, context management, retries, termination, a required log schema. It is scored on 10 supplied SWE-bench Pro tasks plus hidden ones with the official evaluator; the handout warns that some supplied tasks may be impossible and asks which and why. You also build four tasks of your own (reference fix, Dockerfile, fail-to-pass and pass-to-pass tests): two the agent solves only after a harness change, two it still cannot solve	Agent loop, tool design, context management, failure handling, building evaluation tasks	[cs2680.seas.harvard.edu] ; spec [cs2680.seas.harvard.edu] (model calls need a course key; see Notes)	CODE ARCH EVAL
A3 Mads-Opt: Make every token count	A harness competition. Your agent runs in a container, opens up to 5 task containers at a time through a dispatcher, and decides scheduling, retries after grading feedback and when to give up, using three priced model tiers. A leaderboard of 26 tasks scores tasks solved, cost per solved task and time per solved task, ranked in skyline layers among submissions that solve at least 7; run limit 4 hours and 10 dollars per submission	Cost and latency optimization, model routing, task scheduling, parallel tool execution, context compression	[cs2680.seas.harvard.edu] ; spec [cs2680.seas.harvard.edu] (official scoring needs course access; see Notes)	DEPLOY CODE EVAL

Name	What you build	Skills	Starter code / resources	Tags
A4 Mads-Serve: Own the serving engine	Serve the course model (Qwen3.6-35B-A3B NVFP4, a mixture-of-experts model with linear-attention layers) yourself with SGLang v0.5.20 on one 32 GB RTX PRO 4500, point your Assignment 3 agent at it, and run 25 SWE-bench Pro tasks with all of them open at once. Measure the starting run (tasks solved, end-to-end time), poll the server's /metrics, explain and plot where the time goes, and compare SGLang's serving benchmark with the agent's real workload. Then find the cause, list fixes with predictions, and apply them one at a time to the engine (flags or source of SGLang or vLLM 0.31.0, or your own engine; KV-cache dtype and speculative decoding with the model's own multi-token-prediction head are allowed) or to the agent, with the model fixed (no re-quantization, no separate draft model). Each change is recorded as change, observation, reasoning and result. A leaderboard on 26 different tasks ranks tasks solved and end-to-end time	LLM serving engine configuration, profiling a live serving workload, benchmark vs real workload, hypothesis-driven engine and agent co-optimization	[cs2680.seas.harvard.edu] ; spec [cs2680.seas.harvard.edu] ; slides [PDF] ; engine conventions [leaderboard.cs2680.com] (needs the course GPU cluster; see Notes)	DEPLOY EVAL CODE
A5 Mads-Stack: Optimize the full stack	Planned; brief only. Profile the whole path (agent loop, model, serving system), write a performance hypothesis before changing anything, optimize across layers and re-run the same workload to see which changes mattered. It is submitted as a blog post, which (per the blog's section text, 2026-11-10) covers what the author learned across Assignments 2–5, one section per lesson, with the author's own measurements and mistakes as well as what worked	Profiling, hypothesis-driven optimization across layers	None. Brief [cs2680.seas.harvard.edu] (full spec unreleased on 2026-11-11: the spec link still goes to a "to_release" page, and assignment5.tar.gz and slides/assignment5.pdf return 404)	DEPLOY EVAL
Final project	An individual project on one of two tracks. Product: a tool that beats the strongest available alternative on a real problem. Research: an original, falsifiable claim about agents or the systems that run them (measurement, mechanism, reproduction-and-boundary, or instrument). The proposal has nine sections, including the gap shown by running the closest existing systems, a claim, two named baselines, an ablation, and 75/100/125% goals. The repository must reproduce the main result	Problem framing, baselines and ablations, reproducible measurement, technical writing	None. Spec [cs2680.seas.harvard.edu]	EVAL ARCH DEPLOY

Buffalo: CSE 410/510 Introduction to Agentic Engineering

Institution	University at Buffalo (Joaquin Carbonara, David Doermann)
Term	Fall 2026
Website	https://cse.buffalo.edu/courses/cse410/2026_fall/cse410ae/
Level	Upper-undergrad / MS
Archetype	Engineering discipline (framework-agnostic)
Public materials	Slides: none; video: none; code: none (lab specs only)
Best for	Five lab specs and a capstone spec that treat an agent as an engineered, evaluated, bounded system.

Treats an agent as an engineered system "not a magic chatbot": a bounded environment, designed tools, explicit evaluation and safety constraints. No slides or code are public. What makes it useful is the lab sequence: a minimal tool-using agent, retrieval and memory, workflow graphs, an evaluation harness, and human-in-the-loop safety boundaries. The capstone spec, a useful bounded agent with an architecture document, an eval suite and a risk assessment, is a good template for your own portfolio project.

Lectures

#	Title	Key topics	Readings	Materials	Tags
1	**What is agentic engineering?**	Agents, environments, tools, goals, constraints	Reading Note 1 (specific reading not listed)	–	ORG ARCH
2	**Foundation models as components**	Prompting, context, structured outputs	Reading Note 2	–	FND
3	**Tool/function calling and ReAct**	Tool calling, ReAct; studio on linear algebra and cosine similarity	Reading Note 3	–	TOOL RSN
4	**Retrieval-augmented generation and context construction**	RAG, context building	Reading Note 4	–	MEM
5	**Embeddings, vector search, reranking**	Embeddings, vector DBs, reranking	–	–	MEM
6	**Agent workflows**	Graphs, state machines, LangGraph or custom orchestration	–	–	ARCH
7	**Planning and decomposition**	When should an agent plan?	–	–	RSN
8	**Human-agent interaction and memory**	Human-in-the-loop, memory architectures	–	–	MEM SAFE
9	**Evaluation of agents**	From single-prompt to task-level evals	–	–	EVAL
10	**Security and privacy**	Prompt injection, tool misuse, sandboxing, privacy	–	–	SAFE
11	**Multi-agent systems and coordination**	MAS coordination	–	–	MULTI
12	**Coding agents, testing, CI, observability**	Coding agents, testing, CI, observability	–	–	CODE DEPLOY
13	**Applied agents and responsible deployment**	Data science, discovery and education agents; responsible deployment	–	–	APP DEPLOY

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
Lab 1: A Minimal Tool-Using Agent	Python agent choosing among a calculator, a file reader and a search/lookup tool. Submit a design note and successful/failed runs	Tool calling, agent loop, failure analysis	–	TOOL ARCH

Name	What you build	Skills	Starter code / resources	Tags
Lab 2: Retrieval and Memory	Small RAG pipeline over a course-selected corpus. Compare keyword/dense/hybrid retrieval, compute cosine similarity, do error analysis	RAG, embeddings, retrieval eval	–	MEM EVAL
Lab 3: Workflow Graphs and State Machines	Agentic workflow as a graph/state machine with explicit states, transitions, failure handling and logging	Orchestration, state design, logging	–	ARCH
Lab 4: Evaluation Harness	Eval suite for a bounded task: test cases, success criteria, metrics, failure taxonomy, reproducible script	Agent evaluation	–	EVAL
Lab 5: Human-in-the-Loop and Safety Boundaries	Add human approval, structured outputs, guardrails or permission boundaries. Analyze where autonomy should be restricted	HITL, guardrails, permissions	–	SAFE
Math mini-assignment 1	Linear algebra for ML: vectors, cosine similarity	Math for retrieval	–	FND MEM
Final Team Project: Useful Bounded Agent	Working agent for a bounded task. Deliverables: problem statement, environment/tool inventory, architecture diagram, repo, eval dataset/script, results and error analysis, security/privacy/ethics risk assessment, demo, final report	End-to-end agent engineering	–	ARCH EVAL SAFE

UVA: CS 6501 Workshop on Building AI Agents

Institution	University of Virginia (Henry Kautz)
Term	Spring 2026
Website	https://www.cs.virginia.edu/~rmw7my/Courses/AgenticAISpring2026/agenticAI.html
Level	Graduate workshop; Python
Archetype	Hands-on lab workshop (small/open models)
Public materials	Slides: PPTX per topic; video: 1 + podcasts; code: Starter scripts per topic
Best for	The richest set of runnable lab exercises: local LLMs, LangGraph, tools, RAG, VLMs, MCP/A2A, Tinker fine-tuning.

A workshop where most of the programming happens in class. Each topic page has learning goals, an exercise sheet of measurable experiments, PPTX slides and starter scripts. The first half is labs: running an LLM, LangGraph control flow, tool calling, the tool ecosystem, RAG, vision-language agents, MCP and A2A, and QLoRA-style fine-tuning with Tinker. The second half is state-of-the-art lectures: context management, chain-of-thought internals, coding agents, personal agents and distillation, and agents for science, mathematics and visual reasoning. Row numbers in the catalogue are calendar slots. The topic number appears in each title.

Lectures

#	Title	Key topics	Readings	Materials	Tags
1	**Topic 1: Running an LLM**	LLM overview, tokenizers, open models, HF benchmark datasets; run Llama 3.2-1B MMLU eval with/without quantization on CPU/GPU/Colab; build a simple chat agent with context management	Belcak et al., Small Language Models are the Future of Agentic AI ([arXiv:2506.02153]); Illustrated GPT-2 ([jalammar.github.io])	Page: [cs.virginia.edu] ; slides `Topic1Running/01 Running an LLM.pptx`; code `llama_mmlu_eval.py`, `simple_chat_agent.py`; NoteGPT podcast MP3	FND EVAL DEPLOY
3	**Topic 2: Agent Frameworks**	LangGraph nodes/edges, conditional routing, parallel models (Llama + Qwen), chat history, multi-agent chat, checkpointing/crash recovery; optional web-search summarizer	LangGraph Graph API overview ([docs.langchain.com])	Page: [cs.virginia.edu] ; slides `02 Agent Frameworks.pptx`; code `langgraph_simple_llama_agent.py`	ARCH MULTI
5	**Topic 3: Agent Tool Use (online)**	Ollama server, parallel calls, OpenAI API/GPT-4o mini, OpenAI tool-call message format, manual tool handling (calculator), LangGraph multi-tool agent, tool forcing, persistent conversation graph	Schick et al., Toolformer ([arXiv:2302.04761])	Page: [cs.virginia.edu] ; slides `03 Tools.pptx`; code `manual-tool-handling.py`, `langgraph-tool-handling.py`, `gpt4omini_test.py`; guides `ollama_guide.html`, `tool-forcing-strategies.html`; NotebookLM podcast `toolformer.m4a`	TOOL ARCH
6	**Topic 3 lab continued (online)**	Breakout rooms continuing Topic 3	–	–	TOOL
7	**Topic 4: Exploring Tools**	ToolNode vs create_react_ agent, parallel tool dispatch, tools on remote servers, LangGraph built-in and third-party tool library; "2-Hour Agent Project" with a partner	–	Page: [cs.virginia.edu] ; slides `exploring.pptx`; code `toolnode_example.py`, `react_agent_example.py`; `agent_tools_reference.html`; MoltReader [GitHub]	TOOL RSN ARCH
9	**Topic 5: Retrieval Augmented Generation**	Vector DBs, RAG pipeline, chunk size/overlap, top-K, query phrasing, score thresholds, prompt templates, unanswerable questions, cross-document synthesis (Qwen 2.5 1.5B vs GPT-4o mini)	–	Page: [cs.virginia.edu] ; slides `rag.pptx`; notebook `manual_rag_pipeline_universal.ipynb`; `Corpora.zip`; pdf_ocr [GitHub]	MEM EVAL
10	**Topic 6: Vision-Language Models**	CLIP, contrastive alignment, projection layers, image embeddings, image generation; VLM chat agent in LangGraph; video-surveillance agent (OpenCV keyframes + LLaVA)	Illustrated Stable Diffusion ([jalammar.github.io])	Page: [cs.virginia.edu] ; slides `vlm_part1.pptx`, `vlm_part2.pdf`, `vlm_part3.pdf`; `vlm_guide.html`, `image_generation_guide.html`; clip_test [GitHub]	WEB APP FND

#	Title	Key topics	Readings	Materials	Tags
12	**Topic 7: MCP and A2A**	Model Context Protocol, tool discovery, remote Asta tools (AI2), Agent-to-Agent protocol, agent cards, registry, ngrok tunneling; quiz-tournament exercise	–	Page: [cs.virginia.edu] ; slides `mcp.pptx`; `mcp_asta_lesson_plan.html`, `a2a_lesson_plan.html`; code `a2a_agent_template.py`, `a2a_test.py`, `a2a_trivia.py`, `A2a_registry.py`, `semantic_similarity.py`, `test_ngrok.sh`	TOOL MULTI
13	**Topic 7 cont. (A2A) + Topic 8: LLM Fine-Tuning**	Kinds of fine-tuning, fine-tuning vs RAG, QLoRA, Tinker pipeline; text-to-SQL fine-tuning of Llama	QLoRA ([arXiv:2305.14314]), listed on the home page	Page: [cs.virginia.edu] ; slides `finetuning.pptx`; `finetuning_intro_lesson_plan.html`, `sql_finetuning_lesson_plan.html`; data `sql_create_context_v4.json`; `sql_matches.py`	TRAIN MULTI
15	**Topic 9: Context Management**	Compression, selection, persistence; how frontier models manage long contexts	AgentFold ([arXiv:2510.24699]) and Anthropic "Effective context engineering" are listed on the home page; the topic page itself lists none	Page: [cs.virginia.edu] ; slides `context.pptx`; `context_management_lesson_plan.html`	MEM
16	**Topic 10: Understanding Chain of Thought**	CoT prompt varieties, why CoT works, tokenized vs hidden-layer (latent) CoT, hidden thought tokens	CoT ([arXiv:2201.11903]) on the home page; topic page lists none	Page: [cs.virginia.edu] ; slides `chain_of_thought.pptx`; `chain_of_thought_lesson_plan.html`	RSN
17	**Topic 11: Coding Agents**	History from statistical LMs, frontier coding agent architecture (Claude Code), skills files, evolutionary programming	AlphaEvolve ([doi.org]); AVO ([doi.org]); EvoSkill ([doi.org])	Page: [cs.virginia.edu] ; slides `coding_1_history.pptx`, `coding_2_claude.pptx`, `coding_3_evolution.pptx`	CODE ARCH
18	**Topic 12: Personal Agents and Distilling**	Clawdbot to OpenClaw/NullClaw/CoPaw/NemoClaw, Claude Cowork; risks of agents controlling accounts; OpenClaw-RL; knowledge distillation	n/a (resource links only)	Page: [cs.virginia.edu] ; slides `personal.pptx`, `Knowledge Distillation Slides.pptx`; `personal_lesson_plan.html`, `distill.html`	WEB SAFE TRAIN
19	**Topic 13: Agentic AI for Science**	Research as an agent loop, scientific agent stack, AI2 tools (Semantic Scholar, OpenScholar, ScholarQA, Asta), JFC HEP case study, Orchestral AI, HEPTAPOD	AutoDiscovery ([arXiv:2507.00310]); AI agents perform experimental HEP ([inspirehep.net]); HEPTAPOD ([arXiv:2512.15867])	Page: [cs.virginia.edu] ; slides `science.pptx`; HEPTAPOD code [GitHub]	APP MULTI
20	**Topic 14: Agentic AI for Mathematics and Verification (guest: Prof. Kevin Sullivan)**	Lean as lingua franca; AlphaProof, Lean Copilot, APOLLO, CLEVER; formal methods	Lean4CS1 notes ([kevinsullivan.github.io])	Page: [cs.virginia.edu] ; `Kevin Sullivan Agents-Summary.html`, `Kevin Sullivan Agents-Class Note.html`	RSN APP
21	**Topic 15: Visual Reasoning**	Video as reasoning substrate, diffusion transformers, Chain-of-Frames vs Chain-of-Steps, emergent working memory and self-correction, MIRAGE critique of VLM evaluation	Demystifying Video Reasoning ([arXiv:2603.16870]); Chain-of-Frames ([arXiv:2506.00318]); MIRAGE ([arXiv:2603.21687])	Page: [cs.virginia.edu] ; slides `visual.pptx`, `mirage.pptx`; video [video]	RSN EVAL

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
Topic 1 portfolio: Running an LLM	MMLU timing/accuracy study across models, quantization and hardware; chat agent with bounded context and restartability; optional MT-Bench	HF transformers, quantization, benchmarking, context management	`llama_mmlu_eval.py`, `simple_chat_agent.py` via [cs.virginia.edu]	FND EVAL MEM
Topic 2 portfolio: LangGraph agents	Routed, parallel, multi-model LangGraph chat with history and crash recovery; optional web summarizer	LangGraph graphs, routing, checkpointing	`langgraph_simple_llama_agent.py` via [cs.virginia.edu]	ARCH MULTI
Topic 3 portfolio: Tool use	Ollama parallel timing; manual tool calling with a calculator; LangGraph multi-tool agent with tool_map; persistent graph with Mermaid diagram	OpenAI tool format, tool forcing, checkpointing	`manual-tool-handling.py`, `langgraph-tool-handling.py` via [cs.virginia.edu]	TOOL ARCH
Topic 4: 2-Hour Agent Project	Partner-built mini agent using LangGraph/third-party tools	ToolNode, ReAct agents, tool ecosystem	`toolnode_example.py`, `react_agent_example.py` via [cs.virginia.edu]	TOOL ARCH

Name	What you build	Skills	Starter code / resources	Tags
Topic 5: RAG experiments	11 controlled RAG experiments (RAG vs no-RAG, top-K, chunking, prompts, unanswerable questions)	Vector retrieval, RAG evaluation	`manual_rag_pipeline_universal.ipynb`, `Corpora.zip` via [cs.virginia.edu]	MEM EVAL
Topic 6: VLM agents	Image-chat LangGraph agent; video-surveillance agent with OpenCV + LLaVA (optional TTS intruder alert)	VLMs, Ollama llava, Gradio	`gradio_quickstart.py` via [cs.virginia.edu]	WEB APP
Topic 7: MCP + A2A	MCP client for AI2 Asta tools; A2A agents, registry and trivia tournament over ngrok	MCP, A2A, networking	`a2a_agent_template.py` etc. via [cs.virginia.edu]	TOOL MULTI
Topic 8: Text-to-SQL fine-tuning	Fine-tune Llama on text-to-SQL with Tinker and score semantic SQL matches	LoRA/QLoRA, Tinker, eval	`sql_create_context_v4.json`, `sql_matches.py` via [cs.virginia.edu]	TRAIN EVAL
Final project	Team project from a list of 20+ ideas (e.g., distillation, mini coding agent with sub-agents, MemGPT-style memory, debate agents, NL-to-PDDL planner, Lean/SAT assistants, misalignment replication)	End-to-end agent design and evaluation	–	ARCH EVAL APP

UW-Madison: CS 839 AI Agents

Institution	University of Wisconsin–Madison (Aws Albarghouthi)
Term	Spring 2026
Website	https://github.com/barghouthi/ai-agents
Level	Graduate research seminar
Archetype	Research seminar (paper reading)
Public materials	Slides: none; video: none; code: Demo notebook; MIT license
Best for	A curated, critical reading list from ReAct to self-improving and evolutionary agents, plus a no-frameworks coding-agent assignment.

A paper seminar with two main papers and several supplementary papers per session. It runs from CoT, ReAct, Toolformer and ToT to coding agents, deep research, agents for science, prompt-injection security, agent optimization (DSPy, TextGrad, ADAS, GEPA), self-improving and evolutionary agents, search, multi-agent systems and memory. The buy-in assignment, a CLI coding agent built on raw API calls with no agent frameworks, is one of the best single exercises in the collection. The repository is MIT-licensed.

Lectures

#	Title	Key topics	Readings	Materials	Tags
1	**Class overview**	Overview of the agents space	–	–	ORG
2	**Building agents**	Instructor intro to building LLM agents	Building Effective Agents (Anthropic) [anthropic.com] (listed as "see also" in week 1)	Demo notebook [GitHub] (session association assumed)	ARCH TOOL
3	**Reasoning and acting I**	Chain-of-thought, ReAct	CoT [arXiv:2201.11903] ; ReAct [arXiv:2210.03629] ; supp.: Self-Consistency [arXiv:2203.11171] , Kimi K2 [arXiv:2507.20534] , Llama 3 Herd [arXiv:2407.21783] , Constitutional AI [arXiv:2212.08073]	–	RSN FND
4	**Reasoning and acting II**	Tool learning, Tree/Graph of Thoughts	Toolformer [arXiv:2302.04761] ; Tree of Thoughts [arXiv:2305.10601] ; supp.: LLMs as Tool Makers [arXiv:2305.17126] , ART [arXiv:2303.09014] , Graph of Thoughts [ojs.aaai.org]	–	TOOL RSN
5	**Coding agents I**	SWE benchmarks, SWE-agent ACI	SWE-bench [arXiv:2310.06770] ; SWE-agent [arXiv:2405.15793] ; supp.: Terminal-Bench [openreview.net] , Agentless [arXiv:2407.01489]	–	CODE EVAL
6	**Coding agents II**	Open agent platforms, secure code generation benchmarks	OpenHands [arXiv:2407.16741] ; BaxBench [arXiv:2502.11844] ; supp.: RepairAgent [arXiv:2403.17134] , RedCode [arXiv:2411.07781] , WebArena [arXiv:2307.13854]	–	CODE SAFE EVAL
7	**Deep research agents I**	Article writing, autonomous research labs	STORM [arXiv:2402.14207] ; Agent Laboratory [arXiv:2501.04227] ; supp.: ResearchAgent [arXiv:2404.07738] , Deep research agents roadmap [arXiv:2506.18096] , Anthropic multi-agent research system [anthropic.com]	–	APP MULTI
8	**Deep research agents II**	Literature QA agents, research benchmarks	PaperQA [arXiv:2312.07559] ; LiveResearchBench [arXiv:2510.14240] ; supp.: Can LLMs Generate Novel Research Ideas? [arXiv:2409.04109] , DR Tulu [arXiv:2511.19399]	–	APP MEM EVAL

#	Title	Key topics	Readings	Materials	Tags
9	**Agents for science I**	AI Scientist, autonomous chemistry	The AI Scientist [arXiv:2408.06292] ; Autonomous chemical research with LLMs [nature.com] ; supp.: ChemCrow [arXiv:2304.05376]	–	APP
10	**Agents for science II**	AI co-scientist, hypothesis falsification	Towards an AI Co-Scientist [arXiv:2502.18864] ; Agentic Sequential Falsifications [arXiv:2502.09858] ; supp.: SciAgents [arXiv:2409.05556]	–	APP MULTI
11	**Security & prompt injection I**	Indirect prompt injection, AgentDojo benchmark	Indirect Prompt Injection [arXiv:2302.12173] ; AgentDojo [arXiv:2406.13352] ; supp.: Jailbroken [arXiv:2307.02483]	–	SAFE EVAL
12	**Security & prompt injection II**	Capability-based defenses, browser agent security	CaMeL [arXiv:2503.18813] ; ceLLMate [arXiv:2512.12594] ; supp.: Lethal trifecta [simonwillison.net] , Meta practical agent security [ai.meta.com]	–	SAFE
13	**Optimizing agents I**	Declarative LM programs, textual gradients	DSPy [arXiv:2310.03714] ; TextGrad [arXiv:2406.07496] ; supp.: LLMs as Optimizers [arXiv:2309.03409]	–	ARCH TRAIN
14	**Optimizing agents II**	Automated agent design, reflective prompt evolution	ADAS [arXiv:2408.08435] ; GEPA [arXiv:2507.19457] ; supp.: EvoPrompt [arXiv:2309.08532] , Trace [arXiv:2406.16218]	–	ARCH TRAIN
15	**Self-improving agents I**	Lifelong skill libraries, self-referential agents	Voyager [arXiv:2305.16291] ; Gödel Agent [arXiv:2410.04444]	–	ARCH WEB
16	**Self-improving agents II**	Evolutionary coding agents, live self-evolving SWE agent	AlphaEvolve [deepmind.google] ; Live-SWE-agent [arXiv:2511.13646] ; supp.: LATS [arXiv:2310.04406]	–	CODE RSN
17	**Searching the solution space I**	Large-scale sampling and search for code and math	AlphaCode [arXiv:2203.07814] ; AlphaGeometry [nature.com] ; supp.: AlphaProof [nature.com] , AlphaZero [arXiv:1712.01815]	–	RSN CODE
18	**Searching the solution space II**	LLM-driven evolutionary search	ShinkaEvolve [arXiv:2509.19349] ; Evolution through Large Models [arXiv:2206.08896] ; supp.: Digital Red Queen [arXiv:2601.03335] , Promptbreeder [arXiv:2309.16797]	–	RSN
19	**Multi-agent systems I**	Generative agents, role-playing agents	Generative Agents [arXiv:2304.03442] ; CAMEL [arXiv:2303.17760] ; supp.: Multiagent Debate [arXiv:2305.14325]	–	MULTI
20	**Multi-agent systems II**	Multi-agent frameworks, scaling laws for agent systems, single vs multi-agent	AutoGen [arXiv:2308.08155] ; Science of Scaling Agent Systems [arXiv:2512.08296] ; supp.: Single agent systems [openhands.dev] , Prompt Infection [arXiv:2410.07283] , MetaGPT [arXiv:2308.00352] , AgentVerse [arXiv:2308.10848]	–	MULTI ARCH
21	**Memory I**	OS-style memory hierarchy, cognitive architectures	MemGPT [arXiv:2310.08560] ; CoALA [arXiv:2309.02427] ; supp.: Think-in-Memory [arXiv:2311.08719] , A-Mem [arXiv:2502.12110] , Recursive language models [arXiv:2512.24601]	–	MEM ARCH

#	Title	Key topics	Readings	Materials	Tags
22	**Memory & other topics II**	Mixture-of-agents, context folding, long-horizon task measurement	Mixture-of-Agents [arXiv:2406.04692] ; Context-Folding [arXiv:2510.11967] ; supp.: More Agents Is All You Need [arXiv:2402.05120] , Measuring AI Ability to Complete Long Tasks [arXiv:2503.14499]	–	MEM MULTI EVAL

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
Build a Coding Agent	A CLI coding agent like Claude Code / Cursor CLI / Gemini CLI, using raw LLM API calls with no agent frameworks (LangChain, Strands, CrewAI banned; thin wrappers like litellm allowed). It must parse and execute tool calls, have a polished UI, a README, and one non-trivial program the agent built	Agent loop, function calling, tool execution, CLI UX	None. Spec: [GitHub]	CODE TOOL ARCH

JHU: EN.601.727 Machine Programming

Institution	Johns Hopkins University (Ziyang Li)
Term	Fall 2026 (in progress)
Website	https://machine-programming.github.io/
Level	Graduate (PL + ML)
Archetype	Program synthesis to coding agents
Public materials	Slides: L1–19, two supplements; video: none; code: A1–A3 public
Best for	Ties classic program synthesis to LLM coding agents, with two MCP lectures and a strong web-app agent assignment.

Treats code generation from both the symbolic side (enumerative search, types, specifications) and the LLM side (decoding, prompting, iterative refinement, agentic frameworks, MCP, pre- and post-training of code models). The assignments are the main attraction: an enumerative synthesizer with an LLM step (A1), a rigorous pass@k evaluation of coding LLMs across three languages and four prompting strategies (A2), and a tool-calling web-app agent with secure file tools that builds a chat room, a Tetris variant and a creative app (A3). The second half covers formal verification, security, HPC, neurosymbolic programming and six student-led paper sessions in which two students defend and attack each paper. The course's readings page is organised by topic, not by lecture, so the readings in its lecture table are topical matches rather than assigned readings.

Lectures

#	Title	Key topics	Readings	Materials	Tags
1	**Overview & Introduction to Machine Programming**	Course intro; what machine programming is	–	[PDF]	ORG
2	**Syntax, Semantics, and Bottom-up Inductive Synthesis**	DSL grammars/semantics; bottom-up enumeration from examples	–	[PDF]	RSN
3	**Type Systems and Top-down Enumerative Synthesis**	Types for pruning; top-down search	–	[PDF]	RSN
4	**Functional Specifications and Synthesis**	Synthesis from logical/functional specs	–	[PDF]	RSN
5	**Language Modeling for Synthesis**	LMs as program generators	Code LLM readings (readings page)	[PDF]	FND CODE
6	**Decoding and Prompting for Synthesis**	Decoding strategies; prompting for code	–	[PDF]	FND CODE
7	**Iterative Refinement and Agentic Framework**	Feedback/refinement loops; agent framework	–	[PDF]	ARCH CODE
8	**Agentic Frameworks: Model Context Protocol**	MCP	–	[PDF]	TOOL ARCH
9	**Agentic Frameworks: Model Context Protocol (2)**	MCP, continued	–	[PDF]	TOOL ARCH
10	**Agentic Frameworks: Selected Topics**	Agent framework topics (unspecified)	Agents readings: MASAI, AutoCodeRover	[PDF]	ARCH CODE
11	**Pre-training of Coding Language Models (1)**	Code-LLM pre-training (data, tokenization, objectives)	e.g., StarCoder, Code Llama, DeepSeek Coder, The Stack, FIM (readings page)	[PDF]	TRAIN CODE
12	**Pre-training of Coding Language Models (2)**	Pre-training, continued	as above	[PDF]	TRAIN CODE
13	**Post-training of Coding Language Models: SFT & RL**	Post-training of code LLMs: supervised fine-tuning and reinforcement learning	e.g., CodeDPO, LintSeq (readings page)	[PDF]	TRAIN CODE

#	Title	Key topics	Readings	Materials	Tags
14	**Special Topics on Coding Language Models**	Coding-LLM special topics (unspecified; slides not readable)	–	[PDF]	TRAIN CODE
15	**PL Kaleidoscope I: Formal Verification for Mathematics**	Formal statements checked by a prover kernel; automated vs interactive provers (Lean/Rocq/Isabelle/Z3); autoformalization (NL to formal) and proof search as synthesis tasks; live Lean demo; autoformalization benchmarks; faithfulness of AI formalizations (OpenAI Navier-Stokes Lean case)	arXiv 2109.00110, 2407.11214, 2610.08144, 2606.26525, 2608.15432, 2610.18498 (cited in slides)	[PDF]	APP CODE
16	**PL Kaleidoscope II: Formal Program Verification**	Program verification framed as synthesis tasks: specification synthesis (what the program should do), specification inference (invariants) and verification (is it correct), from source code and docs to a formal implementation, spec and proof; demo: a tiny three-address virtual machine (assembly interpreter) deeply embedded in Lean, with abs/max/factorial programs and a formal termination spec (lecture 15's abs was shallowly embedded)	–	[PDF] ; demo [machine-programming.github.io]	APP CODE
17	**PL Kaleidoscope III: Software Security and Static Analysis**	Software analysis as checking a program against a spec (types, tests, verification, static and dynamic analysis); the soundness/completeness/termination "impossible triangle", precision and recall; vulnerability life-cycle; path traversal (CWE-22) and two failed fixes; taint-based detection with CodeQL (TaintedPathQuery) and Semgrep; a first pass over dynamic testing, fuzzing, proof-of-concept exploits and LLMs in offensive security. The deck's own title is "Software Security and Testing"	Alshahwan et al., "Automated Unit Test Improvement using Large Language Models at Meta", FSE 2024 (cited in slides)	[PDF]	SAFE CODE APP
18	**PL Kaleidoscope IV: Software Security and Dynamic Analysis**	Dynamic analysis: unit testing and line coverage, LLM test improvement at Meta; the fuzzing loop (instrumentation, harness, seed inputs, mutation engine), with the libpng OSS-Fuzz harness, AFL, OSS-Fuzz and Fuzz Introspector; where LLMs help (instrumentation for more vulnerability patterns, harness generation, seed generation and mutation; google/oss-fuzz-gen); defensive vs offensive security, PoC exploits, LLM-assisted offensive security	Alshahwan et al., FSE 2024; [GitHub] ; [anthropic.com] (both cited in slides)	[PDF]	SAFE CODE APP
19	**PL Kaleidoscope V: Accelerated Computing**	AI-discovered algorithms (AlphaDev, AlphaEvolve); what makes a GPU implementation "equivalent", "safe" and "performs better"; CUDA softmax demo; kernel design space (memory access pattern, data structure, dispatch); LLM-written GPU kernels and their benchmarks (KernelBench, KernelBench-X); FlashAttention and FlashAttention-4; LLM and agentic porting of legacy scientific codes (a weather simulation, the FESOM2 ocean model, Fortran to C++/Kokkos)	arXiv 2502.10517 (KernelBench), 2205.14135 (FlashAttention), 2506.13131 (AlphaEvolve), 2608.16884 (AlphaEvolve and the matrix-multiplication exponent); also AlphaDev, KernelBench-X, FlashAttention-4 (MLSys 2026), Hoshino et al. 2026, Koldunov et al. 2026, Gupta et al. 2026 (cited in slides)	[PDF] ; demo [machine-programming.github.io]	APP CODE

#	Title	Key topics	Readings	Materials	Tags
26	**PL Kaleidoscope VI: Visual Programming**	Visual programming	–	–	APP
27	**Special Topics on Neurosymbolic Programming**	Neurosymbolic programming (e.g., Scallop)	–	–	RSN APP

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
A1: Inductive Synthesis	A bottom-up enumerative synthesizer for a geometric-shape DSL (grow / eliminate observational equivalents / search); your own string DSL and synthesizer; then prompts plus output parsing so Claude Sonnet 4.6 (via Bedrock) solves at least 80% of string tasks	Enumerative search, pruning by observational equivalence, DSL design, prompt design, parsing LLM output	[GitHub]	RSN FND CODE
A2: Evaluating Coding LLMs	Execution harnesses for Python/Rust/OCaml; 4 prompting strategies (zero-shot, two-step CoT, iterative refinement with feedback up to 3 rounds, a custom method of about 5 calls or fewer); pass@1/2/3 pipeline over 24 experiments (Claude Haiku vs Sonnet x 3 languages x 4 strategies) on a 16-item xCodeEval/MBPP set; plots	Sandboxed execution with timeouts, error classification, pass@k, experiment scripting, visualization	[GitHub]	EVAL CODE RSN
A3: Coding LLM Agents	"WAA" (Web-App Agent): a Python tool-calling loop (tool registry, system prompt, LLM query, parse, execute, terminate or hit the turn limit), file-system tools with path validation and protected files, TODO tools; the agent then builds a chat room, a Tetris variant in three milestones (mechanics, rules/scoring, leaderboard) and a creative app, plus a report; runs on Bedrock Claude Sonnet 4.6, max_turns 75; an optional extension adds a database plus DB tools	Agent loop design, tool and termination protocols, secure file tools, error recovery, mock-LLM testing, Jest/Supertest/Playwright	[GitHub]	ARCH TOOL CODE

CU Boulder: CSCI 7000-005 Building AI Agents

Institution	University of Colorado Boulder (Danny Dig)
Term	Spring 2026
Website	https://danny.cs.colorado.edu/courses/csci7000-005_Sp26/index.html
Level	Graduate seminar; competent SE assumed
Archetype	Research seminar (agentic software engineering)
Public materials	Slides: 4 instructor decks; video: none; code: none
Best for	Research reading on agentic software engineering: refactoring agents, SWE-agent, skills, memory, MCP design choices.

A seminar with two papers per meeting. Its reading list leans toward agentic software engineering, from agent surveys and an agentic-SE roadmap through refactoring agents and the classic patterns (ReAct, Reflexion, Toolformer, Gorilla, PAL, AutoGen, MetaGPT, debate) to 2025–2026 work on skills, memory, safety, coding-agent configuration and MCP design. The "tool experience report" (run a paper's tool on your own case and critique it) is a good habit to borrow.

Lectures

#	Title	Key topics	Readings	Materials	Tags
1	**Course introduction (Dig)**	How to read research papers	Griswold, "How to Read an Engineering Research Paper" [cseweb.ucsd.edu]	[PDF]	ORG
2	**Survey of agents (Dig)**	LLM agent surveys: general and SE	Wang et al., LLM-based autonomous agents survey [link.springer.com] ; Liu et al., LLM-Based Agents for SE survey [arXiv:2409.02977]	[PDF]	ARCH CODE
3	**Agentic SE roadmap (Dig)**	Foundational pillars and research roadmap for agentic SE	Hassan et al., Agentic Software Engineering [arXiv:2509.06216]	[PDF]	CODE ARCH
4	**Refactoring agents (students)**	Multi-agent rename refactoring; refactoring benchmark	Bellur et al., Multi-Agent Coordinated Rename Refactoring [arXiv:2601.00482] ; Gautam et al., RefactorBench [openreview.net]	–	CODE MULTI EVAL
5	**MM-Assist & MANTRA (Dig)**	LLM-assisted refactoring (move method); multi-agent refactoring	Bellur et al., Together We Are Better [PDF] ; Xu et al., MANTRA [arXiv:2503.14340]	[PDF]	CODE MULTI
9	**ReAct & Web API code gen (students)**	Reasoning+acting loop; API integration code benchmark	Yao et al., ReAct [arXiv:2210.03629] ; Maninger et al., Benchmarking Web API Integration Code Generation [arXiv:2509.20172]	–	RSN TOOL EVAL
11	**Reflexion & Toolformer (students)**	Verbal self-reflection; self-taught tool use	Shinn et al., Reflexion [arXiv:2303.11366] ; Schick et al., Toolformer [arXiv:2302.04761]	–	RSN TOOL TRAIN
12	**Gorilla & PAL (students)**	API-call LLMs; program-aided reasoning	Patil et al., Gorilla [arXiv:2305.15334] ; Gao et al., PAL [arXiv:2211.10435]	–	TOOL RSN
13	**SWE-agent & AutoCodeRover (students)**	Agent-computer interfaces; repo-level program repair	Yang et al., SWE-agent [arXiv:2405.15793] ; Zhang et al., AutoCodeRover [arXiv:2404.05427]	–	CODE ARCH
14	**AutoGen & MetaGPT (students)**	Multi-agent conversation frameworks; SOP-driven role agents	Wu et al., AutoGen [arXiv:2308.08155] ; Hong et al., MetaGPT [arXiv:2308.00352]	–	MULTI ARCH
15	**Multiagent debate & Agentless (students)**	Debate for factuality/reasoning; demystifying SE agents	Du et al., Multiagent Debate [openreview.net] ; Xia et al., Demystifying LLM-Based SE Agents [dl.acm.org]	–	MULTI RSN CODE
16	**LangGraph tutorial & OpenHands**	Hands-on LangGraph; open coding-agent platform; tool experience report	Wang et al., OpenHands [arXiv:2407.16741]	–	ARCH CODE

#	Title	Key topics	Readings	Materials	Tags
18	**RefAgent & Agentic Skills (students)**	Multi-agent refactoring; SoK on agent skills	Oueslati et al., RefAgent [arXiv:2511.03153] ; Jiang et al., SoK: Agentic Skills [arXiv:2602.20867]	–	CODE ARCH TOOL
19	**AppAgent & coding-agent configuration (students)**	Smartphone GUI agent; config files of AI coding agents	Zhang et al., AppAgent [arXiv:2312.13771] ; Santos et al., Decoding the Configuration of AI Coding Agents [arXiv:2511.09268]	–	WEB CODE
21	**MemSkill & More with Less (students)**	Memory skills for agents; efficiency	Zhang et al., MemSkill [arXiv:2602.02474] ; Gao & Peng, More with Less [arXiv:2510.16786]	–	MEM DEPLOY
22	**MetaClaw & code smells (students)**	MetaClaw; causal analysis of smells in LLM code	Xia et al., MetaClaw [arXiv:2603.17187] ; Velasco et al., Causal Perspective on Smells in LLM-Generated Code [arXiv:2511.15817]	–	CODE EVAL
23	**SafePro & ToolDreamer (students)**	Agent safety; tool retrieval/selection	Zhou et al., SafePro [arXiv:2601.06663] ; Sengupta et al., ToolDreamer [arXiv:2510.19791]	–	SAFE TOOL
24	**Swiss Cheese safety & MCP design (students)**	Layered AI-safety model; tool orchestration vs code execution in MCP	Shamsujjoha et al., Swiss Cheese Model for AI Safety [arXiv:2408.02205] ; Felendler et al., From Tool Orchestration to Code Execution: MCP Design Choices [arXiv:2602.15945]	–	SAFE TOOL
25	**Guest talk: Ameya Ketkar (Uber), building AI agents in industry**	Industry agents; multi-agent SRE incident analysis	Kataria, Intelligent Site Reliability Engineering [researchgate.net]	–	DEPLOY MULTI APP

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
Tool Experience Report	Download the tool a paper introduces, run it on your own use case, and write a one-page reflection (experience, strengths, weaknesses, practical impact, recommendations)	Hands-on evaluation of agent tools	–	EVAL CODE

Mannheim: IE 685/686 Large Language Models and Agents

Institution	Universität Mannheim (DWS) (Ralph Peeters, Christian Bizer, Aaron Steiner)
Term	Spring 2026
Website	https://www.uni-mannheim.de/dws/teaching/course-details/courses-for-master-candidates/ie-685-large-language-models-and-agents/
Level	Master's; Python + basic ML
Archetype	Applied master's course + team project
Public materials	Slides: 8 PDF decks; video: none; code: 5 exercise notebooks (zip)
Best for	Compact slide decks from LLM basics to context engineering, plus nine well-specified team-project briefs.

A short lecture block (introduction, post-training, prompting, agents and tool use with MCP, RAG, context engineering with deep agents, safety), each with an exercise notebook, followed by a team project. The nine project briefs (multi-agent gaming, QA agents, job hunting, a shopping assistant, a tutor, a personal assistant, news-reading personas, Wikipedia generation, skill mining) are ready-made portfolio ideas, and each comes with suggested benchmarks.

Lectures

#	Title	Key topics	Readings	Materials	Tags
1	**Introduction (to LLMs)**	Introduction to LLMs, course organization	Zhao et al., A Survey of LLMs [arXiv:2303.18223] ; Zhou et al., Survey on Pretrained Foundation Models [link.springer.com]	Slides [PDF]	FND ORG
2	**Post-training and Efficient Adaptation**	Post-training (instruction tuning / alignment), parameter-efficient adaptation	–	Slides [PDF] ; exercise [notebook zip]	TRAIN FND
3	**Prompt Engineering**	Prompting techniques	–	Slides [PDF] ; exercise [notebook zip]	FND RSN
4	**LLM Agents and Tool Use**	LLM-based agents, tool use and environment interaction, MCP	Wang et al., Survey on LLM-based Autonomous Agents [link.springer.com]	Slides (78 pp.) [PDF] ; exercise [notebook zip]	ARCH TOOL
5	**Retrieval Augmented Generation**	RAG	–	Slides [PDF] ; exercise [zip]	MEM
6	**Context Engineering**	Context engineering for LLM agents; deep agents	–	Slides [PDF] ; exercise [zip]	MEM ARCH
7	**Introduction to Student Projects (IE686)**	Project goals, procedure, 9 topic areas + own idea, schedule	–	Slides [PDF]	ORG
9	**LLM Security and Safety (+ project coaching)**	Safety and security of LLM agents	Wang et al., Full Stack Safety survey [arXiv:2504.15585]	Slides [PDF]	SAFE
x1	**Evaluation of LLMs and LLM agents**	Listed as syllabus topic 8	Mohammadi et al., Evaluation and benchmarking of LLM agents (KDD 2025) [dl.acm.org]	–	EVAL

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
Exercise: LLM Introduction	Intro notebook for working with LLMs	LLM APIs/basics	[notebook zip]	FND
Exercise: LangChain Introduction	LangChain basics notebook	LangChain	[notebook zip]	FND ARCH
Exercise: Agent Programming with LangChain + MCP	Tool-using agents with LangChain and MCP	Agents, tool calling, MCP	[notebook zip]	TOOL ARCH

Name	What you build	Skills	Starter code / resources	Tags
Exercise: RAG	RAG pipeline notebook	Retrieval, RAG	[zip]	MEM
Exercise: Deep Agent	Deep-agent / context-engineering exercise	Context engineering, planning agents	[zip]	MEM ARCH
Project topic 1: Multi-Agent Gaming	Agent teams that cooperate in, or compete at, games (Codenames worked well before)	Multi-agent coordination, eval by win rate/ELO	AgentBench [GitHub] ; SmartPlay [GitHub]	MULTI EVAL
Project topic 2: QA Agents	RAG (multi-)agent QA over web search that explains its evidence	RAG, web search, benchmark eval	BrowseComp [openai.com] ; Meta CRAG KDD Cup 2024 [aicrowd.com]	MEM WEB EVAL
Project topic 3: Job Hunting Agents	From a query plus CV, search job APIs, extract facts, present structured results	Tool/API use, extraction	[apislist.com]	TOOL APP
Project topic 4: Online Shopping Assistant	Product search/recommendation agent with structured feature comparison	Web agents, benchmarks	AgentBench ; WebArena [webarena.dev] ; WebShop [webshop-pnlp.github.io]	WEB APP EVAL
Project topic 5: Personal Tutor	Agent that explains concepts, finds knowledge gaps, tracks progress	Memory, personalization	–	APP MEM
Project topic 6: Personal Assistant	Agent that manages calendar/mail/chat	Tool use, personal-data integration	–	TOOL APP
Project topic 7: Personas Reading the News	Persona agents browsing political news; study effects on memory/beliefs	Agent simulation, memory	Personas provided by instructors (not public)	MULTI MEM APP
Project topic 8: Wikipedia Article Generation	RAG/agent system to generate Wikipedia articles, evaluated against real articles with rubrics	RAG, LLM-as-judge	WikiGenBench, WikiAutoGen (named, no URLs)	MEM EVAL APP
Project topic 9: Skill-Mining from Public Data	RAG system that summarizes developer skills/roles from GitHub profiles	RAG, extraction	GitHub profiles	MEM APP

UChicago: AI Agents for Social Science and Society

Institution	University of Chicago (KnowledgeLab) (James A. Evans)
Term	Winter 2026
Website	https://github.com/KnowledgeLab/AI-Agents-for-Social-Science-and-Society-2026
Level	Grad / advanced undergrad (computational social science)
Archetype	Applied: agents as research instruments
Public materials	Slides: none; video: none; code: 9 weekly notebooks
Best for	Agents as simulated subjects and research assistants; notebooks on multi-agent simulation, LoRA, RL, steering and red-teaming.

Treats LLM agents as tools for social science: simulated subjects and "digital doubles", research assistants and automated scientists. Agent-engineering content is concentrated in week 3 (prompted multi-agent simulation with AutoGen and Concordia, RAG) and week 5 (LoRA fine-tuning, benchmarking, tools and MCP). Other weeks cover deep-learning foundations, causal designs, interpretability and steering, RL, multimodal agents, and alignment. Two notebooks (weeks 6 and 8) were too large to inspect.

Lectures

#	Title	Key topics	Readings	Materials	Tags
0	**Introduction / Review**	Python, math review, Claude Code setup	Python tutorial [docs.python.org] ; Yang et al. "SWE-agent" (2024); *Deep Learning* ch. 2-3; Claude Code setup tutorial (Canvas)	–	ORG CODE
1	**Deep Learning and Social Agents**	Deep vs. shallow nets; links between DL, statistical learning, and social-science agent models	Evans *Thinking with Deep Learning* (preface, ch. 1, 3); Cheng et al. "Exploring LLM-Based Intelligent Agents"; Bubeck et al. "Sparks of AGI"; Farrell, Gopnik, Shalizi, Evans "Large AI Models are Cultural and Social Technologies" (Science 2025); Park et al. "Generative Agents" (UIST 2023)	Notebook [GitHub]	FND APP
2	**Text Learning, Transformers, and Diffusion Models**	Word2vec to transformers to diffusion; CoT, ToT, graph-of-thought	Vaswani et al. "Attention Is All You Need"; Jurafsky & Martin ch. 10; Lu et al. "Cultural tendencies in generative AI"; Zhang & Evans "LM Perplexity Predicts Scientific Surprise". Supplemental: Wei et al. CoT; Lightman et al. "Let's Verify Step by Step"; Yao et al. "Tree of Thoughts"	Notebook [GitHub]	FND RSN
3	**LLMs Prompted for Multi-Agent Simulation**	LLM digital doubles; in-silico social simulation; RAG	Kozlowski & Evans "Simulating Subjects"; Leibo et al. "From Predictive Pattern Completion to Social and Cultural Norms"; Cao, Pan, Evans; Xie et al. (PNAS 2025); Swanson et al. "The Virtual Lab of AI agents" (Nature 2025). Supplemental: RAG (Lewis et al.), Concordia, AutoGen, AFlow, LangGraph, Faiss, AI Scientist v1/v2, "Why do multi-agent LLM systems fail?", Argyle et al. "Out of One, Many"	Notebook [GitHub]	MULTI MEM APP

#	Title	Key topics	Readings	Materials	Tags
4	**Experimental Designs with AI Agent and Human Interactions**	Causal identification; mixed human/AI subjects; persuasion experiments	Broska et al. "Mixed Subjects Design"; Costello et al. "Durably reducing conspiracy beliefs" (Science 2024); Potter et al. "Hidden Persuaders" (EMNLP 2024); Lai et al. "Biased AI improves human decision-making"; Horton "Homo Silicus"; Koch et al. "Deep learning for causal inference"	Notebook [GitHub]	APP EVAL
5	**Sampling, Fine Tuning, Benchmarking, and Tools**	Sampling bias; LoRA/QLoRA; agent benchmarks; tools and MCP	Hu et al. "LoRA"; Mohammadi et al. "Evaluation and benchmarking of LLM agents" (KDD 2025); Anthropic "What is MCP?"; Shumailov et al. "Model collapse" (Nature 2024); Chu et al. "media diets predict public opinion". Supplemental: Kapoor et al. "AI Agents That Matter"; QLoRA; DPO; AgentBench; DSPy; AIOS	Notebook [GitHub]	TRAIN EVAL TOOL
6	**Auto-Encoders, Interpretability, and Steering Agents**	Sparse autoencoders; personas; activation steering	Bricken et al. "Towards Monosemanticity"; Cunningham et al. "Sparse Autoencoders Find Highly Interpretable Features"; Kim, Evans, Schein "Linear representations of political perspective" (ICLR 2025); Chen et al. "Persona vectors". Supplemental: Scaling Monosemanticity, ActAdd, ITI, In-Context Vectors, Transcoders	Notebook [GitHub]	SAFE FND
7	**Reinforcement Learning to Optimize AI Agents and Institutions**	RL basics; RLHF; RL for reasoning; multi-agent RL	Sutton-style RL ch. 1; Géron ch. 18; Mnih et al. DQN (Nature 2015); Bai et al. "Constitutional AI"; Ouyang et al. InstructGPT; Guo et al. DeepSeek-R1 (Nature 2025); Kim et al. "Reasoning Models Generate Societies of Thought"; Zhang, Neubig, Yue (pre/mid-training vs. RL); "Hypothetical Minds"; "RATE". Supplemental: Christiano et al.; Agent Lightning; OpenPipe ART (GRPO)	Notebook [GitHub]	TRAIN RSN MULTI
8	**Multi-Modal and Embodied Agents**	Audio, video, and image agents; diffusion; robot embodiment	Bishop ch. 10, 20 (CNNs, diffusion); Yao et al. "ReAct"; OpenAI GPT-4V system card; Song et al. "Embodied Task Planning with LLMs"; Ludwig & Mullainathan "ML as a Tool for Hypothesis Generation" (QJE 2024); Guilbeault et al. "Age and gender distortion"	Notebook [GitHub]	WEB RSN APP
9	**Alignment, Ethics, Safety, and Novelty**	Value alignment; deception; guardrails; effects on the space of ideas	Russell "Human-Compatible AI"; Hubinger et al. "Sleeper Agents"; Ngo et al. "Alignment Problem from a DL Perspective"; Tomasev et al. "Distributional AGI Safety"; Zou et al. "Representation Engineering"; Gabriel, Keeling, Manzini, Evans "We need a new ethics for a world of AI agents" (Nature 2025). Supplemental: AGrail; Si et al. "Can LLMs Generate Novel Research Ideas?"; Moral Machine	Notebook [GitHub]	SAFE APP

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
Week 1 notebook (Intro NNs)	Feed-forward nets, wide & deep and deep & cross networks on your own dataset; compare activations, dropout, optimizers; error analysis	PyTorch, scikit-learn MLP	[GitHub]	FND
Week 2 notebook	3 of 5 modules: word2vec on your own corpus, seq2seq/attention, fine-tuning a transformer with SHAP, BERT similarity and perplexity, diffusion hyperparameters; memo pilot	gensim, spaCy, HF transformers, shap, umap	[GitHub]	FND
Week 3 notebook	Prompting (0-shot, few-shot, chain-of-reasoning) via the OpenAI API; simulated survey and social-media respondents compared with human data; AutoGen multi-agent dialogues with reflection; Concordia situated simulation; RAG over 5-10 documents vs. a vanilla model	openai, LangChain, AutoGen, Concordia, FAISS/ChromaDB	[GitHub]	MULTI MEM FND
Week 4 notebook	Deep causal inference (S/T-learner, TARNet on IHDP), causal mediation, double/debiased ML, prediction-powered inference on your own question	TensorFlow/Keras, causal ML	[GitHub]	APP EVAL
Week 5 notebook	3 of 4 modules: sampling; LoRA/QLoRA fine-tuning with varied ranks plus CKA comparison; designing an agent benchmark; a tool-using agent and an MCP server	LoRA, benchmarking, MCP	[GitHub]	TRAIN EVAL TOOL
Week 6 notebook	Interpretability and steering (contents not read: file over 10 MB)	–	[GitHub]	SAFE FND
Week 7 notebook (RL)	Bandits (epsilon-greedy, annealing, UCB), RL for alignment, then RL for reasoning or multi-agent RL; memo pilot	gymnasium, stable-baselines3, PPO/DQN/GRPO	[GitHub]	TRAIN MULTI
Week 8 notebook (Multimodal)	Multimodal and embodied agents (contents not read: file over 10 MB)	–	[GitHub]	WEB APP
Week 9 notebook	Any 4 of 7 modules: safety judge, Constitutional AI, deception detection, moral geometry, automated red-teaming, AGrail guardrails	Qwen2.5-7B-Instruct (4-bit), sentence-transformers, AGrail	[GitHub]	SAFE EVAL
Final project	Agent project for social insight: at least 3 data types and 3 model types from 3 weeks, qualitatively validated, with an open repo	End-to-end applied agent research	–	APP ORG

Stanford 329A: CS329A Self-Improving AI Agents

Institution	Stanford University (Aakanksha Chowdhery, Azalia Mirhoseini)
Term	Autumn 2025
Website	https://cs329a.stanford.edu/
Level	Graduate; CS224N or CS229S
Archetype	Research course (self-improvement levers) + homeworks
Public materials	Slides: none linked; video: 9 recordings (Stanford Online); code: HW1–HW3 public repos (HW1 MIT, HW3 Apache-2.0)
Best for	The clearest path through test-time compute, verifiers, RL and open-ended self-improvement, with three public homeworks.

Asks how an agent improves through interaction with itself and its environment, and takes the levers in order: repeated sampling and test-time compute, verifiers and process reward models, learning from tool and execution feedback, planning and tree search, and train-time RL (STaR, GRPO, DAPO). The second half covers open-ended self-improvement (ADAS, AI Scientist, AlphaEvolve), deep-research and software-engineering agents, memory, and long-horizon evaluation, with five guest talks. The homeworks follow the same arc: test-time scaling on AIME25, code verification with LLM judges and synthetic unit tests, then a self-improving agentic workflow. No slide decks are posted. The video links were matched to lectures through a third-party index and then checked on YouTube.

Lectures

#	Title	Key topics	Readings	Materials	Tags
1	**Course Overview**	Framing of self-improving agents	–	[video]	ORG
2	**Test-time Compute Scaling**	Repeated sampling, inference scaling laws, architecture search over inference techniques	Large Language Monkeys; Archon; Scaling LLM Test-Time Compute Optimally (Snell et al.); How Do Large Language Monkeys Get Their Power (Laws)?	[video]	RSN EVAL
3	**Robust Verification**	Outcome/process verifiers, weak verifiers, generation-verification gap	Shrinking the Generation-Verification Gap with Weak Verifiers; Training Verifiers to Solve Math Word Problems; Let's Verify Step by Step; Math-Shepherd	[video]	RSN EVAL
4	**Learning from Feedback with Tools/Code**	Reason+act loops, RL from execution feedback, AI feedback	ReAct; RLEF; Constitutional AI	[video]	TOOL CODE TRAIN
5	**Multi-step Reasoning/Planning**	Multi-step RL with synthetic data, tree search, decomposition, parallel planning	SWiRL; Language Agent Tree Search; SPRINT; ADaPT; Wider or Deeper? (adaptive branching tree search)	[video]	RSN TRAIN
6	**Train Time Scaling/Scaling RL**	Self-taught reasoning, GRPO, large-scale RL recipes	STaR; DeepSeekMath; DAPO	[video]	TRAIN RSN
7	**Open-Ended Evolution of Self-Improving Agents**	Automated agent design, automated science, evolutionary coding agents	Automated Design of Agentic Systems; The AI Scientist; AlphaEvolve	–	ARCH CODE APP
8	**Self-Improvement with Search & Deep Research Agents**	Search-based code generation, agentic search for reasoning models, deep research	AlphaCode; AlphaCode 2 Tech Report; Search-o1	[video]	WEB RSN CODE
9	**Guest: Melvin Johnson (Google DeepMind), Evolution of Post-training from Chatbots to Agents**	Post-training for agents (from title only)	–	–	ORG TRAIN
13	**Agentic Frameworks for Software Engineering**	Test-time scaling for SWE, GPU-kernel generation, agent-system interfaces	CodeMonkeys; KernelBench; Improving Parallel Program Performance with LLM Optimizers via Agent-System Interfaces	–	CODE ARCH

#	Title	Key topics	Readings	Materials	Tags
14	**Guest: Augmenting Agents with Memory (Junchen Jiang, LM-Cache/UChicago)**	Long-context representations, OS-style memory, KV-cache reuse	Cartridges; MemGPT; CacheBlend	–	MEM DEPLOY
15	**Guest: Denny Zhou (Google DeepMind), LLM Reasoning**	LLM reasoning (from title only)	–	–	ORG RSN
16	**Guest: Thang Luong (Google DeepMind), Towards AI Superhuman Reasoning**	Superhuman reasoning (from title only)	–	–	ORG RSN
17	**Agentic Evaluations & Long-Horizon Tasks**	Task-length metrics, economically valuable tasks, research-synthesis benchmarks	Measuring AI Ability to Complete Long Tasks; GDPVal; DeepScholar-Bench	[video]	EVAL
18	**Guest: Misha Laskin (Reflection AI), Building Agentic Systems for Autonomy**	Autonomous agentic systems (from title only)	–	–	ORG ARCH
19	**Guest: Danny Driess (Physical Intelligence), Multimodal AI Agents in Robotics**	Multimodal robot agents (from title only)	–	–	ORG APP
20	**Future Research Areas**	Open problems in self-improving agents (from title only)	–	[video]	FND

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
HW1: Test-time compute scaling	On AIME25 (30 problems, `math-ai/aime25`) with Qwen3-Next-80B-A3B-Instruct via Together AI: zero-shot baseline; `MajorityVoting` over budgets 1-16; best-of-N with an LLM aggregator (`LLMVoting`, generative reward model); an RLEF-style `SelfImprovementSystem` (generate, critique, regenerate); plots plus a cost-per-API-call analysis	Repeated sampling, majority voting, LLM-as-judge, critique-and-refine loops, accuracy-vs-compute analysis	[GitHub]	RSN EVAL
HW2: Code generation and verification	On HumanEval with Llama-3.2-3B-Instruct-Turbo, code run in a Docker sandbox: `calculate_accuracy` and failure analysis; pass@k for k = 3, 6, 9; an `LLMJudge` that picks among candidates via JSON output; an `LLMUnitTestGenerator` (Qwen2.5-72B) that writes 5 tests from the docstring; checks how often ground truth passes the synthetic tests; pass@3 with synthetic vs ground-truth tests	Sandboxed execution, pass@k, LLM-as-judge, synthetic unit tests as weak verifiers, error analysis	[GitHub]	CODE EVAL RSN
HW3: Self-improving agentic workflow	An `APIManager` with Google Custom Search, Polygon stock data, Wolfram Alpha and weather (Nominatim + Open-Meteo) tools; LLM-based `route_query` with Pydantic/JSON mode; a `MultiLMAgent` with query decomposition, multi-model fusion, iterative self-refinement and an adaptive `run_pipeline` (target: 70% or more QA accuracy); a `DeepResearchAgent` that writes a cited 4-5 paragraph report	Tool routing, structured outputs, decomposition, ensembling/fusion, self-refinement loops, deep-research agents, LLM-judged QA eval	[GitHub]	TOOL WEB ARCH EVAL

Berkeley '25: CS194/294-196 Agentic AI

Institution	UC Berkeley (RDI) (Dawn Song (with guest speakers))
Term	Fall 2025
Website	https://rdi.berkeley.edu/agentic-ai/f25
Level	Undergraduate and graduate; ML/DL background
Archetype	Guest-lecture series + AgentBeats project (also a MOOC)
Public materials	Slides: 9 of 13 sessions; video: 12 of 13 sessions; code: AgentBeats tutorial repo (no license stated)
Best for	Industry and lab speakers on post-training, agent evaluation and deployment, and an evaluator-agent project (AgentBeats).

A guest-speaker series (OpenAI, NVIDIA, Microsoft, Meta, Sierra, Google DeepMind and others) bracketed by Dawn Song's introduction and closing lecture on agent safety and security. Recurring themes are post-training for verifiable agentic tasks, agent evaluation and its statistics, multi-agent systems, and lessons from production deployment. The project is the distinctive part. In phase 1 you build a *green* agent, an evaluator that turns a benchmark into tasks, an environment and scoring. In phase 2 you build the agent under test, called *white* in the course and *purple* in the current AgentBeats competition. The same lectures ran as a MOOC at agenticai-learning.org/f25.

Lectures

#	Title	Key topics	Readings	Materials	Tags
1	**Introduction**	Speaker: Dawn Song (UC Berkeley). Course intro and logistics	–	slides	ORG FND
2	**LLM Agents Overview**	Speaker: Yann Dubois (OpenAI). LLM foundations and agent overview	Kimi K2: Open Agentic Intelligence; DeepSeek-V3 Technical Report	slides video	FND TRAIN
3	**Evolution of system designs from an AI engineer perspective**	Speaker: Yangqing Jia (NVIDIA). AI system and infrastructure design	–	slides video	ARCH DEPLOY
4	**Post-Training Verifiable Agents**	Speaker: Jiantao Jiao (NVIDIA). Post-training agents on verifiable tasks	Introducing SWE-bench Verified; BrowseComp	slides video	TRAIN EVAL CODE WEB
5	**Agent Evaluation & Project Overview**	Speaker: n/a (not listed). Agent evaluation; AgentBeats project overview	Survey on Evaluation of LLM-based Agents	slides video	EVAL ORG
6	**Some Challenges and Lessons from Training Agentic Models**	Speaker: Weizhu Chen (Microsoft). Training agentic models	–	slides video	TRAIN
7	**Multi-Agent AI**	Speaker: Noam Brown (OpenAI). Multi-agent AI	–	slides video	MULTI RSN
8	**Predictable Noise in LLM**	Speaker: Sida Wang (Meta). Noise and error bars in LLM evals	Adding Error Bars to Evals	slides video	EVAL
9	**AI Agents to Automate Scientific Discoveries**	Speaker: James Zou (Stanford). Agents for science (Virtual Lab, Paper2Agent)	The Virtual Lab of AI agents designs new SARS-CoV-2 nanobodies; Paper2Agent	video	APP MULTI
10	**Practical Lessons from Deploying Real-World AI Agents**	Speaker: Clay Bavor (Sierra). Production customer-facing agents; simulation testing	τ2-Bench; Sierra Voice Sims blog	slides video	DEPLOY EVAL APP
11	**Multi-Agent Systems in the Era of LLMs**	Speaker: Oriol Vinyals (Google DeepMind). Multi-agent systems with LLMs	–	video	MULTI
12	**Autonomous Agents: Embodiment, Interaction, and Learning**	Speaker: Peter Stone (UT Austin / Sony AI). Embodied agents and RL (Gran Turismo, SLAC)	Outracing Champion Gran Turismo Drivers with Deep RL; SLAC	video	APP TRAIN
13	**Agentic AI Safety & Security**	Speaker: Dawn Song (UC Berkeley). Safety and security of agentic AI	–	video (livestream)	SAFE

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
Project Phase 1: Green agent	An evaluator ("green") agent on AgentBeats that wraps a benchmark or game as an agent. It defines tasks, environment and scoring, receives participant endpoints plus a config over A2A, runs the assessment and returns JSON results as artifacts.	Benchmark design, environment and scoring design, A2A agent servers, Docker packaging, reproducible evaluation	[GitHub]	EVAL MULTI ARCH
Project Phase 2: White agent	An agent under test ("white"; "purple" in current AgentBeats docs) that competes on green-agent benchmarks.	Agent design, tool use, A2A interoperability, performance on evaluator agents	[GitHub]	ARCH TOOL EVAL

Berkeley Adv.: CS294/194-280 Advanced Large Language Model Agents

Institution	UC Berkeley (RDI) (Dawn Song, Xinyun Chen, Kaiyu Yang)
Term	Spring 2025
Website	https://rdi.berkeley.edu/adv-llm-agents/sp25
Level	Upper-division undergraduate and graduate
Archetype	Guest-lecture series + Lean labs (also a MOOC)
Public materials	Slides: All 12 lectures; video: All 12 lectures; code: 2 Lean labs in a public Drive folder (no license)
Best for	Research talks on reasoning, post-training, coding and web agents and formal math, plus a compiler-verified agent lab.

Picks up where Berkeley's Fall 2024 LLM Agents course left off: inference-time reasoning, iterative preference optimization and open post-training recipes (Tulu 3), language-agent memory and planning, coding agents and vulnerability detection, and multimodal web and computer-use agents. A four-lecture block on formal mathematics (AlphaProof, autoformalization, theorem proving, abstraction discovery) is followed by a closing lecture on agent safety and security. Lab 2 is the main draw for agent builders: a verify-and-retry agent loop in which the Lean compiler is the verifier, and grading rejects `sorry`. Lab 1, writing two Lean 4 problems with formal specifications, is dataset work rather than an agent task.

Lectures

#	Title	Key topics	Readings	Materials	Tags
1	**Inference-Time Techniques for LLM Reasoning**	Speaker: Xinyun Chen (Google DeepMind). Inference-time reasoning, self-correction, self-debugging; course intro deck	LLMs as Optimizers [arXiv:2309.03409] ; LLMs Cannot Self-Correct Reasoning Yet [arXiv:2310.01798] ; Teaching LLMs to Self-Debug [arXiv:2304.05128]	slides intro video	RSN ORG
2	**Learning to reason with LLMs**	Speaker: Jason Weston (Meta). Reasoning via preference optimization and self-verification	DPO [arXiv:2305.18290] ; Iterative Reasoning Preference Optimization [arXiv:2404.19733] ; Chain-of-Verification [arXiv:2309.11495]	slides video	TRAIN RSN
3	**On Reasoning, Memory, and Planning of Language Agents**	Speaker: Yu Su (Ohio State University). Implicit reasoning, long-term memory, planning with world models	Grokked Transformers [arXiv:2405.15071] ; HippoRAG [arXiv:2405.14831] ; Is Your LLM Secretly a World Model of the Internet? [arXiv:2411.06559]	slides video	RSN MEM ARCH
4	**Open Training Recipes for Reasoning in Language Models**	Speaker: Hanna Hajishirzi (University of Washington). Open post-training (OLMo/Tulu), DPO vs PPO	Tulu 3 [arXiv:2411.15124] ; Unpacking DPO and PPO [arXiv:2406.09279] ; OpenScholar [arXiv:2411.14199]	slides video	TRAIN RSN
5	**Coding Agents and AI for Vulnerability Detection**	Speaker: Charles Sutton (Google DeepMind). Coding agents, interactive tools, vulnerability finding (Big Sleep)	Interactive Tools for LM Agents [arXiv:2409.16165] ; From Naptime to Big Sleep [googleprojectzero.blogspot.com]	slides video	CODE TOOL SAFE
6	**Multimodal Autonomous AI Agents**	Speaker: Ruslan Salakhutdinov (CMU/Meta). Web agents, visual web benchmarks, tree search	Mind2Web [arXiv:2306.06070] ; WebArena [arXiv:2307.13854] ; VisualWebArena [jykoh.com] ; Tree Search for LM Agents [jykoh.com]	slides video	WEB EVAL RSN
7	**Multimodal Agents – From Perception to Action**	Speaker: Caiming Xiong (Salesforce AI Research). Computer-use/GUI agents, OS benchmarks	OSWorld [arXiv:2404.07972] ; AGUVIS [arXiv:2412.04454]	slides video	WEB EVAL

#	Title	Key topics	Readings	Materials	Tags
8	**AlphaProof: when reinforcement learning meets formal mathematics**	Speaker: Thomas Hubert (Google DeepMind). RL plus Lean for IMO-level proofs	IMO silver-medal blog [deepmind.google] ; Mastering Chess and Shogi [arXiv:1712.01815] ; The Future of Mathematics? [video] ; Building the Mathematical Library of the Future [quantamagazine.org]	slides video	RSN TRAIN APP
9	**Language models for autoformalization and theorem proving**	Speaker: Kaiyu Yang (Meta FAIR). Autoformalization, retrieval-augmented Lean proving	LeanDojo [arXiv:2306.15626] ; Autoformalization with LLMs [arXiv:2205.12615] ; Autoformalizing Euclidean Geometry [arXiv:2405.17216]	slides video	RSN APP TOOL
10	**Bridging Informal and Formal Mathematical Reasoning (Berkeley page: "Advanced topics in theorem proving")**	Speaker: Sean Welleck (CMU). Informal-to-formal proving, context, self-taught Lean reasoning	Draft, Sketch, and Prove [arXiv:2210.12283] ; miniCTX [arXiv:2408.03350] ; Lean-STaR [arXiv:2407.10040] ; ImProver [arXiv:2410.04753]	slides video	RSN APP
11	**Abstraction and Discovery with LLM Agents**	Speaker: Swarat Chaudhuri (UT Austin). In-context theorem-proving agents, learned concept libraries	An In-Context Learning Agent for Formal Theorem-Proving [arXiv:2310.04353] ; Symbolic Regression with a Learned Concept Library [arXiv:2409.09359]	slides video	RSN ARCH APP
12	**Towards building safe and secure agentic AI**	Speaker: Dawn Song (UC Berkeley). Agent security: prompt-injection detection, memory/RAG poisoning, privilege control	Privtrans [PDF] ; DataSentinel [arXiv:2504.11358] ; AgentPoison [arXiv:2407.12784] ; Progent [arXiv:2504.11703]	slides video	SAFE DEPLOY

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
Lab 1: Dataset contribution (Lean)	2 verifiable coding problems, each with `description.txt`, `signature.json`, `test.json` (at least 5 tests with expected and "unexpected" values) and a `task.lean` holding a `_code` implementation and `_spec` theorems between fixed CODE/SPEC markers; implementations should be non-trivial (about 30+ lines) and the specs should fully constrain behavior	Lean 4 functional programming and tactic proofs (induction, simp, rw, cases), writing formal specs, test design	[drive.google.com] (instructions); [drive.google.com] (lab1-datapoint.zip, not opened)	CODE RSN EVAL
Lab 2: Verified Lean coding agent	`main_workflow` in `src/main.py`: from a task description and Lean template, return `"code"` and `"proof"`; suggested planning / generation / verification agents with Lean-error feedback loops and optional RAG over `documents/` (`src/embedding_db.py`); graded by compiling the code with the proof as `sorry`, then with the proof; `sorry`, trivial theorems and hard-coding get no credit; submit with `make zip`, plus an optional 2-page design PDF	Agent loop design, compiler-in-the-loop verification, retry/feedback, RAG, multi-agent decomposition, Lean proofs	[drive.google.com] (instructions); [drive.google.com] (lab2-starter-code.zip, not opened)	ARCH CODE RSN MULTI MEM

Stanford 329T: CS329T Trustworthy ML: Building and Evaluating Agentic Systems

Institution	Stanford University (Anupam Datta, John Mitchell)
Term	Fall 2025
Website	https://web.stanford.edu/class/cs329t/syllabus.html
Level	Graduate; CS229 + deep learning
Archetype	Project course on agent evaluation
Public materials	Slides: 11 decks (10 sessions); video: none (not recorded); code: HW1 handout PDF; starter branch 404
Best for	Evaluation-first agent building: Goal-Plan-Action evaluation, LLM judges, MCP tool descriptions, GEPA and red-teaming.

A project course on building agentic applications and then evaluating them so they can be improved step by step. After LLM background, prompting and RAG, it introduces its main evaluation idea, checking whether an agent's goals, plans and actions line up (*Agent GPA*). Guest talks cover systematic agent evaluation and refining LLM judges, MCP and writing tool descriptions, data curation and RL for agents, GEPA, and agent security. The one homework traces a LangGraph multi-agent research system in TruLens, has you label its failure modes by hand, and compares your labels with an LLM judge's. The starter branch linked from the handout no longer exists; an updated version of the notebook is on the TruLens main branch.

Lectures

#	Title	Key topics	Readings	Materials	Tags
1	**Course introduction and overview**	2023/2024 project examples; LLM background; prompting, fine-tuning, agentic AI	LangGraph agentic concepts [langchain-ai.github.io] ; OpenAI "A practical guide to building agents" [PDF]	slides	ORG FND ARCH
2	**Models, Prompting and RAG**	LLM strengths and limits; prompting techniques; retrieval-augmented generation	[arXiv:2407.12858] ; [arXiv:2406.06608] ; [arXiv:2202.03629] ; supplement: DeepLearning.AI "Building and Evaluating Data Agents" [deeplearning.ai]	slides, HW1	FND MEM
3	**Agentic AI**	Agent foundations; sample agent systems; evaluating goals, plans and actions	Anthropic "Building effective agents" [anthropic.com] ; "What is your Agent's GPA? Evaluating and improving agent trustworthiness" (blog; no URL given)	slides	ARCH EVAL
4	**Recap, further discussion, and planning for projects**	Review of lectures 2-3; project directions	Together "Open Deep Research" cookbook [GitHub]	slides	ORG WEB
7	**Systematic evaluation of agentic systems (guest: Allison Jia)**	Goal/plan/action evaluation; refining the LLM judge; evaluation with and without ground truth	[arXiv:2507.21504] ; [arXiv:2410.10934] ; [arXiv:2510.08847]	slides	EVAL
8	**MCP, effective tool descriptions, evals (guests: Yusuf Ozuysal, Ishita Gupta)**	Model Context Protocol; writing tools for agents	Anthropic MCP announcement [anthropic.com] ; MCP intro [modelcontextprotocol.io] ; Anthropic "Writing tools for agents" [anthropic.com]	slides (Ozuysal), slides (Gupta)	TOOL EVAL
9	**Deep dive into LLMs**	LLM internals and training overview	Andrej Karpathy video [video]	slides	FND TRAIN

#	Title	Key topics	Readings	Materials	Tags
10	**Data curation and RL to optimize agents (guest: Trung Vu, Bespoke Labs)**	Synthetic data; multi-turn tool-use RL; agent environments and benchmarks	Bespoke Labs blog [bespokelabs.ai] ; [arXiv:2504.04736] ; environments: SWE-bench [arXiv:2310.06770] , SWE-smith [arXiv:2504.21798] , ARE [arXiv:2509.17158] (GAIA2 [huggingface.co]), OSWorld [arXiv:2404.07972] , Terminal-Bench [tbench.ai] ; RL: DAPO [arXiv:2503.14476] , ScaleRL [arXiv:2510.13786] ; optional: [arXiv:2506.04178] , [arXiv:2507.01921]	slides	TRAIN TOOL EVAL
11	**GEPA (guest: Lakshya Agrawal)**	Reflective prompt evolution vs. RL	GEPA [arXiv:2507.19457] ; DSPy [dspy.ai] ; Colab [colab.research.google.com] ; ARC-AGI notebook [GitHub]	slides	TRAIN RSN
12	**Security for agents: adversarial attacks and guardrails (guests: Matt Fredrikson, Andy Zou)**	Attacks, defenses and benchmarks for agents	Offense: [arXiv:2307.15043] , [arXiv:2507.20526] ; defense: [arXiv:2310.01405] , [arXiv:2406.04313] ; benchmarks: [arXiv:2402.04249] , [arXiv:2410.09024] , Gray Swan Arena [app.grayswan.ai]	slides	SAFE EVAL
15	**Multi-modal frontier models with agentic capabilities (guest: Deepak Ramachandran, Google DeepMind)**	Multimodal agents	Reading: TBD	n/a (no slides posted)	FND ARCH
18	**Final Project Fair**	2.5-hour session, 3:30-6:00pm: 2-3 minute team openers, then poster and demo session	–	–	ORG APP

Assignments and projects

Name	What you build	Skills	Starter code / resources	Tags
HW1: Measuring and Improving Agent GPA	Work through a provided notebook with a LangGraph multi-agent web-research system (planner and executor nodes plus other agents; Tavily web search, OpenAI models), traced in TruLens. Annotate traces by hand with Goal-Plan-Action failure modes, rerun with LLM-judge metrics (`session.get_leaderboard()`), compare judge scores with your annotations, then change prompts or architecture and rerun. Prerequisite: DeepLearning.AI "Building and Evaluating Data Agents" (about 2 hours). Deliverable: annotation rationale, metrics analysis and justified improvements	LangGraph agent orchestration, web-search tools, tracing, human annotation of traces, LLM-as-judge validation against human labels, failure-mode analysis, prompt/architecture iteration	Linked in the PDF (404 as of 2026-10-09): [GitHub] . A notebook with the same filename exists on main: [GitHub] (added to main 2025-10-30 in PR #2237, after HW1 was due; since changed for LangChain 1.0 (#2279) and the Feedback-to-Metric API migration (#2513), so it is not the version students used. Branch josh/stanford-course no longer exists; checked via GitHub API 2026-10-09)	EVAL MULTI WEB TOOL ARCH

Supplementary Resources

These non-course resources are referenced throughout the modules.

Model Context Protocol docs (version 2026-07-28)

https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro TOOL ARCH

MCP is the open standard for connecting an AI application (the *host*) to external context and tools. The host creates one *client* per *server*. Servers expose *tools* (callable functions with JSON-Schema inputs), *resources* (contextual data) and *prompts* (templates), over *stdio* for local servers or *Streamable HTTP* for remote ones, with OAuth recommended. **Version caveat:** the 2026-07-28 revision makes the protocol stateless. Every request carries its version and capabilities in `_meta`, servers must support a new `server/discover` method that replaces the old initialize handshake (clients may call it, but can also send requests without it), change notifications move to an opt-in `subscriptions/listen` stream, and *sampling* and *logging* are deprecated. Course material written before mid-2026 (including several lectures in this guide) teaches the older stateful handshake. The concepts still carry over, but check the current docs before you write code.

Anthropic Engineering: Code execution with MCP

https://www.anthropic.com/engineering/code-execution-with-mcp TOOL MEM DEPLOY

Argues that agents connected to many MCP servers should call tools *through code* instead of through direct tool calls. Each server is exposed as a file tree of small code APIs. The agent reads only the definitions it needs (progressive disclosure) and writes a script that runs in a sandbox, so intermediate data never passes through the context window. In the worked example, context drops from about 150,000 to 2,000 tokens. The cost is running a secure sandbox. This ties tool use to context engineering and to Skills, and CU Boulder assigns a paper that studies the same design choice (CU Boulder 24).

OpenClaw documentation

https://docs.openclaw.ai/ ARCH DEPLOY SAFE

An MIT-licensed, self-hosted, always-on personal assistant. A single *gateway* process owns sessions, routing and *channels* (Discord, Signal, Slack, WhatsApp, . . .). Agents get tools, cron jobs, hooks, webhooks and on-demand skills from the ClawHub registry. It is a useful reference architecture for long-running, multi-channel agents and a cautionary case study: CMU's safety lecture cites an OpenClaw incident report (CMU 13), and UVA discusses the OpenClaw family in its personal-agents lecture (UVA 18). *Catalogued only.* Under your install policy, vet it before installing anything.

Berkeley BAIR: TinyAgent, Function Calling at the Edge

https://bair.berkeley.edu/blog/2024/05/29/tiny-agent/ TOOL TRAIN DEPLOY

A case study in specializing a *small* model for agentic tool use. GPT-4-Turbo generates synthetic function-calling plans, which are validated programmatically. A 1.1B or 7B model is LoRA-fine-tuned to emit LLMCompiler-style DAG plans over 16 macOS tools. A small DeBERTa classifier replaces embedding retrieval for tool selection, roughly halving the prompt. The model is then quantized to 4-bit for llama.cpp. The 1.1B model (80.4%) beats GPT-4-Turbo (79.1%) on this fixed toolset while running locally. Code: SqueezeAILab/TinyAgent (MIT). Read it after Module 3 (Tool Use and Function Calling) and the SFT lecture in Module 11 (Training and Optimizing Agents).

MIT 6.566 guest lecture: AI Agent Security (Anish Athalye, April 2026)

https://github.com/anishathalye/ai-agent-security-lecture SAFE TOOL ARCH

A single lecture, with a recording and a repository of thirteen small Python scripts, that builds an agent from scratch and then attacks it. The scripts go from raw completion and chat templates, through tools, native tool calling, an agent loop and a CodeAct-style code agent, to a web-summarization prompt-injection demo. They end with two defenses: the dual-LLM pattern and CaMeL, which runs benign and adversarial scenarios with and without CaMeL.

The assigned reading is the CaMeL paper, *Defeating Prompt Injections by Design* (Debenedetti et al., 2025). Some scripts need Ollama with a local model and others need an API key. No license is stated. Run it after Module 3 (Tool Use and Function Calling) and alongside Module 12 (Safety, Security and Oversight). Under your install policy, vet `uv` and the dependencies first.

Anthropic: Building effective agent automations (Martin and Avilla, October 2026)

https://claude.dev/blog/building-effective-agent-automations/ DEPLOY SAFE TOOL

A worked example of an unattended, scheduled agent ("daily-brief", on Claude Managed Agents) that reads sources, tracks what changed and posts a brief to Slack. It is organised as six components: sources, destination, agent, schedule, memory and guardrails. Its rules are the concrete form of the durable-state and idempotency advice in Module 13 (Deployment, Efficiency and Operations). Read each source from a stored bookmark, not a fixed time window. Report a failed read as unreadable, never as a quiet day. Count a post as sent only when Slack confirms it, and only then update the bookmarks and the ledger. Re-read preferences every run from a store the agent cannot edit. Use read-only credentials wherever the agent only reads, and cap each run's spending at three to five times a normal run at first. Posting requires no human approval. Read-only permissions, a host allowlist and spending limits constrain the automation, but malicious source content can still alter the brief. The post gives design reasoning and failure examples but no measurements. Our suggested companion reading: the OpenClaw documentation, for comparison with an always-on architecture.

Notion: How we built a personal data scientist for every Notion employee (David Thomas, October 2026)

https://www.notion.com/blog/how-we-built-a-personal-data-scientist-for-every-notion-employee MEM DEPLOY EVAL APP

A production case study of Data Scout, an internal agent that answers plain-language questions from governed Snowflake data through MCP. Each user signs in with their own credentials, so existing access controls apply. A curated, owned library separates data context (metrics, sources, filters), business context and routing guidance. Queries carry structured metadata so traces show which definitions were used. A nightly review agent records missing, stale or conflicting context; a separate triage agent deduplicates findings, applies authorised fixes, and sends ambiguous or higher-risk changes to a person. The post's main lesson is that a query can run cleanly and still answer the wrong question, through wrong meaning, routing, measurement or interpretation. Notion reports that weekly requests to its data team fell by about 80% within two months and that about half of its employees use the agent. These figures are self-reported, and the post says its evaluation is still maturing: usage and query success do not prove accuracy. Read it with Module 5 (Memory, Retrieval and Context Engineering) for context engineering and Module 13 (Deployment, Efficiency and Operations) for operations.

Jesse Barkley: local agent demos for the CMU executive Agentic AI program (2026)

https://github.com/jabarkle ARCH TOOL MEM RSN MULTI

Five small repositories written as office-hours support for Carnegie Mellon's executive program *Agentic AI: Building Autonomous Systems for Real-World Applications*. They are teaching demos by one instructor, not a course, and they are the only place in this guide where one system grows in stages. Everything runs on a laptop through Ollama with a 3–4B model, with no API key.

- `EmeritusMCP`: the smallest useful MCP example. A 32-line server with one read-only tool, and an agent that answers the same question with and without it. Dependencies are pinned.
- `Local-Agent-Demo`: a "teacher's assistant" agent. The loop is written out by hand, with a cap of three tool steps. Because the model has no native tool calling, each step is a schema-constrained JSON decision followed by a separate answer call, so you can see that a tool call is text the harness parses. Tools live in a separate MCP server that the agent discovers at start-up. Long-term memory is a JSON file searched by embedding similarity, and two `SKILL.md` procedures load only when a request needs them. A trace panel shows what the model sees at each step.
- `Agent-with-Subagent`: the same system plus one subagent that starts with an empty context and receives a written briefing. Its README states the delegation test plainly: big input, small output, no need to remember the middle.
- `Agent-with-subagent-and-langgraph`: the same system again as a LangGraph `StateGraph` with five nodes (recall, decide, act, answer, reflect). Read it beside the hand-written loop to see what the framework adds and

hides.
- `EmeritusAgenticAIStarterCode`: Tree of Thoughts as a 75-line beam search in plain Python, then the same search in LangGraph and in CrewAI. The evaluator is the model scoring its own partial paths, so apply the caution in Module 6 (Reasoning, Planning, Reflection and Search): check that the score carries information before trusting the search.

Caveats. The MCP code uses version 2 of the official Python SDK (`MCPServer`, `Client`) over stdio. The code was read for this guide but not run. No repository states a licence, there are no tests beyond a set-up check, compiled caches and a filled memory file are committed, and three of the five leave dependency versions open (`>=`). Under your install policy, pin and vet the packages before installing. Do them in the order listed, after Module 2 (The Agent Loop and Agent Architectures).

CADSmith: multi-agent CAD generation with programmatic geometric validation (Barkley, Loghmani and Barati Farimani, 2026)

https://arxiv.org/abs/2603.26512 MULTI EVAL APP

A worked example of a fixed multi-agent pipeline whose feedback is a real measurement, not a model's opinion of its own work. A planner writes a design specification, a coder generates CadQuery code with retrieval over the API documentation, an executor runs it in a subprocess and reads exact geometry from the OpenCASCADE kernel, a validator combines those measurements with a vision-language judge that sees three rendered views, and a refiner repairs the code. An inner loop retries execution errors up to three times and an outer loop retries geometric errors up to five. The judge is a stronger model than the coder. The authors report results on their own benchmark of 100 hand-written prompts with hand-written reference scripts, against a single-call baseline and an ablation without vision: mean Chamfer distance falls from 28.37 (single call) to 18.19 (pipeline without vision) to 0.74 (with vision). They also show a part that passed every check with visible gaps. These figures are self-reported and were not reproduced here. Note that it is a workflow in the sense of Module 2 (The Agent Loop and Agent Architectures): the code fixes the order of the roles. Code (no licence stated). Read it with Module 7 (Multi-Agent Systems) for roles and with Module 10 (Evaluating Agents) for the baseline-and-ablation design.

Google Colab

https://colab.research.google.com/ DEPLOY

A hosted Jupyter runtime. Use it to run course notebooks (UChicago, Northeastern, Mannheim, UVA) without a local setup, and to keep experiments with downloaded code off your own machine.

Appendices

Codebase and Tool Index

Repositories, frameworks, benchmarks and tools referenced by each course (as recorded during research).

CMU

- Course GitHub org: [GitHub] (assignment-1, assignment-2, assignment-3, chess-app)
- Modal (sandboxes and compute): [modal.com]
- uv: [docs.astral.sh]
- OpenHands (lecture 2 readings, Frameworks 1): [GitHub] (repo URL from general knowledge, not seen on the course site)
- LangGraph (Frameworks 2): [GitHub] (repo from general knowledge); lecture 15 cites the docs at [docs.langchain.com]
- Monitoring tools cited in lecture 15: LangSmith evaluation [docs.langchain.com]; Docent [transluce.org]; Hodoscope [hodoscope.dev]
- Model Context Protocol spec (lecture 2): [modelcontextprotocol.io] (from general knowledge)
- Agent Skills protocol (A1 reference): URL n/a (named in the A1 handout)
- Harbor (A2 task packaging): URL n/a
- Tinker 0.24.0 and SkyRL Tinker server (A3): URL n/a. SkyRL and Miles are cited as RL-systems examples.
- vLLM / SGLang (served policies, A1 and A3)
- Other harness code cited in lecture 3 readings: Codex, OpenCode, Pi, Hermes Agent (URLs n/a)

Stanford '25

- Assignments repo: [GitHub] (Python 3.12, Poetry, Conda)
- Ollama (local models in Fall 2025 week 1): [ollama.com]
- FastMCP: [GitHub]
- MCP spec: [modelcontextprotocol.io] ; reference servers: [GitHub] ; TypeScript SDK: [GitHub]
- Claude Code docs: [code.claude.com] , skills: [code.claude.com] , settings: [docs.claude.com] ; Messages API: [docs.claude.com]
- Agent Skills standard: [agentskills.io] ; sample skills: [GitHub]
- mitmproxy (week 1, Fall 2026), plus the tutorial "Intercepting Claude Code Requests": [ai.moda]
- HumanLayer advanced context engineering: [GitHub]
- awesome-claude-agents: [GitHub] ; SuperClaude: [GitHub]
- Warp: [warp.dev] ; Semgrep: [semgrep.dev] ; Graphite: [graphite.dev] ; Vercel: [vercel.com] ; bolt.new ; Cursor ; Resolve.ai: [resolve.ai] ; Cognition/Devin: [cognition.ai]
- Fall 2026 partners: Browserbase [browserbase.com] , OpenHands [openhands.dev] , CrewAI [crewai.com] , CopilotKit [copilotkit.ai] , Milvus [milvus.io] , Marimo [marimo.io] , Phoenix [phoenix.arize.com] , Unsloth [unsloth.ai] , Anyscale [anyscale.com] , cmux [cmux.dev] , Pi [pi.dev] , HeyGen [heygen.com]

Michigan

- Aider (open-source AI pair-programming CLI; also the codebase students extend in P2; the course site gives no repo URL)
- Codex (OpenAI), including the Codex app-server, on gpt-6-luna through the U-M GPT Toolkit; Claude Code and Claude in Chrome; the ChatGPT Chrome extension
- Ollama and llama.cpp for local models; qwen3.5:4b and qwen3.5:9b
- Course repos: [GitHub] (slides, demos, EGM skills), [GitHub] (local model setup, MIT), [GitHub] (lecture script to audio), [GitHub] (student project showcase)
- EGM skills (peanuts, elephant, goldfish, egm-implement, mean-review), with walkthroughs and a page on how a skill is put together: [GitHub]
- Named in decks without URLs: Playwright and Playwright MCP, the Excalidraw MCP server, OpenClaw, Hermes Agent, Jev (TypeSafe), RouteLLM, MemGPT, the Open Knowledge Format, Anthropic's "Code execution with MCP" (Nov 4, 2025), Cloudflare's "Code Mode" (Sep 26, 2025)
- Also listed in the course stack: Python with MCP, git and GitHub, pytest, PrairieLearn
- Acknowledged influences: Dan Disler (IndyDevDan), whose Principled AI Coding series shaped Apply; Paul Gauthier (creator of Aider)

Stanford 329Z

- Course repos (MIT): [GitHub], [GitHub], [GitHub] (DSPy, LangGraph and Temporal notebook with GEPA optimization and "tinycal", a small calendar app whose SDK functions are the agent's tools). The site itself is [GitHub]
- DSPy (reading: [arXiv:2310.03714]) and LangGraph: [docs.langchain.com]

- Model Context Protocol specification, rev. 2025-06-18: [modelcontextprotocol.io]
- Agent harnesses: mini-SWE-agent [GitHub], OpenHands software-agent SDK [GitHub], Codex [GitHub], Gemini CLI [GitHub] (the last four as named in the HW1 handout); Pi and Claude Code (no URLs on the site)
- Benchmarks and eval frameworks: Terminal-Bench [tbench.ai], SWE-bench, HumanEval, Harbor framework (used in HW2; no URL in the handout), BFCL, ToolBench, τ-bench
- Proactive agents: OpenClaw [GitHub]
- Other tools named: Temporal (durable orchestration), OpenRouter (model API in HW1 and lecture 5), uv, Docker, sandbox-exec / bubblewrap (HW1 terminal sandbox), Jupyter
- Anthropic engineering posts used as readings: [anthropic.com], [anthropic.com], [anthropic.com], [anthropic.com], [anthropic.com]

Northeastern

- Course repo: [GitHub] (folders DADA1, DADA2, DADA3, W02b, W03a, W03b, W04a, W04b, W05a, W05b, W06a, W06b, W07a, W07b, W08a, W09a, W09b, W10a, completed; requirements.txt; example.env)
- requirements.txt: langchain, langchain-openai, langchain-anthropic, langchain-google-genai, langchain-tavily, langchain-ollama, langgraph, python-dotenv, ipykernel, requests, gradio, langchain-openrouter (W05a adds langchain-pinecone, numpy, optional langchain-nvidia-ai-endpoints; W07a adds arxiv, rank-bm25; W08a adds langgraph-swarm; W09a adds deepagents; W09b adds langchain-daytona and, optionally, the python:3.12-slim Docker image). DADA3 has its own requirements.txt (langchain, langgraph, deepagents, langchain-daytona, the Gemini/Ollama/OpenAI/Anthropic integrations, pydantic, gradio)
- LangChain / LangGraph docs: [docs.langchain.com] ; Deep Agents docs: [docs.langchain.com] ; LangSmith eval/cost docs: [docs.langchain.com]
- Daytona sandboxes: [app.daytona.io] ; Docker Desktop: [docker.com]
- Ollama: [ollama.com] ; Qwen 3.5 models: [ollama.com]
- uv: [astral.sh]
- Pinecone: [pinecone.io]
- Hugging Face Spaces: [huggingface.co]
- Render: [render.com] ; GitHub CLI: [GitHub]
- Model Context Protocol: [modelcontextprotocol.io] (on the schedule for T 11/3 until the 2026-11-06 change; no longer listed)
- APIs used: Gemini (default model, GOOGLE_API_KEY), Tavily, Ticketmaster Discovery, Open-Meteo, OpenRouter, NVIDIA embeddings, Wikipedia search (no key), arXiv, Semantic Scholar (optional S2_API_KEY), Daytona (DAYTONA_API_KEY, W09b and DADA3), Anthropic (Claude Haiku as the W10a judge, optional)

Harvard

- Starter code: archives linked from the handouts, [cs2680.seas.harvard.edu] and [cs2680.seas.harvard.edu] (HTTP 200, gzip, on 2026-10-18; not downloaded), and [cs2680.seas.harvard.edu] (on 2026-10-26 I listed its 33 entries and read its README and `engine/flags.txt`; nothing extracted or run): `assignment4/starter_code/` with `dispatcher/` (dispatcher, egress proxy), `engine/` (`engine.json`, `flags.txt`, `launch.sh`), `evaluation_scripts/` (25 tasks, `run_all.py`, `trace_logger.py`, `verdict.py`), `src/` (`agentic_loop.py`, `sandbox.py`), `madsServe.py` and `init.sh`. The README lists six changes from the Assignment 3 framework: new tasks, every task live at once, the clock starting with the run, your server in place of the course API, the `engine/` folder, and new names. Assignment 4 engine conventions (frameworks, `engine.json`, what `launch.sh` gets): [leaderboard.cs2680.com] . The former course starter repository [GitHub] returned 404 on 2026-10-18 and is no longer in the org's public repo list (as recorded on 2026-10-16: `assignment2/evaluation_scripts`, `assignment3/starter_code` with `dispatcher/`, `src/agentic_loop.py`, `src/sandbox.py`, `evaluation_scripts/`). On 2026-10-16, branch `dev-assignment4` (created Oct 13, not merged to `main`) added `assignment4/starter_code`
- Assignment 1 student gallery: [GitHub] (six implementations; the README says the bundled recordings and demo modes can be explored without Claude Code)
- Student blog and its source: [blog.cs2680.com] ; [GitHub] . Four posts on 2026-10-16, all under Assignment 1, none newer than Oct 9: staff posts "Six interfaces for understanding coding agents", "Lessons for Agentic Coding from Student Projects", "Why Claude Code Kept Reaching for Bash", and one student post ("Controller", a hardware-style terminal for Claude Code). Still the same four posts on 2026-11-11; the repo's Nov 9–10 commits added an Assignment 5 post template and section text
- Claude Code usage-feedback site ("Your Claude Code usage": per the Nov 2 deck, each student's usage, a comparison with classmates and tips; live Oct 28; the page needs JavaScript, so I did not see its content): [how-i-used-claude.cs2680.com]
- Reference notes: agent evaluation and benchmarks [cs2680.seas.harvard.edu] ; cost-effective agents [cs2680.seas.harvard.edu] ; transformers and LLM foundations [cs2680.seas.harvard.edu]
- Computing guides: [cs2680.seas.harvard.edu] (Claude Code, model API, course HPC, AWS)
- SWE-bench Pro evaluator used by Assignments 2 and 3: [GitHub]
- Claude Code headless mode: [code.claude.com]

- Background reading named by the course: OSTEP [pages.cs.wisc.edu] ; Machine Learning Systems [mlsysbook.ai] ; Foundations of Large Language Models [arXiv:2501.09223] ; The Little Book of Deep Learning [fleuret.org]

Buffalo

- Chip Huyen, "Agents": [huyenchip.com]
- Anthropic, "Building Effective Agents": [anthropic.com]
- OpenAI Agents SDK: [openai.github.io]
- LangGraph: [docs.langchain.com]
- LlamaIndex: [docs.llamaindex.ai]
- Also mentioned without URLs: OpenAI function/tool calling docs, Ollama, OpenAI-compatible APIs, vector databases/local vector stores, Jupyter, Git/GitHub
- Books (no URLs): Russell & Norvig, *AI: A Modern Approach*; Caldwell, *The Agentic AI Bible*; Huyen, *AI Engineering* (O'Reilly 2025); Sutton & Barto, *Reinforcement Learning* 2nd ed.
- Research references (no URLs given): ReAct, Toolformer, RAG (Lewis et al.), Generative Agents, SWE-bench, WebArena

UVA

- Hugging Face smolagents: [huggingface.co]
- LangChain / LangGraph: [langchain.com] ; LangGraph Graph API: [docs.langchain.com]
- Model Context Protocol: [modelcontextprotocol.io] ; HF MCP course: [huggingface.co]
- Unsloth: [unsloth.ai]
- Tinker (Thinking Machines): [thinkingmachines.ai] ; install docs [tinker-docs.thinkingmachines.ai]
- Ollama (llava model), Google Colab, OpenAI Platform: [platform.openai.com]
- Henry Kautz repos: gpt_with_tools [gitlab.com] ; MoltReader [GitHub] ; pdf_ocr [GitHub] ; clip_test [GitHub]
- HEPTAPOD: [GitHub] ; Orchestral AI: [orchestral-ai.com]
- OpenClaw [openclaw.ai] ; NullClaw [nullclaw.org] ; NVIDIA NemoClaw [nvidia.com] ; Claude Cowork [anthropic.com]
- Brave Search API [brave.com] ; requests ; beautifulsoup4 ; OpenCV ; gradio ; edge-tts
- Background reading on the home page: Anthropic "Effective context engineering for AI agents" [anthropic.com] ; Chip Huyen "Building LLM applications for production" [huyenchip.com] ; Antonio Gulli "Agent Design Patterns" preview [docs.google.com]
- Additional core papers on the home page: ReAct [arXiv:2210.03629] ; Self-Refine [arXiv:2303.17651] ; Tree of Thoughts [arXiv:2305.10601] ; Generative Agents [arXiv:2304.03442] ; Magentic-One [arXiv:2411.04468] ; ToolOrchestra [arXiv:2511.21689] ; narrow fine-tuning misalignment (Nature 2026) [doi.org]

UW-Madison

- Course demo notebook: [GitHub]
- DSPy (codebase mentioned alongside the DSPy paper): [arXiv:2310.03714]
- litellm (allowed thin wrapper in the assignment); LangChain, Strands, CrewAI (named as disallowed)
- Example coding-agent CLIs: Claude Code, Cursor CLI, Gemini CLI
- Benchmarks/environments in the readings: SWE-bench, Terminal-Bench, AgentDojo, WebArena, BaxBench, RedCode, LiveResearchBench

JHU

- Course site source: [GitHub]
- Assignment repos: [GitHub] , [GitHub] , [GitHub] (all Python, no license listed)
- Amazon Bedrock (Converse API) with Claude Haiku / Claude Sonnet 4.6 (A1, A2); A3's example config now uses Bedrock `us.anthropic.claude-sonnet-4-6` (it showed Google Gemini `gemini-2.5-pro` as of 2026-10-09)
- A3 stack: Node.js/npm, Express, WebSockets (optional), Jest, Supertest, Playwright
- A2 toolchains: Python `subprocess/py_compile`, Rust `cargo`, OCaml `ocaml/ocamlc` (dune optional)
- Benchmarks/harnesses on the readings page: LiveCodeBench [livecodebench.github.io] , SWE-bench [arXiv:2310.06770] , BigCode Eval Harness [GitHub] , BigCodeBench [huggingface.co] , LMArena [lmarena.ai]
- Agent papers on the readings page: MASAI [arXiv:2406.11638] , AutoCodeRover [arXiv:2404.05427]
- Project-idea tools: Scallop [scallop.github.io] , plus Lean, Coq, Z3, CodeQL, Datalog, Prolog, PDDL (Wikipedia links on the project page)
- Lecture supplements: lecture 16's Lean demo (VM.lean, no mathlib) [machine-programming.github.io] ; lecture 19's CUDA softmax demo [machine-programming.github.io]
- Tools named in lectures 17-19 (no links given except where shown): CodeQL, Semgrep, AFL, OSS-Fuzz, Fuzz Introspector, libpng's OSS-Fuzz harness [GitHub] , oss-fuzz-gen [GitHub] , KernelBench-X github.com/BonnieW05/KernelBenchX , CUDA, Kokkos

CU Boulder

- LangGraph (in-class tutorial on 03/09). The site gives no URL.
- OpenHands (paper): [arXiv:2407.16741]
- SWE-agent (paper): [arXiv:2405.15793] ; AutoCodeRover (paper): [arXiv:2404.05427]
- AutoGen (paper): [arXiv:2308.08155] ; MetaGPT (paper): [arXiv:2308.00352]

Mannheim

- LangChain: [langchain.com]
- Tavily (search API): [tavily.com]
- MCP (named in the exercise filename)
- AgentBench: [GitHub]
- SmartPlay: [GitHub]
- WebArena: [webarena.dev] ; WebShop: [webshop-pnlp.github.io]
- BrowseComp: [openai.com] ; Meta CRAG (KDD Cup 2024): [aicrowd.com]

UChicago

- Course repo: [GitHub] (notebooks plus `Tutorials-Homework_Notebooks/data/`)
- Python tutorial: [docs.python.org]
- Named in the syllabus or notebooks without URLs: Claude Code, LangGraph, LangChain, AutoGen, Concordia (DeepMind), Faiss, ChromaDB, DSPy, AIOS, AFlow, Kubeflow, MLflow, OpenPipe ART, Agent Lightning, Neuronpedia, AGrail, PyTorch, Keras/TensorFlow, scikit-learn, gensim, spaCy, Hugging Face transformers, shap, gymnasium, stable-baselines3, Petals (Llama-2), OpenAI API, SWE-agent
- Google Colab (Pro recommended)

Stanford 329A

- Homework repos: [GitHub] (MIT), [GitHub] (no license shown), [GitHub] (Apache-2.0). All Python notebooks plus a course package (`cs329_hw1`, `cs329_hw`, `cs329a_hw3`)
- Together AI API (all homeworks): Qwen3-Next-80B-A3B-Instruct, Llama-3.2-3B-Instruct-Turbo, Qwen2.5-72B-Instruct-Turbo, Meta-Llama-3.1-8B-Instruct-Turbo, Llama-3.3-70B-Instruct-Turbo-Free, gemma-3n-E4B-it, gpt-oss-20B
- Benchmarks/datasets: AIME25 (Hugging Face `math-ai/aime25`), HumanEval ([arXiv:2107.03374]); a QA dataset bundled with HW3 (not described)
- HW3 external APIs: Google Custom Search, Polygon, Wolfram Alpha, Geopy Nominatim, Open-Meteo; Pydantic for structured outputs
- Docker (HW2 `humaneval-sandbox` image), conda, python-dotenv, tenacity, ThreadPoolExecutor
- Key papers with code/systems from readings: Archon [arXiv:2409.15254], MemGPT [arXiv:2310.08560], CacheBlend [arXiv:2405.16444], KernelBench [arXiv:2502.10517], CodeMonkeys [arXiv:2501.14723], DAPO [arXiv:2503.14476], AlphaEvolve [PDF]

Berkeley '25

- AgentBeats tutorial: [GitHub] (examples plus a local assessment runner, `uv run agentbeats-run scenarios/debate/scenario.toml`; scenarios: debate with an LLM-as-judge green agent, and tau2; Python/uv, Docker images for linux/amd64, published to GitHub Container Registry; no license listed). The README points to separate agent-template and green-agent-template repos.
- The current competition page links starter code at [GitHub] and docs at [docs.agentbeats.dev] (not opened)
- AgentBeats platform: [agentbeats.dev] (not opened)
- A2A (Agent-to-Agent) protocol: [a2a-protocol.org] ; A2A SDK or Google ADK for building agents
- LLM providers named in the tutorial: Google Gemini, OpenAI, Anthropic, OpenRouter, Ollama
- Benchmarks in the readings: SWE-bench Verified, BrowseComp, τ2-Bench

Berkeley Adv.

- Lab materials folder (4 files, modified 24 Jun 2025): [drive.google.com] — Lab1_Instructions.md (12 KB), lab1-datapoint.zip (2 KB), Lab2_Instructions.md (10 KB), lab2-starter-code.zip (66 KB)
- Lab FAQ (Google Doc, did not render): [docs.google.com]
- Lab 2 stack, per the instructions: Lean 4 with Mathlib and Aesop, built with Lake (`lake lean Lean4CodeGenerator.lean`); Python harness with `src/agents.py` (GPT-4o for generation, o3-mini for light reasoning), `src/embedding_models.py` (OpenAI and lightweight embeddings), `src/embedding_db.py`; `make test` / `make zip`; approved models only (o1-pro named as not approved); up to 60 s of extended thinking per question
- Lean learning resources named in Lab 1: Functional Programming in Lean, Theorem Proving in Lean 4, the mathlib4 "All

Tactics in Lean 4" list, Natural Number Game (NNG4)
- Problem sources suggested in Lab 1: LiveCodeBench, APPS, MBPP, LeetCode, HackerRank, Codeforces
- Systems and benchmarks in the readings: LeanDojo, WebArena, VisualWebArena, Mind2Web, OSWorld, AGUVIS, Tulu 3/OLMo, HippoRAG, Big Sleep (Project Zero)

Stanford 329T

- TruLens (open-source LLM app tracing and evaluation): [GitHub] . HW1 uses `run_dashboard()`, human feedback on traces, LLM-judge metrics and `session.get_leaderboard()`
- LangGraph: agentic concepts page [langchain-ai.github.io] ; the main-branch notebook pins `langgraph==0.5.4`
- Tavily search API and the OpenAI API (HW1 needs both keys; the handout estimates under $2.00 in OpenAI cost)
- Substitute HW1 notebook (main branch): [GitHub] . It uses `trulens`, `trulens-apps-langgraph`, `trulens-providers-openai`, `langchain_tavily`, `langchain_openai` and `langchain_experimental`, with nodes planner, executor, web_researcher, chart_generator, chart_summarizer and synthesizer. Judge metrics: Plan Quality, Plan Adherence, Execution Efficiency, Logical Consistency, plus the RAG triad (Groundedness, Answer Relevance, Context Relevance)
- DeepLearning.AI short course "Building and Evaluating Data Agents": [deeplearning.ai]
- Model Context Protocol: [modelcontextprotocol.io]
- DSPy [dspy.ai] and GEPA [GitHub]
- Together cookbook, Open Deep Research: [GitHub]
- Agent benchmarks/environments: SWE-bench, SWE-smith, ARE/GAIA2, OSWorld, Terminal-Bench [tbench.ai] , Gray Swan Arena [app.grayswan.ai]

Supplements

- Model Context Protocol docs (version 2026-07-28): https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro
- Anthropic Engineering: Code execution with MCP: https://www.anthropic.com/engineering/code-execution-with-mcp
- OpenClaw documentation: https://docs.openclaw.ai/
- Berkeley BAIR: TinyAgent, Function Calling at the Edge: https://bair.berkeley.edu/blog/2024/05/29/tiny-agent/
- MIT 6.566 guest lecture: AI Agent Security (Anish Athalye, April 2026): https://github.com/anishathalye/ai-agent-security-lecture
- Anthropic: Building effective agent automations (Martin and Avilla, October 2026): https://claude.dev/blog/building-effective-agent-automations/
- Notion: How we built a personal data scientist for every Notion employee (David Thomas, October 2026): https://www.notion.com/blog/how-we-built-a-personal-data-scientist-for-every-notion-employee
- Jesse Barkley: local agent demos for the CMU executive Agentic AI program (2026): https://github.com/jabarkle
- CADSmith: multi-agent CAD generation with programmatic geometric validation (Barkley, Loghmani and Barati Farimani, 2026): https://arxiv.org/abs/2603.26512
- Google Colab: https://colab.research.google.com/
- TinyAgent code: github.com/SqueezeAILab/TinyAgent